MIT 6.172 性能工程（Performance Engineering of Software Systems）学习笔记

课程：MIT 6.172 *Performance Engineering of Software Systems*，Fall 2018，18 学分（18-unit），本科层次。 **讲师**（OCW 课程信息栏）：Prof. Charles Leiserson、Prof. Julian Shun。讲义单页署名的客座/合作者另见各讲：L5、L9、L13 由 Tao B. Schardl 署名；L18 由 Saman Amarasinghe 署名；L19 由 Helen Xu 署名；L21 由 Jon Bentley 署名；L23 由 Steven Johnson 署名（依据各讲义 PDF 元数据 Author 字段）。 **OCW 主页**：https://ocw.mit.edu/courses/6-172-performance-engineering-of-software-systems-fall-2018/ **站内页面**：Syllabus `/pages/syllabus/` | Calendar `/pages/calendar/` | Lecture Slides `/pages/lecture-slides/` | Assignments `/pages/assignments/` | Projects `/pages/projects/` | Quizzes `/pages/quizzes/` | Recitation Problems `/pages/recitation-problems/` | Lecture Videos `/video_galleries/lecture-videos/` **教材**：**OCW 明确写“没有必读教材”**（Syllabus: *Reading materials will be posted on the class website.*；Readings 页: *6.172 has no required readings.*）。Readings 页给的是一份**补充阅读清单**（Cilk++ 平台、Cilk-5 实现、Cilkview、Hoard、SuperMalloc、Cache-Oblivious Algorithms、GraphIt、OpenTuner、Tapir、*Producing Wrong Data Without Doing Anything Obviously Wrong*、CLRS 第 27 章等）。本文写作时另外参照两本常被当作 6.172 伴读的书：Bryant & O'Hallaron *Computer Systems: A Programmer's Perspective*（CSAPP，本目录已有笔记）与 Herlihy & Shavit *The Art of Multiprocessor Programming*——**这两本是本文的补充视角，不是 OCW 规定的教材。** **本文数据来源**：23 份讲义 PDF（`MIT6_172F18_lec1..lec23.pdf`，全部 HTTP 200，`pdftotext -layout` 抽取后逐讲阅读）、10 份作业 PDF（`MIT6_172F18hw1..hw10.pdf`）、4 份项目 PDF（`project1..project4.pdf`）、Leiserchess 规则 PDF、4 份 Practice Quiz 及其 3 份答案 PDF，以及上述 OCW 页面。**文中所有具体数字（运行时间、加速比、缓存缺失数、分配器吞吐、分值比例）均来自这些材料**；凡材料未明说、由笔者换算或推断之处，均标“推断”。 **一处必须纠正的前提**：任务描述里说“6 个 lab”，**与 OCW 不符**。2018 版 6.172 的实际结构是 **10 个每周作业（HW1–HW10）+ 4 个项目（Project 1–4）**，见 `/pages/assignments/` 与 `/pages/projects/`。本文按 OCW 的真实结构写（第 23、24 章），并在第 2.4 节专门说明这个差异，方便你对照其他二手资料时不被带偏。 **访问日期**：所有 URL 于 **2026-11** 访问。 **版权声明**：课程讲义、作业、项目说明、测验的版权归 MIT 与各位作者所有；OCW 页面标注课程材料以 Creative Commons 许可发布，但讲义中**部分图片/图表明确标注“excluded from our Creative Commons license”**（例如 Intel 微架构图、Gelsinger 功耗密度图、部分 Flickr 图片）。因此本文不复述原文段落与图，只做中文重述、结构化整理与工程化解读，并逐节标注来源讲次。

1.1 一句话定位

1.2 先修要求（Syllabus 原文）

1.3 评分结构（Syllabus 页 Grading，逐项抄录）

1.4 课程表结构（Calendar 页）

1.5 参考材料与“读什么”

1.6 六个“这门课真正想让你记住”的结论

2.1 23 讲的完整地图

2.2 知识依赖图

2.3 “工作量 / 存储 / 并行”三条主线的分界
2.4 关于“6 个 lab”的纠正（重要）
3.1 动机：如果你的测量噪声比优化幅度大，你就在做随机游走
3.2 机制：quiescing（把机器“静”下来）
3.3 DVFS 为什么“破坏”测量
3.4 代码与数据对齐：改个文件名就能变慢
3.5 计时器的选择：为什么讲义说“别用 rdtsc”
3.6 测量手段的五分类
3.7 统计量怎么选：最小值、算术平均、几何平均
3.7.1 为什么“最小值”最能代表原始性能
3.7.2 但不同场景要用不同统计量
3.7.3 比值的平均必须用几何平均
3.7.4 两个程序谁更快：head-to-head + P 值
3.8 用最小二乘把“时间”拆成“指令”和“缺失”
3.8.1 数据
3.8.2 模型
3.8.3 建模的三个陷阱
3.9 基本工作流（背下来）
3.10 实测/陷阱：本章的三条实战教训
4.1 为什么用矩阵乘法开场
4.2 机器与峰值
4.3 版本 1–3：语言与执行模型
4.3.1 版本 1：Python 三层循环
4.3.2 版本 2：Java
4.3.3 版本 3：C（clang/LLVM 5.0）
4.3.4 解释器为什么慢（讲义的循环图）
4.4 版本 4：交换循环顺序（+6.50×）
4.4.1 六种顺序的实测
4.4.2 为什么：行主序 + 空间局部性
4.4.3 用 cachegrind 验证
4.5 版本 5：编译选项（再 +3.25×）
4.6 版本 6：并行循环（+17.97×，总 6921×）
4.6.1 只用了 1 个核
4.6.2 并行哪一层？实测三种选择
4.6.3 结果表
4.7 版本 7：分块（tiling，再 +1.70×）
4.7.1 数据重用分析（讲义的两道算术题）
4.7.2 分块代码
4.7.3 块长 s 怎么选：实测
4.7.4 缓存计数器对比
4.8 版本 8：递归分治（再 +1.38×）
4.8.1 分块思想
4.8.2 第一次尝试：50 倍变慢
4.8.3 粗化递归（coarsening）
4.8.4 结果与计数器
4.9 版本 9：编译器自动向量化（再 +1.87×）
4.9.1 让编译器报告向量化
4.9.2 向量化编译开关
4.9.3 结果
4.10 版本 10：手写 AVX intrinsic（再 +1.76×）
4.11 版本 11：Intel MKL（0.41 秒）
4.12 完整 11 版本表（本章核心数据）
4.13 从这张表能学到的五件事
4.14 陷阱与实测提醒
5.1 “work” 的精确定义
5.2 新版 Bentley 规则全表（21 条）
5.3 几个值得展开的例子
5.3.1 Packing / encoding：日期
5.3.2 Caching：一个 30% 的数字
5.3.3 Sparsity：CSR 的第一次出场
5.3.4 Sentinels：把两次判断变成一次
5.3.5 Ordering tests：把便宜的、常成功的放前面
5.3.6 Combining tests：全加器查表
5.3.7 Coarsening recursion：第 4 章已经见过
5.4 讲义的收尾建议（值得当纪律）
6.1 为什么在 2018 年还讲位技巧
6.2 补码的两个恒等式
6.3 基本位操作速查（讲义表格）
6.4 无临时变量交换
6.5 无分支最小值与 popcount
6.5.1 无分支最小值
6.5.2 关于无分支的警告（很重要）
6.5.3 popcount 的三种写法
6.6 de Bruijn 序列求 $\lg$（只对 2 的幂）

6.7 Queens 问题的位向量表示（讲义 + Quiz 1 第 2 题）
6.8 取模与 2 的幂
6.9 向上取整到 2 的幂（讲义的分步图）
6.10 本章的实战提醒
7.1 从 5 级流水线到 Haswell
7.1.1 经典 5 级
7.1.2 现实：停顿
7.1.3 三类冒险（讲义定义）
7.2 复杂运算的延迟（讲义表格，可直接背）
7.3 超纯量 + 乱序 + 重命名
7.3.1 超纯量（superscalar）
7.3.2 乱序（out-of-order）
7.3.3 寄存器重命名
7.4 分支预测
7.4.1 代价
7.4.2 2 位饱和计数器
7.4.3 带历史
7.5 向量硬件的引入（承上启下）
7.6 本章的实战清单
8.1 为什么要看 IR 而不是直接看汇编
8.2 编译流水线
8.3 栈帧与调用约定（讲义要点）
8.4 从 IR 看三个典型优化
8.5 实战：怎么拿到这三层
9.1 编译器是“保守的机械变换器”
9.2 编译器会做的：内联与 LICM
9.2.1 内联
9.2.2 LICM（循环不变量外提）
9.3 编译器做不到的：别名分析
9.3.1 别名分析是不可判定的
9.3.2 案例 1：向量化被“可能重叠”挡住
9.3.3 加 `restrict` 就有单版本
9.3.4 案例 2：`sqrt()` 挡住了 LICM
9.3.5 案例 3：无符号溢出挡住向量化
9.4 诊断失败的三件套
9.5 编译器的“不可能三角”
10.1 向量单元的模型
10.2 向量化宽度怎么算
10.3 对齐：`movdqu` 与 `movdqa`
10.3.1 无 `restrict`：双版本 + 非对齐指令
10.3.2 加 `restrict`：单版本，但仍非对齐
10.3.3 加 `__builtin_assume_aligned`：`movdqa`
10.3.4 三次编译的对比表
10.4 为什么不是所有循环都能向量化
10.4.1 `if` → `max`：一次改写换来 `pmaxub`
10.4.2 浮点归约必须先 `-ffast-math`
10.5 跨步、gather、scatter
10.6 打包更小的数据类型（HW3 的核心实验）
10.7 `#pragma clang loop`
10.8 本章清单：向量化排查顺序
11.1 三种映射方式与地址切分
11.2 缺失的四种类型（讲义定义）
11.3 ideal-cache 模型
11.3.1 LRU 引理
11.3.2 Cache-Miss 引理
11.3.3 Tall-cache 假设
11.3.4 子矩阵引理
11.4 矩阵乘法的缺失分析（三种情形）
11.4.1 冲突缺失的经典例子（讲义那张图）
11.5 分块：$\Theta(n^3/(B\sqrt{M}))$
11.6 多级分块与“巫毒参数”
11.7 递归分治：同一复杂度，零参数
11.7.1 递归式与解
11.7.2 并行版本的缓存行为（重要定理）
11.8 本章自查
12.1 从 stencil 开始：一个“循环写法必然慢”的例子
12.2 时间分块（trapezoidal 递归）
12.3 归并排序的 cache 分析
12.3.1 二元归并
12.3.2 多路归并（R 路）
12.3.3 取最优 R
12.4 Funnelsort：真正的 cache-oblivious 排序
12.5 其他 cache-oblivious 结果（讲义表）
12.6 cache-oblivious 的五个优点（讲义原文）

12.7 讲义对矩阵乘的补充
13.1 为什么需要一致性协议
13.2 MSI 协议
13.3 四类并发平台（讲义分类）
13.4 Fibonacci：第一个并行程序
13.5 本章要点回顾
14.1 计算 DAG
14.2 两个度量与两条定律
14.3 组合规则
14.4 一个手算例子（讲义原图）
14.5 Amdahl 定律 = span law 的特例
14.6 各算法的 work/span/parallelism（讲义表，必背）
14.7 贪心调度定理
14.8 具体算法的 span 分析（讲义案例）
14.9 Cilkscale：从一次串行执行推出并行性
15.1 确定性的定义
15.2 determinacy race 的精确形式
15.2.1 位域陷阱
15.3 哈希表：一个必然出错的例子
15.4 互斥的实现
15.4.1 自旋锁（讲义汇编）
15.4.2 让出锁 vs 自旋锁
15.4.3 竞争锁（competitive mutex）
15.5 死锁
15.5.1 三个必要条件（讲义）
15.5.2 锁序定理（防死锁的 silver bullet）
15.5.3 哲学家就餐
15.5.4 Cilk 特有的死锁
15.6 事务内存（TM）
16.1 顺序一致性（SC）
16.1.1 经典反例（讲义的两处理器例子）
16.2 为什么现代机器不是 SC
16.3 x86-64 TSO（Total Store Order）
16.4 内存栅栏
16.5 用 C11 恢复正确性
16.6 CAS 与无锁互斥
16.6.1 CAS 语义（讲义原文）
16.6.2 用 CAS 实现互斥（$\Theta(1)$ 空间）
16.7 无锁栈：push/pop 与 ABA
16.7.1 push
16.7.2 pop
16.7.3 为什么“先读再 CAS”更快
16.7.4 ABA 问题（讲义的分步示例）
16.7.5 两种解法
16.8 无锁的性质与代价
16.9 HW10 的九个问题（锁 vs 无锁 FIFO）
17.1 问题定义
17.2 四种基本策略
17.2.1 定长 free list 的细节（讲义的分步图）
17.2.2 位图 + 位技巧
17.3 外部碎片与页级管理
17.4 Binned free lists（BFL）
17.4.1 分配算法
17.4.2 BFL 定理
17.4.3 合并（coalescing）
17.5 为什么“用虚拟内存分配、永不释放”不可行
17.6 垃圾回收三种方案
17.6.1 复制式 GC 的三个细节
17.6.2 GC 术语表（讲义）
17.6.3 增量复制的两个经典算法
17.7 本章与作业/项目的对应
18.1 三个策略与它们的致命缺陷
18.1.1 “为什么小块的竞争更严重”
18.2 Hoard：全局 + 本地 + 超级块
18.2.1 结构
18.2.2 分配伪代码（讲义原文）
18.2.3 释放与不变量
18.2.4 Blowup 定理
18.2.5 碎片术语表（讲义）
18.2.6 分配器速度实测（讲义表，32 线程）
18.2.7 jemalloc 的三个差异（讲义）
18.3 伪共享（false sharing）
18.3.1 机制图
18.3.2 三种成因（讲义）

18.3.3 修法

18.4 本章要点

19.1 worker deque

19.2 三条关键性质

19.3 编译器与运行时的分工（讲义表）

19.4 工作窃取为什么“缓存友好”（与 11.7.2 呼应）

20.1 并行排序的三条路线

20.2 图的表示：CSR（第 5 章的回归）

20.3 并行 BFS（自顶向下）

20.3.1 work/span 分析（讲义）

20.3.2 实测（讲义）

20.3.3 确定性版本（讲义的两阶段法）

20.4 方向优化 BFS（direction-optimizing）

20.4.1 观察：frontier 的形状

20.4.2 两种 BFS

20.4.3 切换策略

21.1 三个 DSL 案例

21.1.1 GraphIt 的例子（讲义原文）

21.2 自动调优为什么必要

21.2.1 集成搜索（ensemble）

21.3 自动调优的风险（Project 3 的明文警告）

22.1 Bentley 的 TSP：六个版本的演化

22.1.1 算法 1：朴素枚举

22.1.2 常数因子：编译器

22.1.3 常数因子：硬件

22.1.4 常数因子：内部改动

22.1.5 算法 3/4：减少排列数与增量求和

22.1.6 算法 5：剪枝（本课程最重要的一课）

22.1.7 算法 6：更强的下界（剩余点的 MST）

22.1.8 阶乘有多大（讲义的震撼教育）

22.2 Leiserchess：从“能跑”到“能赢”

22.2.1 游戏规则要点（L19）

22.2.2 引擎结构（L19）

22.2.3 PVS 与剪枝

22.2.4 投机并行（L20）

22.2.5 P4 的评分逻辑

23.1 作业总览

23.2 HW1：工具链与 C 基本功

23.2.1 覆盖的知识点

23.2.2 几个必会的 gdb 操作

23.2.3 覆盖率的一个反直觉用途

23.3 HW2：性能分析与归并排序的五轮优化

23.3.1 工具用法

23.3.2 五轮优化的顺序（这就是“优化优先级”的实证）

23.3.3 递归内联的坑（Write-up 3）

23.4 HW3：向量化的 15 个 write-up

23.5 HW4：Cilk、Reducer 与竞态检测

23.6 HW5：性能工程理论

23.7 HW6：简单存储分配器（P3 的缩小版）

23.7.1 三个任务

23.7.2 mdriver 的用法

23.7.3 OpenTuner 的调用

23.8 HW7：自己写动态分析工具（CSI 框架）

23.8.1 CSI 的四个概念

23.8.2 CSI-cov 的 40 行

23.8.3 线性回归任务

23.9 HW8：cache 复杂度分析

23.10 HW9：确定性执行

23.10.1 三类非确定性来源

23.10.2 哈希表的两个任务

23.11 HW10：锁 vs 无锁 FIFO

24.1 项目总览与共同规则

24.2 Project 1：Bit Hacks

24.2.1 任务

24.2.2 三种算法（讲义原文都给了）

24.2.3 命令行与测试

24.2.4 评分表（讲义原文）

24.2.5 实现顺序建议（推断，但符合 handout 逻辑）

24.3 Project 2：Collision Detection

24.3.1 任务分解

24.3.2 四叉树的关键设计问题

24.3.3 正确性要求

24.3.4 性能要求与命令

24.3.5 评分表

24.4 Project 3：Serial Dynamic Memory Allocation
24.4.1 接口与文件
24.4.2 四条硬规则
24.4.3 评分公式（讲义原文）
24.4.4 mdriver 参数
24.4.5 评分表
24.4.6 实现路线（推断，按 handout 的 hints 排序）
24.5 Project 4：Leiserchess
24.5.1 三个阶段
24.5.2 优化清单（讲义明确提到“low-hanging fruit”）
24.5.3 测试工具
24.5.4 一个反直觉的权衡
25.1 形式（四份 Practice Quiz 首页共同写明）
25.2 四份练习卷的题目分布（可当复习清单）
Quiz 1（14 页，80 分）
Quiz 2（17 页，80 分）
Quiz 3（14 页，80 分）
Quiz 4（13 页，80 分）
25.3 从题目分布推出的复习优先级
25.4 crib sheet 建议（推断，但符合“一张双面”的约束）
26.1 与本目录其他笔记的对应关系
26.2 三组“同一概念、不同抽象层”的对照
26.2.1 分块（blocking / tiling）
26.2.2 并行
26.2.3 性能度量
26.3 建议的交叉阅读顺序
27.1 复习清单（按“能不能手算”分级）
A 级：必须能手算/手写
B 级：必须能解释为什么
C 级：必须能说出名字与结论
27.2 自测题（20 题，附答案）
题目
答案
27.3 三条“如果只记三件事”
28.1 OCW 页面（2026-11 访问，全部 HTTP 200）
28.2 讲义 PDF（23 份，按讲次）
28.3 作业与项目 PDF
28.4 补充阅读（OCW Readings 页列出的 14 条）
28.5 两本常被当作伴读的书（非 OCW 指定教材）
28.6 事实核查说明

第 1 章 课程概览：6.172 到底在教什么

1.1 一句话定位

6.172 是一门**“把程序跑快”当成一门独立工程学科**的课。它不教你写新算法，而是教你：给定一个已经正确的程序，如何**测量**它、**定位**瓶颈、**在五个层次上**（算法工作量 → 存储层次 → 指令级并行/向量 → 线程级并行 → 编译器边界）逐层榨出性能，并且**每一层都用可复现的测量数字说话**。

Syllabus 的官方课程描述（原文照抄，中文要点在后）：

> Modern computing platforms provide unprecedented amounts of raw computational power. But with great power comes great complexity… obtaining good performance requires a comprehensive understanding of all layers of the underlying platform, deep insight into the computation at hand, and the ingenuity and creativity required to obtain an effective mapping of the computation onto the machine.
>
> 6.172 is an 18-unit class that provides a hands-on, project-based introduction to building scalable and high-performance software systems. Topics include performance analysis, algorithmic techniques for high performance, instruction-level optimizations, caching optimizations, parallel programming, and building scalable systems.

把它拆成能力项：

能力	对应讲次	你必须能亲手做的事
测量与建模	L10、L1	用 `perf` / `cachegrind` / `gprof` 定位热点，拟合 `T = a·I + b·C` 并解释残差
减少工作量	L2、L3、L21	用 Bentley 规则和位技巧把指令数砍掉一个量级
理解机器	L4、L5	读 x86-64 汇编，解释流水线停顿、乱序、分支预测失败代价
利用向量	L4、HW3	让 clang 自动向量化，并能在失败时从 `-Rpass` 报告和汇编里找出原因
利用存储层次	L14、L15、HW8	手算 cache 缺失复杂度，写分块与 cache-oblivious 代码
利用多核	L6–L8、L13	用 Cilk 表达并行，算 work/span，判断“再加核有没有用”
保证并发正确	L7、L16、L17	找 determinacy race，写无锁栈，解释 ABA 与内存序
管理内存	L11、L12、P3	写一个吞吐与空间利用率双高的 malloc
知道编译器边界	L9	判断“这段代码编译器会不会帮我做”，不会就自己动手

1.2 先修要求（Syllabus 原文）

> The prerequisites for this class are 6.004 Computation Structures, 6.006 Introduction to Algorithms, and 6.031 Software Construction (formerly 6.005 Software Construction). **The course programming language is C.**

这条先修清单信息量很大，逐条翻译成“你缺什么会卡住”：

先修	实际用到的部分	缺了会在哪里卡
6.004 Computation Structures	流水线、冒险、cache 组织、虚拟内存	L4（乱序/重命名/分支预测）、L14（组相联、冲突缺失）
6.006 Introduction to Algorithms	递归式、摊还分析、图算法、动态规划	L8（Master Method 与 span 递归式）、L22（BFS/PageRank）
6.031 Software Construction	测试、断言、回归、 specs	所有作业的“先正确再快”纪律、Project 的测试覆盖评分
C 语言	指针、`malloc`、位运算、`make`	全课程（没有 C 基本没法做 HW1）

推断：这门课对 C 的要求高于“会写 Hello World”。HW1 直接让你处理 `int*` 与 `char**` 的区别、指针与数组的等价陷阱、按值传参、`const char*` 与 `char* const` 的区别；如果你没被 C 的指针折磨过，第一周会很痛。

1.3 评分结构（Syllabus 页 *Grading*，逐项抄录）

项目	权重	说明
Weekly Homeworks	10%	HW1–HW10，每周一份，多为“照着讲义做实验 + 写 short answer”
Quizzes	30%	两次闭卷 Quiz（Calendar：Week 7 Quiz #1、Week 12 Quiz #2）
Project 1	10%	Bit Hacks（位旋转）
Project 2	12%	Collision Detection（四叉树 + Cilk）
Project 3	12%	Serial Dynamic Memory Allocation（malloc）
Project 4	24%	Leiserchess（博弈引擎）
Class Participation	2%	出勤/参与

四个项目合计 **58%**，是这门课的绝对重心；Quizzes 30% 但只有两次，单次权重 15%。

Syllabus 对评分的原话有一句很关键的限定：

> Each assignment will describe how you will be graded. The scores you receive on each assignment will be combined to produce your final grade after being weighted approximately.

“approximately（大致）”意味着上表是近似权重，**具体以每份 handout 的评分表为准**——事实上每个项目 PDF 里都有一张自己的 Beta/Final 分值表（见第 24 章）。

1.4 课程表结构（Calendar 页）

Calendar 一共 15 周，每周 2 次 1.5 小时讲座 + 1 次 recitation。把它压成一张“节奏表”：

周	讲座	Recitation	关键节点
1	L1 矩阵乘法的 11 个版本	R1 Basic Tools	HW1 布置
2	L2 Bentley 规则、L3 位技巧	R2 Profiling	HW2 布置；P1 Beta 布置
3	L4 体系结构与向量化、L5 C 到汇编	无	HW3 布置；P1 Beta 报告截止；P1 Final 布置
4	L6 多核编程、L7 竞态与并行	R3 Cilk and Reducers	HW4 布置；P1 MITPOSSE 截止
5	L8 多线程算法分析、L9 编译器能做什么	R4 多线程程序分析	HW5 布置；P1 Final 截止；P2 Beta 布置
6	L10 测量与计时	R5 Quiz 1 复习	P2 Beta 截止；P2 Final 布置
7	L11 存储分配	R6 简单内存分配	**Quiz #1**；HW6 布置；P2 MITPOSSE 截止
8	L12 并行存储分配、L13 Cilk 运行时	R7 编译器插桩	HW7 布置；P2 Final 截止；P3 Beta 布置
9	L14 缓存与 cache-efficient 算法、L15 cache-oblivious	R8 Caching and Cachegrind	HW8 布置；P3 Beta 截止；P3 Final 布置
10	L16 非确定性并行、L17 无锁同步	R9 确定性执行	HW9 布置；P3 MITPOSSE 截止
11	L18 DSL 与自动调优、L19 Leiserchess 代码走读	R10 Quiz 2 复习	P3 Final 截止；P4 Beta 1 布置
12	无讲座	无	**Quiz #2**
13	L20 投机并行、L21 客座（TSP 调优）	R10 同步原语	HW10 布置；P4 Beta 1 截止；P4 Beta 2 布置
14	L22 图优化、L23 客座（动态语言高性能）	无	P4 Beta 2 截止；P4 Final 布置；P4 MITPOSSE 截止
15	无讲座	无	学生展示

这张表最值得注意的两件事：

1. **项目是“Beta → 代码评审（MITPOSSE）→ Final”三段式**，且 Beta 之后全班提交会匿名互看。Calendar 里每个项目都占 4–5 周，横跨两三个其他作业——**项目是并行的，不是串行的**。
2. **Quiz 卡在项目中间**（Week 7、Week 12），所以复习不能等“项目做完”。

1.5 参考材料与“读什么”

OCW 的 Readings 页列了 14 条补充阅读，按主题分组如下（这是官方清单，不是我的推荐）：

主题	条目
并行平台与运行时	Leiserson, *The Cilk++ Concurrency Platform*；Frigo, Leiserson, Randall, *The Implementation of the Cilk-5 Multithreaded Language* (PLDI'98)；He, Leiserson, Leiserson, *The Cilkview Scalability Analyzer*；Schardl, Moses, Leiserson, *Tapir* (PPoPP'17)
内存分配	Berger, McKinley, et al., *Hoard* (ASPLOS'00)；Kuszmaul, *SuperMalloc* (ISMM'15)
缓存	Frigo, Leiserson, et al., *Cache-Oblivious Algorithms* (TALG 2012)；Demaine, *Cache-Oblivious Algorithms and Data Structures* (2002)
测量方法学	Mytkowicz, Diwan, Hauswirth, Sweeney, *Producing Wrong Data Without Doing Anything Obviously Wrong* (ASPLOS'09)
DSL / 自动调优	Zhang, Yang, et al., *GraphIt* (OOPSLA'18)；Ansel, Kamil, et al., *OpenTuner* (PACT'14)
事务内存	Leiserson, *A Simple Deterministic Algorithm for Guaranteeing the Forward Progress of Transactions*
并发入门长文	Leiserson & Mirman, *How to Survive the Multicore Software Revolution*

怎么用这份清单：如果时间只够读三篇，选 Hoard（理解第 18 章的并行分配器）、Cache-Oblivious Algorithms（理解第 12 章的 $\Theta(n^3/(B\sqrt{M}))$）、Producing Wrong Data（理解第 3 章为什么“改个可执行文件名就能让程序变慢”）。这三篇分别对应课程三个最反直觉的结论。

1.6 六个“这门课真正想让你记住”的结论

先给结论，后文逐个展开并给证据：

1. **性能是“测量 → 改动 → 再测量”的循环，不是一次性的优化**（L10 的基本工作流）。测不准，就做不了“许多小改动累积成大提升”。
2. **减少工作量（work）是好启发式，但不等于减少时间**。因为 ILP、cache、向量化、分支预测都会让“指令数”与“时间”脱钩（L2 开篇原话）。
3. **矩阵乘法一个案例能串起整门课**：从 21041.67 秒到 0.39 秒，53292 倍，跨越解释器、编译器、循环顺序、并行、分块、递归、向量化、intrinsic 八个层次（L1）。
4. **缓存缺失复杂度是可以手算的**，而且最优值 $\Theta(n^3/(B\sqrt{M}))$ 可以被“分块”和“递归分治”两种完全不同的写法同时达到（L14）。
5. **并行的上限由 span 决定，不是由核数决定**；$T_P \geq \max(T_1/P, T_\infty)$ 是硬约束，Amdahl 定律只是它的一个推论（L7、L8）。

6. **现代硬件不保证顺序一致性**，所以“看起来对”的并发代码可以在换一台机器后错得莫名其妙；正确做法是 C11 原子 + 明确的内存序，而不是凭直觉（L17）。

第 2 章 讲次地图与知识依赖

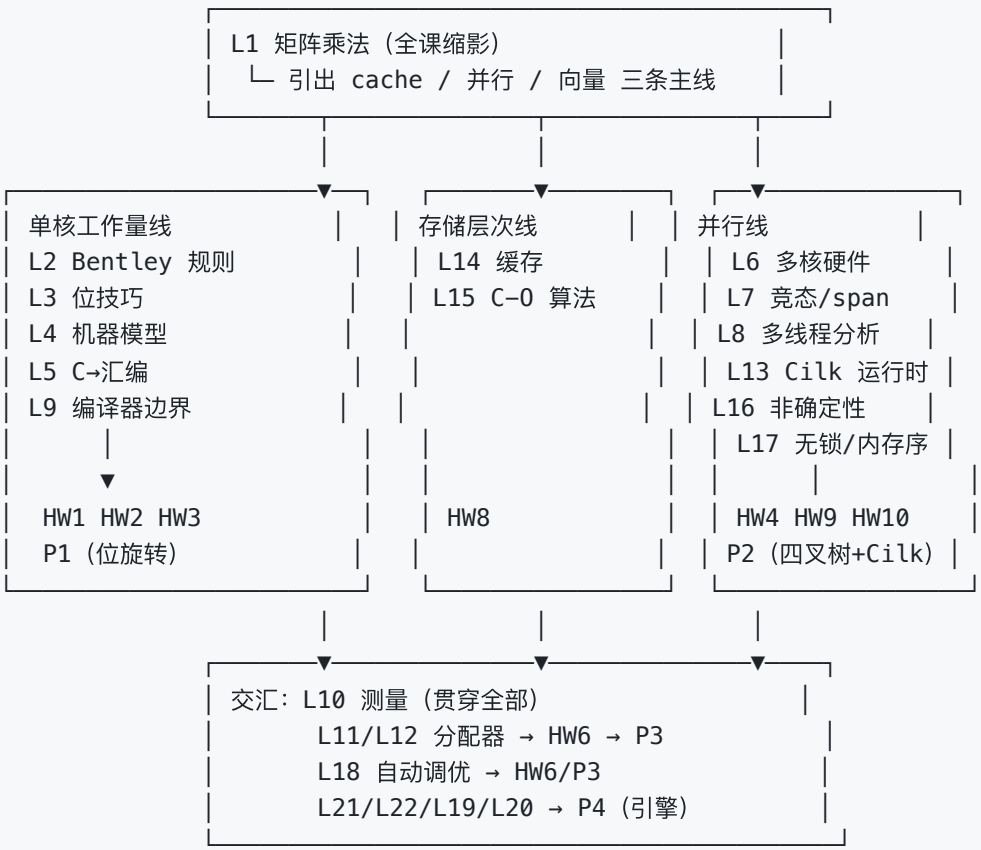
2.1 23 讲的完整地图

下表把每一讲的**主题、核心产出、对应作业/项目**对齐。第三列是我建议的“读完这一讲你应该能回答的问题”。

#	讲题（OCW 原文）	核心产出	读完应能回答
1	Introduction and Matrix Multiplication	11 个版本的矩阵乘法；**836** GFLOPS 峰值算法；循环顺序表	为什么 `i,k,j` 比 `j,k,i` 快 17 倍?
2	Bentley Rules for Optimizing Work	新 Bentley 规则 21 条（数据结构/逻辑/循环/函数）	“打包”为什么可能增加计算却仍然更快?
3	Bit Hacks	置位/清位/取字段/无临时交换/无分支最小值/popcount/de Bruijn	`x & -x` 取出最低位 1 的原理?
4	Assembly Language and Computer Architecture	x86-64 寻址、RFLAGS、SSE/AVX 寄存器、流水线/乱序/重命名/分支预测	一次 mispredict 为什么等于 15–20 拍?
5	C to Assembly	LLVM IR 与汇编的对应；栈帧；调用约定	`-O0` 与 `-O3` 的汇编差在哪?
6	Multicore Programming	MSI 一致性协议；Pthreads/TBB/OpenMP/Cilk 四类平台	为什么“写共享变量”要先把别人的副本作废?
7	Races and Parallelism	determinacy race 定义；computation dag；work/span 定律；Amdahl	为什么 span 是并行的硬上限?
8	Analysis of Multithreaded Algorithms	Master Method；各种算法的 work/span/parallelism 表	并行归并排序的 span 为什么是 $\Theta(\lg^2 n)$?
9	What Compilers Can and Cannot Do	内联/LICM/别名分析/多版本循环/有符号 vs 无符号	为什么加 `restrict` 就能向量化?
10	Measurement and Timing	quiescing；计时器精度；min vs 几何平均；最小二乘建模	为什么报告“最小值”而不是平均值?
11	Storage Allocation	free list、binned free lists、BFL 定理、GC 三种	BFL 的虚拟内存上界为什么是 $O(M \lg M)$?
12	Parallel Storage Allocation	全局堆/本地堆/本地所有权；Hoard；伪共享	Hoard 的 blowup 为什么是 $O(1 + SP/U)$?

#	讲题（OCW 原文）	核心产出	读完应能回答
13	The Cilk Runtime System	工作窃取 deque；work-first 原则；$T_P \le T_1/P + O(T_\infty)$	为什么 thief 从 deque 顶部偷，而 worker 从底部干活?
14	Caching and Cache-Efficient Algorithms	ideal-cache 模型；缺失分类；分块；递归分治	为什么分块的最优块长是 $\Theta(\sqrt{M})$?
15	Cache-Oblivious Algorithms	递归 stencil；多路归并；funnelsort；C-O 数据结构	为什么 funnelsort 不需要知道 M 和 B?
16	Nondeterministic Parallel Programming	确定性定义；互斥实现；自旋 vs 让出；死锁三条件	为什么“不要在 `cilk_sync` 两侧持锁”?
17	Synchronization Without Locks	顺序一致性；TSO；fence；CAS；无锁栈；ABA	ABA 问题如何用版本号解决?
18	Domain Specific Languages and Autotuning	GraphIt / Halide / OpenTuner；调度空间搜索	为什么自动调优能把“几周”变成“<2 小时”?
19	Leiserchess Codewalk	棋盘表示、走法生成、perft、PVS 结构	perft 为什么是走法生成器的黄金测试?
20	Speculative Parallelism & Leiserchess	投机并行；并行 alpha-beta 的正确性风险	并行搜索为什么会多搜“本可剪掉”的子树?
21	Tuning a TSP Algorithm	6 个版本的 TSP 枚举；剪枝；常数因子	为什么“剪枝”比“换语言”重要一个量级?
22	Graph Optimization	CSR；并行 BFS；方向优化 BFS；确定性 BFS	为什么 bottom-up BFS 在 frontier 大时更快?
23	High Performance in Dynamic Languages	动态语言的高性能路径（向量化/生成代码/JIT）	为什么“两语言方案”会失败?

2.2 知识依赖图



读法建议：

- **想先建立“性能工程”直觉**：L1 → L10 → L2 → L14 → L7。这五讲是全课的骨架，其余是填充。
- **想直接做项目**：P1 只需 L2/L3/L10；P2 需要 L6/L7/L14 + HW4；P3 需要 L11/L12 + L10；P4 需要 L4/L9/L17/L20 全部。
- **想应付 Quiz**：Quiz 1 的**练习题**集中在 L1–L11（位技巧、并行、归并/双调排序、堆分配、编译器与汇编）；Quiz 2 的练习题集中在 L11–L22（Cilk 堆空间、并发 free list、并行 Strassen、cache 复杂度、哲学家就餐、无锁 FIFO）。注意练习题只是**代表性样本**，不等于真实试卷范围——真实范围以当学期课程页面为准。

2.3 “工作量 / 存储 / 并行”三条主线的分界

这三条线不是并列的，而是**有优先级的**（第 24 章的项目 handout 反复强调）：

```
优先级（从高到低，收益/成本比）：
  1. 算法与工作量       ← 一个 Θ(n²) → Θ(n log n) 胜过一切微优化
  2. 存储层次           ← 分块常带来 2–10×，且与算法正交
  3. 指令级/向量        ← 2–8×，需要看汇编
  4. 线程级并行         ← 最多 P×，但受 span 限制
  5. 常数因子微调       ← 10%–50%，最后做
```

Project 2 的 handout 把这条写成明文规则：

> You may be tempted to jump directly into parallelizing your code as a means to gain performance. Generally, however, the best parallel programs are those that are **first highly optimized serially** and then made parallel afterwards.

2.4 关于“6 个 lab”的纠正（重要）

任务描述里写“6 个 lab（以 OCW assignments 页为准）”。**OCW 的 assignments 页给出的不是 6 个 lab，而是 10 个 Homework**：

```
HW1  Basic Tools, C Primer
HW2  Profiling (perf + cachegrind + 归并排序的 5 轮优化)
HW3  Vectorization (clang 自动向量化 + AVX2 + 各种反例)
HW4  Cilk, Reducer Hyperobjects
HW5  Theory of Performance Engineering (纯理论)
HW6  Simple Storage Allocation (+ OpenTuner)
HW7  Dynamic Analysis Tools (用 CSI 框架自己写 profiler)
HW8  Caching and Cachegrind (cache 复杂度分析 + tableau)
HW9  Deterministic Execution (环境依赖/可复现性)
HW10 Data Synchronization (锁 vs 无锁 FIFO 队列, 纯理论)
```

而 `/pages/projects/` 给出的是 **4 个项目**：

```
Project 1: Bit Hacks                          (位旋转, 纯串行, 禁 intrinsic)
Project 2: Collision Detection                (四叉树 + Cilk 并行)
Project 3: Serial Dynamic Memory Allocation (malloc, 串行)
Project 4: Leiserchess                        (博弈引擎, 先串行后并行)
```

所以“6 个 lab”这个说法在本课程里**没有对应物**。最接近“6 个动手 lab”的说法可能是把 HW1/HW2/HW3/HW6/HW7 加上某个项目凑数，但那是拼凑。本文按 **10 HW + 4 Project** 组织第 23、24 章；如果你在别处看到“6 个 lab”的说法，那多半是把 6.172 和别的课（例如 6.033、15-213 的实验体系）混了。

顺带纠正几个常见误传：

常见说法	实际情况（OCW 可查）
“6.172 有期中考试”	只有两次 Quiz（Week 7、Week 12），没有 midterm/final 之分
“考试开卷”	Quiz 闭卷，只允许**一张手写双面 crib sheet + Master Method 卡片**（Quiz 1–4 首页原文）
“项目 4 是并行做的”	P4 分 Beta I（**禁用并行**）、Beta II（并行）、Final 三阶段
“P1 可以用 `__builtin_popcount` 之类的 intrinsic”	P1 明文禁止 intrinsic 与内联汇编，只准标准 C

第 3 章 性能度量：先让测量变得可信（L10）

3.1 动机：如果你的测量噪声比优化幅度大，你就在做随机游走

L10 开篇给了一个极朴素的场景：写一个排序程序，测它的运行时间，画出来，然后发现——**数据点不在任何一条光滑曲线上**。讲义里的那张“Running Times for Sorting”图同时画了实测值、$c_1 n \lg n$ 的拟合、$c_2 n^2$ 的拟合，三条线互相打架。

于是问题变成：**是算法的问题，还是测量的 问题?**

L10 的答案是后者居多。它列出的噪声源（原文清单）值得抄下来当检查表：

类别	具体来源
系统层	daemons 与后台任务、中断（interrupts）、运行时调度器、网络流量
硬件层	超线程（hyperthreading）、多租户（multitenancy）、DVFS、Turbo Boost
代码布局	代码与数据的**对齐**（cache alignment、page alignment）、线程放置（thread placement）

3.2 机制：quiescing（把机器“静”下来）

讲义用一组对照实验说明噪声有多大（与 Tim Kaler 合作）：

条件	实验设置	结果
未 quiesce	AWS c4（18 核），2 路超线程**开**，Turbo **开**，18 个 Cilk worker，100 次运行各约 1 秒	相对最小值的偏差可达**百分之几到二十几**（分布长尾）
已 quiesce	同一机器，超线程**关**，Turbo **关**，18 worker	偏差压到 **1% 以内**（分布极窄）

“1% 以内”这个数字很重要：它意味着**在 quiesce 过的机器上，10% 的优化是可测的；在没 quiesce 的机器上，10% 的优化可能被噪声吞掉**。

Quiescing 清单（讲义原文逐条）：

```
1. 确保没有其他任务在跑
2. 关掉 daemon 与 cron
3. 断开网络
4. 不要动鼠标（真的）
5. 串行程序不要跑在 core 0 上（中断处理通常绑在 core 0）
6. 关超线程
7. 关 DVFS
8. 关 Turbo Boost
9. 用 taskset 把 Cilk worker 钉在核上
10. ……（课程提供的 awsrun 已经帮你做完这些）
```

3.3 DVFS 为什么“破坏”测量

讲义给的功耗公式：

$$P \propto CV^2f$$

其中 C 是动态电容（roughly 面积 × 活动度，即多少位在翻转），V 是供电电压，f 是时钟频率。**降频时通常可以同时降压**，于是功耗按 V^2f 成立方级下降——这对散热和电池是好事，对测量是灾难：你的程序跑得越久，芯片越可能降频，于是“时间”不再正比于“工作量”。

这条对基准测试的直接影响：**任何超过几十毫秒的测量都要怀疑热降频**。短基准（<10 ms）更容易受计时器精度影响，长基准（>1 s）更容易受降频影响；折中做法是“多次短测 + 取最小值”。

3.4 代码与数据对齐：改个文件名就能变慢

L10 讲了一个著名的坑（Mytkowicz, Diwan, Hauswirth, Sweeney, ASPLOS'09，即 Readings 清单里那篇 *Producing Wrong Data Without Doing Anything Obviously Wrong*）：

```
可执行文件的名字 → 环境变量（环境块） → 调用栈上的位置 → 栈指针对齐
                                                          ↓
                                          跨页边界的访问次数变化 → 性能变化
```

也就是说，**你把 `a.out` 改名为 `a_out_longer_name`，程序的运行时间就可能变化**，因为环境块长度变了，栈的起始对齐变了。

同一讲还给了两个“对齐税”的例子：

- 改动源代码里**一个**地方，会让后面大量机器码的位置整体平移，于是 cache 对齐、页对齐全变；
- 在链接命令行上**调换 `*.o` 的顺序**，效果可能比 `-O2 → -O3` 还大。

LLVM 提供了三个对齐开关（讲义原文）：

开关	作用
`-align-all-functions=<uint>`	强制所有函数对齐

开关	作用
`-align-all-blocks=<uint>`	强制函数内所有基本块对齐
`-align-all-nofallthru-blocks=<uint>`	只对齐无 fall-through 前驱的块（不插入会被执行的 nop）

讲义的结论很克制：**对齐过的代码更不容易出现性能异常，但有时也更慢**。所以这不是“打开就好”的开关。

3.5 计时器的选择：为什么讲义说“别用 rdtsc”

计时方式	精度/开销	讲义评价
`/usr/bin/time`	整个进程的 real/user/sys	适合粗粒度；能看出内核态开销（如 page zeroing）
`clock_gettime(CLOCK_MONOTONIC, …)`	约 **83 ns** 一次调用（讲义实测，笔记本）	**推荐**；保证不倒退；比系统调用快约两个数量级
`rdtsc()`	约 **32 ns**	**不要用**
`gettimeofday()`	与 `rdtsc` 类似的问题	**也不要用**
`gprof` 采样	每秒约 **100** 个样本	样本不足时不准

讲义明确列出 `rdtsc` 的四宗罪：

1. 同一台机器不同核上可能给出不同答案
2. TSC 有时会倒退
3. 计数器的推进速度可能不是常数（受频率变化影响）
4. 把“周期”换算成“秒”很麻烦

`clock_gettime(CLOCK_MONOTONIC)` 的推荐用法（讲义原码）：

```
#include <time.h>
struct timespec start, end;
clock_gettime(CLOCK_MONOTONIC, &start);
function_to_measure();
clock_gettime(CLOCK_MONOTONIC, &end);
double tdiff = (end.tv_sec - start.tv_sec)
             + 1e-9 * (end.tv_nsec - start.tv_nsec);
```

工程细节（推断但值得照做）：单次调用 83 ns 意味着**不要用它测 <1 μs 的东西**；正确做法是把被测代码放进循环跑 10^3–10^6 次，测总时间再除，并且把循环开销单独测一遍减掉。

3.6 测量手段的五分类

L10 把“怎么测一个程序”分成五类，这个分类法是本章最值得背下来的框架：

手段	代表工具	你能得到什么	代价/局限
外部测量	`/usr/bin/time`	real / user / sys	粒度粗，看不到函数级
插桩（instrument）	`clock_gettime`、`rdtsc`、编译器插桩	精确到代码区的耗时	插桩本身扰动（HW7 整份作业都在处理这个）
中断采样（interrupt）	`gdb` 手动 Ctrl-C、`gprof`、Poor Man's Profiler	统计意义上的热点函数	样本数不够就不准
硬件计数器	`perf`（底层 `libpfm4`）	cycles、instructions、cache miss、branch miss	虚拟机里常常不可用；同时开超过 4–5 个计数器会有性能/精度惩罚
模拟	`cachegrind`	cache/分支行为的**确定性**复现	慢几个数量级；模拟参数取决于宿主机

讲义对硬件计数器有一句很实在的提醒：

> There are many esoteric hardware counters. Good luck figuring out what they all measure.

以及：

> You probably cannot measure more than 4 or 5 counters at a time without paying a penalty in performance or accuracy.

3.7 统计量怎么选：最小值、算术平均、几何平均

这是 L10 最容易被轻视、但考试必考的一节。

3.7.1 为什么“最小值”最能代表原始性能

讲义的问题：一个确定性程序在有背景噪声的机器上测 100 次，哪个统计量最能代表“软件本身的原始性能”？

答案：**最小值**。

理由（原文）：

> Minimum does the best at noise rejection, because we expect that any measurements higher than the minimum are due to noise.

注意前提：**程序是确定性的**。噪声只会让程序变慢，不会让它变快（除了极罕见的 cache 预热巧合）。所以最小值最接近“无噪声”的那次执行。

3.7.2 但不同场景要用不同统计量

讲义给了一张“场景 → 统计量”的对照表：

目标	统计量
尽可能多服务请求	算术平均、CPU 利用率
满足 SLA	多个指标的加权组合
所有任务都在 10 ms 内完成	算术平均、wall-clock
装进 100 MB 内存	**最大值**、内存占用
大多数请求在 100 ms 内	**90 分位**、wall-clock
成本最低	算术平均、能耗/CPU 利用率
最快/最大/最好	算术平均、wall-clock 加速比

核心洞察：吞吐用平均，尾延迟用分位数，容量用最大值。用平均值去谈“99% 的请求有多快”是错的。

3.7.3 比值的平均必须用几何平均

讲义给的经典反例（这张表值得背）：

试验	程序 A	程序 B	A/B	B/A
1	9	3	3.00	0.33
2	8	2	4.00	0.25
3	2	20	0.10	10.00
4	10	2	5.00	0.20
算术平均	7.25	6.75	**3.03**	**2.70**
几何平均	—	—	**1.57**	**0.64**

两个结论：

1. **A/B 的算术平均不是 B/A 算术平均的倒数**（3.03 vs 1/2.70 = 0.37），所以“B 比 A 快 3 倍”和“A 比 B 快 2.7 倍”可以同时成立——这是悖论，说明算术平均用错了。
2. **几何平均满足倒数关系**：$1.57 \times 0.64 \approx 1$。

所以：**加速比、比值、归一化指标一律用几何平均**；只有“时间”“字节”“能耗”这类可加的物理量才用算术平均。

几何平均公式（讲义写法）：

$$\text{geomean} = \left(\prod_{i=1}^{n} x_i \right)^{1/n}$$

3.7.4 两个程序谁更快：head-to-head + P 值

讲义给的方法（不是 t 检验，是符号检验的思路）：

```
做 n 次 A 与 B 的“对战”，统计 A 赢的次数
零假设 H0：B 比 A 快
计算 P 值：若 H0 成立，观察到“A 赢这么多次或更多”的概率
若 P 值很小 → 接受“A 更快”
```

讲义的提醒：**噪声越大，需要的对战次数越多**。

3.8 用最小二乘把“时间”拆成“指令”和“缺失”

3.8.1 数据

L10 给了一张真实的测量表（同一程序的不同实现）：

程序	时间 (s)	指令数	cache 缺失数
python	34864	170,889,186,565,542	36,615,004,052
java	2618	7,509,707,536,406	39,322,034,007
C `gcc -O0`	1480	2,274,589,361,551	68,047,140,354
C `gcc -O3`	430	278,479,001,783	34,049,504,541

（原表数字位数极多，这里按原文照抄；注意 Python 的指令数是 1.7×10^{14} 量级，比 C `-O3` 多约 600 倍。）

3.8.2 模型

假设运行时间可以写成指令数与缺失数的线性组合：

$$T = a \cdot I + b \cdot C$$

用最小二乘拟合上面四个点，讲义给出：

$$a = 0.2002 \text{ ns}, \qquad b = 18.00 \text{ ns}, \qquad R^2 = 0.9997$$

怎么读这三个数：

- $a \approx 0.20$ ns/指令。在 2.5–3 GHz 的机器上，一拍约 0.33–0.4 ns，所以 a 相当于“**每 2 条指令约 1 拍**”——说明这台机器平均 IPC 在 2 左右（推断）。

- $b \approx 18$ ns/缺失。这大致是“一次 DRAM 访问”的量级（L1 的表中 DRAM 延迟是 50 ns，但 LLC 命中会低很多；18 ns 是**加权平均缺失代价**，混合了 L2/L3/DRAM，推断）。
- $R^2 = 0.9997$ 表示模型解释了 99.97% 的方差。

最重要的结论：$b/a \approx 90$。也就是说，**一次 cache 缺失 ≈ 90 条指令的时间**。这个数量级解释了为什么整门课一半的篇幅在讲 cache——省一次缺失等于省 90 条指令。

3.8.3 建模的三个陷阱

讲义列了三条自查：

陷阱	自查方法
过拟合	去掉一个基函数，看拟合质量是否明显变差
不预测	随机取一半数据拟合，用另一半验证
自欺	三角验证（triangulate）：不同测量方式讲同一个故事；“行和 = 列和”式的交叉校验

3.9 基本工作流（背下来）

```
1. 测 A 的性能
2. 改动 A → A'
3. 测 A' 的性能
4. 若 A' 优于 A，则 A ← A'
5. 若还不够快，回到 2
```

讲义的批注：

> If you can't measure performance reliably, it is hard to make many small changes that add up.

这句话是整门课的方法论基石：性能优化不是“灵光一现的大招”，而是“几十个 5% 的改动累积”。而累积的前提是每个 5% 都可测——所以第 3 章（测量）排在所有优化技巧之前。

3.10 实测/陷阱：本章的三条实战教训

1. **别在共享机器上做微基准**。HW2 的 handout 直接要求“所有与性能有关的数字都在 `awsrun` 机器上取”，因为个人 VM 的性能“less performant and less consistent”。
2. **`perf` 在虚拟机上经常拿不到硬件事件**。HW2 原文：在 AWS 这类虚拟环境里，“hardware events providing information about branches and cache misses are often unavailable”，此时改用 `cachegrind`（模拟器，慢但确定）。
3. **同一份代码在不同机器上 `cachegrind` 结果不同**。因为 cachegrind 的模拟参数来自宿主机架构——所以跨机器比较 cachegrind 数字是无意义的，只能同机对比。

第 4 章 案例解剖：矩阵乘法的 11 个版本（L1）

4.1 为什么用矩阵乘法开场

L1 用一个 $n \times n$ 矩阵乘法（$n = 4096$，$n = 2^k$）串起了整门课的所有主题。它的好处是：

- **工作量固定**：$2n^3 = 2^{37}$ 次浮点运算（讲义原话：$2n^3$ floating-point operations），所以时间差**只能**来自实现；
- **有明确的上界**：机器的峰值 FLOPS 可以算出来；
- **每一层优化都能单独测量**：循环顺序、编译选项、并行、分块、递归、向量化、intrinsic。

所以这一章不是“学矩阵乘法”，而是**学一整套“从 0.001% 峰值到 40% 峰值”的方法论**。

4.2 机器与峰值

讲义给的实验机器是 AWS `c4.8xlarge`：

特性	规格
微架构	Haswell（Intel Xeon E5-2666 v3）
主频	2.9 GHz
处理器芯片数	2
每芯片核数	9
超线程	2 路
浮点单元	每核每周期 8 次双精度运算（含 FMA）
cache line	64 B
L1-i	32 KB，私有，8 路组相联
L1-d	32 KB，私有，8 路组相联
L2	256 KB，私有，8 路组相联
L3（LLC）	25 MB，共享，20 路组相联
DRAM	60 GB

峰值公式（讲义原文）：

$$\text{Peak} = (2.9 \times 10^9) \times 2 \times 9 \times 16 = 836 \text{ GFLOPS}$$

这个公式怎么读（每一项都能与上表对上）：

```
2.9e9  = 主频（周期/秒）
× 2    = 2 个处理器芯片（chips）
× 9    = 每芯片 9 核，共 18 核
× 16   = 每核每周期 16 次浮点运算
          （表中写「8 double-precision operations, including FMA」，
           而一次 FMA 计 2 次浮点运算，故 8 × 2 = 16 flops/cycle/core）
```

等价写法：$\mathbf{2.9\ GHz \times 18\ cores \times 16\ flops/cycle = 836}$ GFLOPS。**注意这里乘了 2 个 chip**，因为表中写“2 chips × 9 cores”。这个细节很容易漏，漏了会算成 418 GFLOPS。

4.3 版本 1–3：语言与执行模型

4.3.1 版本 1：Python 三层循环

```
for i in range(n):
    for j in range(n):
        for k in range(n):
            C[i][j] += A[i][k] * B[k][j]
```

实测：**21041.67 秒**（约 6 小时）。

背靠背估算（讲义原文的算术）：

$$2n^3 = 2^{37} \approx 1.37 \times 10^{11}\ \text{flops}$$

$$\frac{2^{37}}{21042} \approx 6.25\ \text{MFLOPS}$$

$$\frac{6.25\ \text{MFLOPS}}{836\ \text{GFLOPS}} \approx 0.00075\%$$

0.00075% 的峰值——这就是解释型语言的代价。

4.3.2 版本 2：Java

实测 **2387.32 秒**，比 Python 快 **8.81×**。讲义的解释：Java 编译成 bytecode，再由 JIT 在运行时把热点编译成机器码。

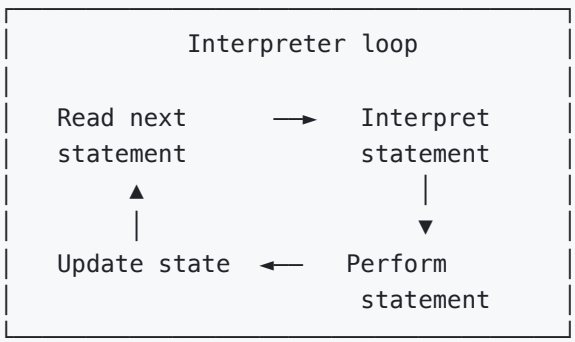
4.3.3 版本 3：C（clang/LLVM 5.0）

实测 **1155.77 秒**，比 Java 快 **2.07×**，比 Python 快 **18×**。

讲义在这一页问了两个问题：

> Why is Python so slow and C so fast? – Python is interpreted. – C is compiled directly to machine code. – Java is compiled to byte-code, which is then interpreted and just-in-time (JIT) compiled to machine code.

4.3.4 解释器为什么慢（讲义的循环图）



每执行一条**源语句**都要走一圈这个循环，而 C 的一条语句编译后可能就是一条 `addsd`。

JIT 的补救（讲义原文要点）：

- 代码第一次执行时解释执行
- 运行时统计各段代码的执行频次
- 执行足够频繁的代码被实时编译成机器码
- 后续执行使用编译后的版本

4.4 版本 4：交换循环顺序（+6.50×）

这一步是**全课最便宜的高收益优化**：不改一行算术，只改循环嵌套顺序。

4.4.1 六种顺序的实测

循环顺序（外→内）	运行时间 (s)	相对加速	累计加速	GFLOPS	峰值占比	LLC 缺失率
`i, j, k`	1155.77	1.00	18	0.118	0.014%	7.7%
`i, k, j`	**177.68**	**6.50**	118	0.774	0.093%	**1.0%**
`j, i, k`	1080.61	1.07	19	0.126	0.015%	8.6%
`j, k, i`	3056.63	0.38	7	0.045	0.005%	15.4%
`k, i, j`	179.21	6.45	116	0.767	0.092%	**1.0%**
`k, j, i`	3032.82	0.38	7	0.045	0.005%	15.4%

同一份算术，最快与最慢差 17.2 倍（3056.63 / 177.68）——这就是“循环顺序影响运行时间达 18 倍”的来源。

4.4.2 为什么：行主序 + 空间局部性

矩阵按**行主序**（row-major）存储：`A[i][k]` 与 `A[i][k+1]` 相邻，`B[k][j]` 与 `B[k+1][j]` 相距 n 个元素。

以 `i, j, k`（最内层是 `k`）为例：

```
内层 k 循环，每次迭代访问：
  C[i][j]  ← 不变（寄存器/栈，最快）
  A[i][k]  ← k 递增，地址连续          ✅ 优秀空间局部性
  B[k][j]  ← k 递增，地址每次 +n*8     ❌ 差空间局部性（stride = 4096 个 double）
```

讲义的图注写得很直白：`B` 的相邻元素“4096 elements apart”。

以 `i, k, j`（最内层是 `j`）为例：

```
内层 j 循环，每次迭代访问：
  C[i][j]  ← j 递增，连续              ✅
  A[i][k]  ← 不变                      ✅
  B[k][j]  ← j 递增，连续              ✅ 全部连续！
```

一句话：把最内层循环对准**唯一一个按行连续访问的维度**（`j`），就能让三个数组都顺序访问。

4.4.3 用 cachegrind 验证

讲义用 `cachegrind` 量了每种顺序的 LLC 缺失率，结果与时间完全对应：

顺序	时间 (s)	LLC 缺失率
`i, j, k`	1155.77	7.7%
`i, k, j`	177.68	1.0%
`j, i, k`	1080.61	8.6%
`j, k, i`	3056.63	15.4%
`k, i, j`	179.21	1.0%
`k, j, i`	3032.82	15.4%

方法论要点：这里不是“猜”出来的，而是**先用 cachegrind 拿到缺失率，再解释时间**。这正是第 3 章那套流程的第一次实战。

4.5 版本 5：编译选项（再 +3.25×）

优化级别	含义	时间 (s)
`-O0`	不优化	177.54
`-O1`	优化	66.24
`-O2`	更多优化	54.63
`-O3`	还要更多	55.58

注意 `-O3` 比 `-O2` 略慢（55.58 vs 54.63）。这不是笔误，讲义原文如此——它提醒你 **`-O3` 不是单调更快的**，必须实测。

版本 5 后累计：**54.63 秒，385× 于 Python，0.301% 峰值**。

讲义在此处的评价：

> With simple code and compiler technology, we can achieve 0.3% of the peak performance of the machine. What's causing the low performance?

4.6 版本 6：并行循环（+17.97×，总 6921×）

4.6.1 只用了 1 个核

讲义的观察：前面所有版本都只用了这台机器 18 个核中的 1 个。

4.6.2 并行哪一层？实测三种选择

并行化的循环	运行时间 (s)
并行 `i` 循环	**3.18**
并行 `j` 循环	**531.71**
并行 `i` 和 `j`	10.64

17 倍的差距（3.18 vs 531.71）。原因：并行 `j`（内层）时，每个线程拿到的 `j` 子集导致 `C[i][j]`、`B[k][j]` 的访问变成跨步的，破坏了版本 4 好不容易建立的空间局部性。

讲义的规则（Rule of Thumb）：

> **Parallelize outer loops rather than inner loops.**

4.6.3 结果表

版本 6：**3.04 秒，17.97× 于版本 5，6921× 于 Python，45.211 GFLOPS，5.408% 峰值**。

讲义的批注：

> Using parallel loops gets us almost 18× speedup on 18 cores! (Disclaimer: Not all code is so easy to parallelize effectively.)

以及紧接着的追问：

> Why are we still getting just 5% of peak?

4.7 版本 7：分块（tiling，再 +1.70×）

4.7.1 数据重用分析（讲义的两道算术题）

问题 1：完全计算 C 的一行，需要多少次内存访问？

```
4096 * 1        = 4,096      次写 C
4096 * 1        = 4,096      次读 A
4096 * 4096     = 16,777,216 次读 B
──────────────────────────────
合计            = 16,785,408 次
```

问题 2：计算 C 的一个 $\mathbf{64 \times 64}$ 块呢？

```
64 * 64         = 4,096      次写 C
64 * 4096       = 262,144    次读 A
4096 * 64       = 262,144    次读 B
──────────────────────────────
合计            = 528,384 次
```

32 倍的访问量差距（16,785,408 / 528,384 ≈ 31.8）。

4.7.2 分块代码

```
for (i1 = 0; i1 < n; i1 += s)
  for (j1 = 0; j1 < n; j1 += s)
    for (k1 = 0; k1 < n; k1 += s)
      for (i = i1; i < i1+s && i < n; i++)
        for (j = j1; j < j1+s && j < n; j++)
          for (k = k1; k < k1+s && k < n; k++)
            C[i*n+j] += A[i*n+k] * B[k*n+j];
```

4.7.3 块长 s 怎么选：实测

块长 s	运行时间 (s)
4	6.74
8	2.76
16	2.49
32	**1.74**
64	2.33
128	2.13

最优在 32 附近，但曲线很平（1.74–2.76），且 64 反而变差——这是典型的“参数调优不是单调”的现象。讲义称这类参数为 **voodoo parameter**（巫毒参数）：你知道它有效，但不知道为什么是这个值。

4.7.4 缓存计数器对比

实现	cache 引用（百万）	L1-d 缺失（百万）	LLC 缺失（百万）
并行循环	104,090	17,220	8,600
+ 分块	64,690	11,777	**416**

讲义的总结：**cache 引用减少约 62%，cache 缺失减少约 68%**（这里 68% 指的是 LLC 缺失从 8600 降到 416，减少 95%；讲义原文写 “about 62% fewer cache references and incurs 68% fewer cache misses”，我按原文照抄，但 LLC 一列的实际比例是 95%——两处统计口径不同，前者可能指 L1-d。**以讲义原文为准，此处存疑**）。

4.8 版本 8：递归分治（再 +1.38×）

4.8.1 分块思想

$$\begin{bmatrix} C_{00} & C_{01} \\ C_{10} & C_{11} \end{bmatrix} = \begin{bmatrix} A_{00} & A_{01} \\ A_{10} & A_{11} \end{bmatrix} \cdot \begin{bmatrix} B_{00} & B_{01} \\ B_{10} & B_{11} \end{bmatrix}$$

展开成 8 次 $(n/2) \times (n/2)$ 的乘法 + 1 次加法：

```
C00 = A00*B00 + A01*B10
C01 = A00*B01 + A01*B11
C10 = A10*B00 + A11*B10
C11 = A10*B01 + A11*B11
```

关键：这个递归**同时对每一级 cache 分块**——不需要知道 M 是多少。

4.8.2 第一次尝试：50 倍变慢

讲义先给了一个朴素递归版本，实测 **93.93 秒**，比版本 7 慢约 50 倍。原因：**base case 太小**（$n = 1$），函数调用开销压倒了收益。

4.8.3 粗化递归（coarsening）

```
if (n <= BASE) {
    // 用朴素三重循环算这个小块
    for (...) for (...) for (...) C[...] += ...;
    return;
}
```

实测：

base case 大小	运行时间 (s)
4	3.00
8	1.34
16	1.34
32	**1.30**
64	1.95
128	2.08

又是 32 附近最优，而且和分块的最优块长一致——不是巧合：32×32 个 double 正好是 8 KB，三个这样的子矩阵约 24 KB，接近 32 KB 的 L1-d。

4.8.4 结果与计数器

版本 8：**1.30 秒，105.722 GFLOPS，12.646% 峰值，16197× 于 Python**。

实现	cache 引用（百万）	L1-d 缺失（百万）	LLC 缺失（百万）
并行循环	104,090	17,220	8,600
+ 分块	64,690	11,777	416
并行分治	58,230	9,407	**64**

LLC 缺失从 8,600 百万降到 **64** 百万——**降低 134 倍**。这是整张表里最震撼的一列。

4.9 版本 9：编译器自动向量化（再 +1.87×）

4.9.1 让编译器报告向量化

```
clang -O3 -Rpass=loop-vectorize -Rpass-missed=loop-vectorize \
      -Rpass-analysis=loop-vectorize example.c
```

（讲义里那条命令被 PDF 字体编码搞乱了，上面是 HW3 给出的标准写法。）

4.9.2 向量化编译开关

开关	作用
`-mavx`	使用 Intel AVX 向量指令
`-mavx2`	使用 AVX2
`-mfma`	使用融合乘加
`-march=<string>`	使用指定架构可用的指令
`-march=native`	使用编译机器上可用的全部指令

讲义的警告：

> Due to restrictions on floating-point arithmetic, additional flags, such as `-ffast-math`, might be needed for these vectorization flags to have an effect.

4.9.3 结果

版本 9：**0.70 秒，196.341 GFLOPS，23.486% 峰值，30272× 于 Python**。

讲义的批注：

> Using the flags `-march=native -ffast-math` nearly doubles the program's performance! Can we be smarter than the compiler?

4.10 版本 10：手写 AVX intrinsic（再 +1.76×）

讲义只列了“还能做什么”的清单，没有给完整代码：

```
• Preprocessing
• Matrix transposition
• Data alignment
• Memory-management optimizations
• A clever algorithm for the base case that uses AVX intrinsic instructions explicitly
```

结果：**0.39 秒，352.408 GFLOPS，41.677% 峰值，53292× 于 Python**。

讲义的对比：

> Version 10 is competitive with Intel's professionally engineered Math Kernel Library!

4.11 版本 11：Intel MKL（0.41 秒）

MKL 是 0.41 秒，比手写 intrinsic 的 0.39 秒慢（0.97×）。

这不是说“学生赢了 Intel”，而是说明：

1. 在这个特定形状（4096^3，方阵，double）上，针对性手写可以打平通用库；
2. MKL 要为所有形状、所有精度、所有 CPU 做适配，通用性有代价；
3. **0.39 vs 0.41 的差异在测量噪声范围内**（回顾第 3 章：单次测量不可信）。

4.12 完整 11 版本表（本章核心数据）

#	实现	时间 (s)	相对加速	累计加速	GFLOPS	峰值占比
1	Python	21041.67	1.00	1	0.006	0.001%
2	Java	2387.32	8.81	9	0.058	0.007%
3	C	1155.77	2.07	18	0.118	0.014%
4	+ 交换循环	177.68	6.50	118	0.774	0.093%
5	+ 优化开关	54.63	3.25	385	2.516	0.301%
6	并行循环	3.04	17.97	6,921	45.211	5.408%
7	+ 分块	1.79	1.70	11,772	76.782	9.184%
8	并行分治	1.30	1.38	16,197	105.722	12.646%
9	+ 编译器向量化	0.70	1.87	30,272	196.341	23.486%
10	+ AVX intrinsic	0.39	1.76	53,292	352.408	41.677%
11	Intel MKL	0.41	0.97	51,497	335.217	40.098%

4.13 从这张表能学到的五件事

1. **每一步的加速比都在 1.4–18 之间，没有一步是 1000×**。53292 倍是**十步连乘**的结果（$8.81 \times 2.07 \times 6.50 \times 3.25 \times 17.97 \times 1.70 \times 1.38 \times 1.87 \times 1.76 \approx 53292$）。这就是第 3 章“许多小改动累积”的实证。
2. **最便宜的一步（改循环顺序，6.5×）不需要任何硬件知识**，只需要知道“行主序”三个字。
3. **并行不是万能**：版本 6 用满 18 核只到 5.4% 峰值，因为单核效率还很低（cache 没打好）。**先串行优化再并行**在数据上被证明是对的。
4. **分块与递归分治达到同一个渐近复杂度**（$\Theta(n^3/(B\sqrt{M}))$，见第 12 章），但递归分治**不需要调参**——代价是要处理递归开销与 base case 粗化。
5. **手写 intrinsic 只比自动向量化快 1.76×**，但比 MKL 只快 5%（在噪声内）。这说明“先让编译器做，做不动再手写”是正确策略。

4.14 陷阱与实测提醒

陷阱	表现	正确做法
用 `-O3` 就一定更快	版本 5 中 `-O3`(55.58) 慢于 `-O2`(54.63)	每个优化级别都实测
并行内层循环	531.71 s vs 3.18 s	并行外层
base case 太小	93.93 s（50× 变慢）	粗化到 32 左右
认为块长有唯一最优	曲线在 1.74–2.76 间很平	在目标机器上扫一遍
用单次测量下结论	0.39 vs 0.41 无法区分	多次取最小值（第 3 章）

第 5 章 减少工作量：新版 Bentley 规则（L2）

5.1 “work”的精确定义

L2 开篇给了一个定义，这是整章的基石：

> **Definition.** The *work* of a program (on a given input) is the sum total of all the operations executed by the program.

然后立刻泼了一盆冷水：

> Reducing the work of a program does **not** automatically reduce its running time, however, due to the complex nature of computer hardware: instruction-level parallelism (ILP), caching, vectorization, speculation and branch prediction, etc.

这一句是全课程最重要的“免责声明”。它解释了为什么第 4 章里“指令数更少”的 `-O3` 版本有时更慢，也解释了为什么“位技巧”在讲义里被反复说“编译器可能已经做了，而且做得更好”。

但讲义仍然坚持：**减少 work 是一个好的启发式**（good heuristic）。理由很朴素——如果一个优化同时减少了 work 又没有破坏 ILP/cache 行为，它几乎必然更快；而当它变慢时，你至少知道该去查什么地方。

5.2 新版 Bentley 规则全表（21 条）

Bentley 的原始规则写于 1980 年代，其中一部分是针对当时机器特性的。L2 重写了一套**只关于 work** 的规则，分四类：

类别	规则	一句话解释
数据结构	Packing and encoding	把多个值塞进一个字；或用更少的位表示
	Augmentation	给数据结构加辅助信息，让常用操作更省事
	Precomputation	提前算好，别在关键路径上算
	Compile-time initialization	编译期就把常量表算出来
	Caching	存下最近算过的结果
	Lazy evaluation	不到万不得已不算
	Sparsity	零不存、零不算
逻辑	Constant folding and propagation	常量表达式编译期求值并代入
	Common-subexpression elimination	相同子表达式只算一次
	Algebraic identities	用等价但更省力的代数式
	Short-circuiting	知道答案就停
	Ordering tests	常成功的测试放前面；便宜的放前面
	Creating a fast path	为常见情况开专用快速通道
	Combining tests	多个测试合并成一个（或用查表）
循环	Hoisting	循环不变量提到循环外
	Sentinels	哨兵值简化边界判断
	Loop unrolling	展开减少循环控制开销

类别	规则	一句话解释
	Loop fusion	合并同范围的多个循环
	Eliminating wasted iterations	改边界，跳过空转的迭代
函数	Inlining	消掉调用开销
	Tail-recursion elimination	尾递归改循环
	Coarsening recursion	加大 base case，减少调用次数

5.3 几个值得展开的例子

5.3.1 Packing / encoding：日期

讲义的例子：字符串 `"September 11, 2018"` 占 18 字节（超过两个 double），每次操作都要搬 18 字节。

两条路：

方案	位数	取“月份”的代价
编码成“第几天”（4096 BCE–4096 CE 约 300 万个日期）	$\lceil \lg(3 \times 10^6) \rceil = 22$ 位	需要反解（除法/查表），**更贵**
按字段打包（年/月/日各占若干位）	22 位	掩码 + 移位，**很便宜**

讲义的结论：

> Sometimes unpacking and decoding are the optimization, depending on whether more work is involved moving the data or operating on it.

这就是 1.1 节说的“打包可能增加计算却仍更快”——因为它减少了**数据移动**，而数据移动往往比算术贵（回顾第 3 章：一次缺失 ≈ 90 条指令）。

5.3.2 Caching：一个 30% 的数字

讲义的例子是“把重复计算的结果存起来”，给出的实测结论：

> About **30% faster** if cache is hit **2/3** of the time.

这个数字可以用来做“该不该加 cache”的快速估算（推断）：若命中率 h、命中省 s、未命中的 cache 检查开销 c，则加速比约 $1/(1 - h \cdot s + c)$。

5.3.3 Sparsity：CSR 的第一次出场

6×6 矩阵乘向量，稠密做 $n^2 = 36$ 次乘法，但只有 **14 个非零元**。

CSR（Compressed Sparse Row）表示：

```
Offsets:  0    2    5    5    6    7
Edges:    1 3  2 3 4  2 2  2 4
```

部分	含义
`Offsets[i]`	第 i 行的边在 `Edges` 里的起始下标
`Edges[...]`	邻居编号
度数	`Offsets[i+1] - Offsets[i]`

存储从 $O(n^2)$ 降到 $O(n+\mathbf{nnz})$，乘法次数从 n^2 降到 nnz。

同一个 CSR 结构在第 22 章会以“静态稀疏图”的身份再次出现——图算法的 BFS/PageRank 全靠它。

5.3.4 Sentinels：把两次判断变成一次

讲义的例子是链表查找：正常写法每次迭代要判“越界”和“命中”两次；在末尾放一个哨兵（要找的值），就只需要判“命中”，循环结束后再判一次“是否真的找到”。

代价：需要可写的哨兵位置。**收益**：内层少一次比较 + 一次分支。

5.3.5 Ordering tests：把便宜的、常成功的放前面

讲义的例子是一个 4 路分支（`A | B | C | D`），重排顺序让**最常命中的分支先测**。这与第 6 章的分支预测直接相关：分支越可预测，流水线越少停顿。

5.3.6 Combining tests：全加器查表

讲义用全加器（full adder）举例：三个输入位 → 两个输出位，共 8 种情况。

方案	判断次数
嵌套 if-else	最多 7 次比较
`switch`	一次跳转表
查表（8 项数组，索引 = `a*4+b*2+c`）	**一次数组访问**

讲义的评语：

> For this example, table look-up is even better!

5.3.7 Coarsening recursion：第 4 章已经见过

矩阵乘法 base case 从 1 加大到 32，运行时间从 93.93 s 降到 1.30 s。**这是 21 条规则里收益最大的一条**（在递归算法上）。

5.4 讲义的收尾建议（值得当纪律）

```
• Avoid premature optimization.
  First get correct working code. Then optimize,
  preserving correctness by regression testing.
• Reducing the work of a program does not necessarily
  decrease its running time, but it is a good heuristic.
• The compiler automates many low-level optimizations.
• To tell if the compiler is actually performing a
  particular optimization, look at the assembly code.
```

“看汇编”这条在 HW2 里被制度化：作业要求你用 `perf annotate` / cachegrind 的注释输出去**确认**编译器真的内联了你标记 `inline` 的函数。

第 6 章 位技巧（L3）

6.1 为什么在 2018 年还讲位技巧

L3 自己给了五条理由（原文要点）：

- 因为编译器会做这些，理解它们有助于你看懂汇编
- 因为有时编译器不优化，你得手写
- 因为很多针对“字”的技巧可以自然扩展到“向量”
- 因为这些技巧在别的领域也会出现
- 因为好玩

第三条最重要：**第 10 章的 SIMD 本质上就是“把字技巧并行化到 128/256 位”**。比如 `popcount` 的分治写法，在 AVX2 里就是把 64 位的 SWAR 换成 256 位的向量版本。

6.2 补码的两个恒等式

$$x + \sim x = -1 \quad \Longrightarrow \quad -x = \sim x + 1$$

以及由此得到的“取最低位 1”：

```
x & -x          // 只保留最低的那个 1，其余清零
```

证明（讲义思路）：设 x 的最低位 1 在第 k 位，则 $\sim x$ 的第 k 位是 0、低位全是 1；加 1 后低位进位到第 k 位变 1、更低位变 0。于是 x 与 $-x$ 只在第 k 位同时为 1。

6.3 基本位操作速查（讲义表格）

目的	表达式	记忆法
置第 k 位	`x |= (1 << k)`	移位 + OR
清第 k 位	`x &= ~(1 << k)`	移位 + 取反 + AND
翻转第 k 位	`x ^= (1 << k)`	移位 + XOR
取位段	`(x >> shift) & mask`	先掩码再移位（或反之）
置位段	`x = (x & ~mask) | (v << shift)`	先清后或
最低位 1	`x & -x`	见上
最低位 1 的下标	`ctz(x)` / de Bruijn	见 6.5

置位段的安全写法（讲义特别标注）：

```
x = (x & ~mask) | ((v << shift) & mask);   // 对 v 也掩码，防止越界污染
```

6.4 无临时变量交换

```
x ^= y;
y ^= x;
x ^= y;
```

正确性来自 XOR 是自逆的：$(x \oplus y) \oplus y = x$。

三步的语义（讲义的分步图）：

```
初始:      x = A,        y = B
step 1:    x = A^B,      y = B
step 2:    x = A^B,      y = (A^B)^B = A
step 3:    x = (A^B)^A = B,  y = A
```

讲义的评语（重要）：

> Poor at exploiting instruction-level parallelism (ILP).

因为三步之间有严格的 RAW 依赖，无法并行。所以在现代 CPU 上，**用临时变量的交换往往更快**（编译器会把它变成寄存器重命名或直接消掉）。这个技巧的真正价值是面试题和面试官的历史遗产。

6.5 无分支最小值与 popcount

6.5.1 无分支最小值

```
min = y ^ ((x ^ y) & -(x < y));
```

推导（讲义原文）：

- C 里 `TRUE` / `FALSE` 是 `1` / `0`；
- 若 `x < y`，则 `-(x<y) = -1 = 全 1`，于是 `y ^ ((x^y) & 全1) = y ^ (x^y) = x`；
- 若 `x >= y`，则 `-(x<y) = 0`，于是 `y ^ 0 = y`。

6.5.2 关于无分支的警告（很重要）

讲义的实测结论：

> This optimization works well on some machines, but on modern machines using **cmov**, the branchless version is usually **slower** than the branching version. Modern compilers can perform this optimization better than you can!

所以第 6 章教的不是“照抄这些技巧”，而是“**知道编译器在做什么，以及什么时候不该手写**”。

6.5.3 popcount 的三种写法

写法 I：反复清除最低位 1

```
int popcount(uint64_t x) {
    int count = 0;
    while (x) { x &= x - 1; count++; }
    return count;
}
```

- 优点：popcount 小时很快；
- 缺点：最坏情况与字长成正比（$\Theta(w)$）。

写法 II：查表

```
static const uint8_t table[256] = { ... };
int popcount(uint64_t x) {
    int c = 0;
    for (int i = 0; i < 8; i++) { c += table[x & 0xFF]; x >>= 8; }
    return c;
}
```

- 缺点：**内存操作是主要瓶颈**。讲义在这一页给了存储层次代价的量级（寄存器 ≪ L1 < L2 < L3 < DRAM），并明确指出“L3 的代价是**按 64 字节 cache line** 计的”。
- 讲义原页的具体拍数受 PDF 字体编码影响无法可靠转录，**以讲义为准**；可对照第 4 章表里可靠的量级：L1 命中约 2 ns、DRAM 约 50 ns（@2.9 GHz 分别约 6 拍与 145 拍，**推断**）。

写法 III：SWAR 并行分治（讲义重点）

```
int popcount(uint64_t x) {
    x =  x - ((x >> 1) & 0x5555555555555555ULL);
    x = (x & 0x3333333333333333ULL) + ((x >> 2) & 0x3333333333333333ULL);
    x = (x + (x >> 4)) & 0x0F0F0F0F0F0F0F0FULL;
    x = (x * 0x0101010101010101ULL) >> 56;
    return (int)x;
}
```

复杂度：$\Theta(\lg w)$，w 为字长。

讲义的逐位示意（用 32 位画）：

```
x                = 11000010010110111111010001111000
x & M0           +  (x>>1)&M0        → 每 2 位一组，组内计数
(x & M1)         +  ((x>>2)&M1)      → 每 4 位一组
(x & M2)         +  ((x>>4)&M2)      → 每 8 位一组
...
最终把所有字节的和收到最低字节
```

注意：讲义特别标注了“避免溢出”——前两步用掩码保证每组不溢出，后面才敢用不加掩码的加法。

写法 IV：硬件指令

```
int __builtin_popcount(unsigned int x);      // GCC/Clang 内建
```

讲义的警告：

> Most modern machines provide popcount instructions, which operate much faster than anything you can code yourself. **Warning**: You may need to enable certain compiler switches to access built-in functions, and your code may be less portable.

（`__builtin_popcount` 通常需要 `-mpopcnt` 或 `-march=native`。）

6.6 de Bruijn 序列求 lg（只对 2 的幂）

问题：$x = 2^k$，求 k。

方法：

```
static const int convert[64] = { 0, 1, 6, 2, 7, 5, 4, 3, ... };  // 64 项
int log2_of_pow2(uint64_t x) {
    return convert[(x * 0x03f79d71b4cb0a89ULL) >> 58];
}
```

原理（讲义原文）：

> A de Bruijn sequence s of length 2^k is a cyclic 0–1 sequence such that each of the 2^k 0–1 strings of length k occurs exactly once as a substring of s.

也就是说：把 x（只有一个 1）乘上那个魔数，**高 6 位会唯一地编码“1 在第几位”**，于是查一次 64 项的表即可。代价：一次乘法 + 一次查表。

6.7 Queens 问题的位向量表示（讲义 + Quiz 1 第 2 题）

N 皇后回溯搜索的关键是**快速判断某个格子是否安全**。朴素做法是 $O(n)$ 扫行/列/对角线；位向量做法是 $O(1)$。

用三个位向量：

位向量	含义
`down`	哪些**列**已有皇后
`left`	哪些**左斜对角线**已被占
`right`	哪些**右斜对角线**已被占

递推时三条线的更新是：

```
down'  = down  | place
left'  = (left  | place) << 1        (往下一行时左斜线左移)
right' = (right | place) >> 1        (往下一行时右斜线右移)
```

Quiz 1 第 2 题给了一个更简洁的写法（答案见 Practice Quiz 1 Solutions）：

```
int32_t queens(uint32_t mask, uint32_t down, uint32_t left, uint32_t right) {
    int32_t count = 0;
    uint32_t possible, place;
    if (down == mask) return 1;                              // (A) mask
    for (possible = ~(down | left | right) & mask;
         possible != 0;                                      // (B) 0
         possible &= ~place) {
        place = possible & (-possible);                      // (C) 最低位 1
        count += queens(mask,
                        down | place,
                        (left | place) << 1,                 // (D)
                        (right | place) >> 1);               // (E)
    }
    return count;
}
```

调用方式：`queens((1 << n) - 1, 0, 0, 0)`。

逐行解读：

- `possible = ~(down|left|right) & mask`：所有“没有被列/两条对角线攻击”的列；
- `possible != 0`：还有候选就继续；
- `place = possible & -possible`：取**最低**的候选位（这决定了搜索顺序，不影响正确性）；
- `possible &= ~place`：把这一位从候选里去掉（而不是 `possible -= place`，两者在这里等价，但前者不依赖无借位）；
- `(left|place) << 1` / `(right|place) >> 1`：**把“本行的攻击线”平移到下一行**。

为什么这比数组快：三个 32 位整数就能表示 32 皇后棋盘的所有约束，全部在寄存器里，无内存访问。

6.8 取模与 2 的幂

```
// 慢：除法
r = x % n;
// 快：仅当 n 是 2 的幂
r = x & (n - 1);
```

前提：$n = 2^k$ 且 $x \geq 0$。对负数，C 的 `%` 语义是“向零取整”，`&` 会给出不同结果——这是一个真实 bug 源。

6.9 向上取整到 2 的幂（讲义的分步图）

```
uint64_t next_pow2(uint64_t n) {
    n--;                        // 关键：处理 n 本身已是 2 的幂的边界
    n |= n >> 1;
    n |= n >> 2;
    n |= n >> 4;
    n |= n >> 8;
    n |= n >> 16;
    n |= n >> 32;
    n++;                        // 得到 2^ceil(lg n)
    return n;
}
```

为什么先减 1：若 `n = 8`，不减则得到 `16`（错）；减 1 后 `7` 被填成 `7`→`15`，加 1 得 `8`（对）。

6.10 本章的实战提醒

提醒	说明
先看汇编再手写	编译器大概率已经做了 popcount/最小值/取模优化
位技巧会破坏 ILP	无临时交换、SWAR 链式依赖都限制乱序
位技巧会破坏向量化	有些写法让编译器无法识别为可向量化，反而更慢
注意有无符号	右移对负数是实现定义/算术移位，混用会出 bug
`-mpopcnt` 一类开关影响可移植性	分发二进制时要小心

第 7 章 机器模型：流水线、乱序、分支预测（L4）

7.1 从 5 级流水线到 Haswell

L4 的叙事结构很讲究：先给你一个**教科书 5 级流水线**，再一层层加上现代特性，每加一层就解释“它解决了什么、代价是什么”。

7.1.1 经典 5 级

```
IF → ID → EX → MA → WB
```

级	做什么
IF（Instruction Fetch）	从内存取指令

级	做什么
ID（Instruction Decode）	译码，确定用哪些部件，取出寄存器参数
EX（Execute）	ALU 运算
MA（Memory Access）	读写数据内存
WB（Write Back）	结果写回寄存器

理想流水线（每拍一条指令完成）：

```
Instr\Cycle  1    2    3    4    5    6    7    8    9
   i        IF   ID   EX   MA   WB
   i+1           IF   ID   EX   MA   WB
   i+2                IF   ID   EX   MA   WB
   i+3                     IF   ID   EX   MA   WB
   i+4                          IF   ID   EX   MA   WB
```

7.1.2 现实：停顿

```
Instr\Cycle  1    2    3    4    5    6    7    8    9
   i        IF   ID   EX   MA   WB
   i+1           IF   ID   ID   ID   EX   MA   WB     ← 数据冒险，停 2 拍
   i+2                IF   IF   IF   ID   EX   MA   WB
   i+3                                  IF   IF   IF   IF
```

7.1.3 三类冒险（讲义定义）

冒险	定义
Structural hazard	两条指令同时要用同一个功能部件
Data hazard	后一条指令依赖前一条的结果
Control hazard	取指/译码被“往哪跳”的决定拖住

数据冒险再细分（讲义用 x86 汇编写）：

类型	含义	例子
True dependence（RAW）	i 写、j 读同一位置	`movsd (%rax),%xmm0` 后 `mulsd %xmm0,%xmm2`
Anti-dependence（WAR）	i 读、j 写同一位置	名字冲突，非真依赖
Output-dependence（WAW）	两条都写同一位置	名字冲突，非真依赖

关键洞察：WAR 与 WAW 是**假依赖**（false dependence），只因为“寄存器名字不够用”才存在。第 7.3 节的重命名就是专门消它们的。

7.2 复杂运算的延迟（讲义表格，可直接背）

操作	x86-64 示例 opcode	延迟（拍）
大多数整数算术/逻辑/移位	`add`、`sub`、`and`、`or`、`xor`、`shl`	1
整数乘法	`imul`	**3**
整数除法	`idiv`	**可变**（很贵）
浮点加法	`addss` / `addsd`	**3**
浮点乘法	`mulss` / `mulsd`	**5**
浮点除法	`divss` / `divsd`	**可变**
浮点 FMA	`vfmadd...`	**5**

怎么用这张表：写内层循环时，`add` 可以每拍发一条（如果有足够 ILP），`mul` / `fma` 要等 5 拍。所以矩阵乘的内层如果是一条依赖链的累加，**吞吐会被延迟卡死**——必须靠多个累加器（unroll + 多 accumulator）来打散依赖。

7.3 超纯量 + 乱序 + 重命名

7.3.1 超纯量（superscalar）

Haswell 的取指/译码**每拍发出 4 个 micro-op**，功能单元分布在 **8 个 port** 上。这意味着理论峰值是 4 IPC（实际受依赖与带宽限制）。

7.3.2 乱序（out-of-order）

讲义的例子（6 条指令，延迟被人为简化）：

#	指令	延迟	依赖
1	`movsd (%rax), %xmm0`	2	—
2	`movsd (%rbx), %xmm2`	5	—
3	`mulsd %xmm0, %xmm2`	3	RAW on 1, 2
4	`addsd %xmm0, %xmm1`	1	RAW on 1；WAR on 2
5	`addsd %xmm1, %xmm1`	1	RAW on 4
6	`mulsd %xmm1, %xmm0`	3	RAW on 5；**WAR on 1**

顺序发射（in-order）：指令 4 必须等指令 2 和 3 都发射完（因为 4 会覆盖 `%xmm1`？不，是因为 4 读 `%xmm1` 而 5 写 `%xmm1`——WAR）。讲义画出 12 拍的执行图。

乱序 + 重命名后：4、5 可以提前，6 不再等 1 的旧值，整体缩到约 8 拍。

7.3.3 寄存器重命名

```
重命名表                          空闲物理寄存器表
ISA 寄存器    物理寄存器          Preg3 Preg8 Preg1 ... Preg0
xmm0          Preg7
xmm1          Preg4
xmm2          Preg2
xmm3          –
```

Haswell 用 **168 个物理寄存器**实现 ISA 的 **16 个**整数寄存器（同样的比例也用于 AVX 寄存器）。重命名表 + 空闲表 + **重排序缓冲（ROB）**共同完成“按序提交、乱序执行”。

讲义的总结（可直接背）：

> The processor uses several strategies to deal with hazards at runtime: – **Stalling**: Freeze earlier pipeline stages. – **Bypassing**: Route the data as soon as it is calculated to an earlier pipeline stage. – **Out-of-order execution**: Execute a later instruction before an earlier one. – **Register renaming**: Remove a dependence by changing its register operands. – **Speculation**: Guess the outcome of the dependence, and restart the calculation only if guess is incorrect.

7.4 分支预测

7.4.1 代价

讲义给的数字（Haswell）：

- **一次 mispredict 代价 15–20 拍**；
- 现代预测器准确率 **>95%**。

算一笔账（推断）：若每 100 条指令有 1 个分支、预测准确率 95%，则每 100 条指令平均 0.05 次 mispredict × 17 拍 ≈ 0.85 拍的损失；若准确率掉到 80%，损失升到 3.4 拍——**IPC 直接掉一半**。这就是为什么第 6 章说“排序测试”“无分支写法”值得讨论。

7.4.2 2 位饱和计数器

讲义给的机制：

编码	含义
11	强 taken

编码	含义
`10`	弱 taken
`01`	弱 not taken
`00`	强 not taken

更新规则：**taken 则加一，not taken 则减一**（饱和）。

为什么用 2 位而不是 1 位：循环的退出分支只在最后一次 not taken，1 位计数器会被“最后一次”翻转，导致下轮循环第一次就预测错；2 位有“ hysteresis”，需要连续两次反向才翻转。

7.4.3 带历史

类型	记录什么
Global history	最近 k 个分支的结果（全芯片）
Local history	某条特定分支最近 k 次的结果

组合方式（讲义列了三种）：

- 历史直接映射到预测
- 历史与指令地址 XOR 后映射（gshare 类）
- 多种策略并行，最后投票

Haswell 的具体结构**未公开**（讲义原话：“Not much is publicly known”），只知道有 BTB（预测间接跳转的目标）、历史上用过两级预测器/循环预测器/混合方案，且 Haswell 重新设计过。

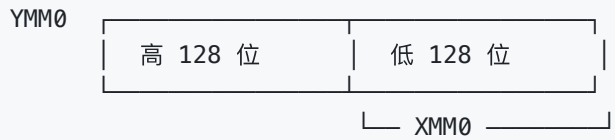
7.5 向量硬件的引入（承上启下）

L4 的后半段转向向量，为第 10 章铺垫。要点：

指令集	寄存器宽度	操作数个数	支持类型
SSE	128 位（XMM）	最多 2	整数、单/双精度浮点
AVX	256 位（YMM）	3（两个源 + 一个独立目的）	单/双精度浮点
AVX2	256 位（YMM）	3	**增加整数向量**
AVX-512	512 位（ZMM）	3+（含掩码）	更多，含 popcount（Haswell 不支持）

3 操作数是 AVX 相对 SSE 的关键改进：SSE 的 `addps %xmm0, %xmm1` 会破坏 `%xmm1` 的原值，导致更多数据搬移；AVX 的 `vaddps %ymm0, %ymm1, %ymm2` 把结果写到独立的目的寄存器，**减少了寄存器压力与假依赖**。

寄存器别名关系（讲义图）：



写 `XMM0` 会清零 `YMM0` 的高 128 位（VEX 编码下），这是一个常被忽略的语义细节。

7.6 本章的实战清单

现象	可能原因	检查方法
IPC 远低于 4	依赖链太长 / 分支不可预测	`perf stat` 看 `branch-misses`、`stalled-cycles-*`
内层循环慢但指令不多	浮点延迟（mul 5 拍）串行化	展开 + 多累加器
加了一行无关代码就变慢	对齐/端口冲突	对照第 3 章的对齐讨论
分支预测器“失灵”	数据相关分支（如快排的 pivot 比较）	改无分支或改数据布局

第 8 章 C 到汇编：LLVM IR 与栈帧（L5）

8.1 为什么要看 IR 而不是直接看汇编

L5 的论点是：**汇编太细，源码太粗，LLVM IR 正好在中间**。

层次	粒度	你在这里能看出什么
C 源码	语句	你写了什么（不是编译器做了什么）
LLVM IR	指令 + SSA 名字	优化后的**数据流**：谁依赖谁、哪些计算被提出循环
x86-64 汇编	机器指令 + 寄存器	实际的寄存器分配、寻址模式、向量寄存器用法

讲义给了一个对照：LLVM IR 里有 `%1 = load double, double* %33, align 8` 这样的指令，**显式带对齐信息**（`align 8`）与别名信息（`!tbaa`、`!alias.scope`）——这些在汇编里就看不见了，而它们恰恰决定了向量化能不能成功（第 9 章）。

8.2 编译流水线

```
源码.c
  │  clang -E
  ▼
预处理后 .i（宏展开、#include 展开、注释剥离）
  │  clang -S -emit-llvm
  ▼
LLVM IR .ll（SSA 形式，物理寄存器尚未分配）
  │  llc / clang -S
  ▼
汇编 .s
  │  as
  ▼
目标文件 .o
  │  ld（或 -flto -fuse-ld=gold）
  ▼
可执行文件
```

讲义强调：**IR 到汇编不是一对一翻译**，中间有一次“指令选择 + 寄存器分配 + 调度”。

8.3 栈帧与调用约定（讲义要点）

概念	内容
栈帧（stack frame）	每次函数调用在栈上分配的一块区域：返回地址、保存的寄存器、局部变量、溢出参数
`%rsp`	栈指针（x86-64），向下增长
参数传递	前 6 个整型参数用 `%rdi, %rsi, %rdx, %rcx, %r8, %r9`；浮点用 `%xmm0–%xmm7`
返回值	整型 `%rax`，浮点 `%xmm0`
对齐	调用前 `%rsp` 必须 16 字节对齐（这也是第 3 章“改文件名影响性能”的物理基础）

为什么这对性能工程重要：HW1 的 `matrix_multiply` 在 `-O0` 下每次 `C->values[i][j] += ...` 都要重新从栈上 load 指针（因为 `-O0` 不做寄存器分配）；`-O3` 会把它们提到寄存器里。这就是 HW1 要求你对比 `-O0` 与 `-O3` 运行时间的原因。

8.4 从 IR 看三个典型优化

优化	IR 上的表现	汇编上的表现
循环不变量外提（LICM）	`getelementptr` 从内层循环移到外层	地址计算指令减少
常量折叠	`add i32 3, 4` 直接变 `i32 7`	常量直接进指令立即数
内联	被调函数的 IR 整体插入调用点	`call` 消失，参数传递消失

读 IR 的最小技能集（够用即可）：

```
%name = opcode type operand, operand      ; 有结果的指令
opcode type operand                       ; 无结果的指令（store、br）
%0, %1, ...                               ; 编译器生成的临时名字
alloca                                    ; 栈上分配
getelementptr (GEP)                       ; 计算结构体/数组元素地址
load / store                              ; 访存
br i1 %cond, label %then, label %else     ; 条件跳转
```

8.5 实战：怎么拿到这三层

```
clang -E foo.c -o foo.i                 # 预处理
clang -S -emit-llvm -O3 foo.c -o foo.ll # LLVM IR
clang -S -O3 foo.c -o foo.s             # 汇编
clang -S -O3 -fverbose-asm foo.c        # 带注释的汇编（可读性最好）
objdump -d --no-show-raw-insn ./foo     # 反汇编（看真实机器码）
```

HW1 的 Makefile 已经封装了 `make`（`-O1` 版本）与 `make DEBUG=1`（`-g -O0`）两种构建；HW3 额外提供 `make ASSEMBLE=1`（只出 `.s`）。

第 9 章 编译器的能与不能（L9）

9.1 编译器是“保守的机械变换器”

L9 开篇的定位：

> Compilers transform code through a sequence of transformation passes. The compiler looks through the code and applies mechanical transformations where it can.

两个关键词：**机械**（mechanical，靠模式匹配与保守分析，不做“聪明的算法设计”）与**保守**（conservative，证明不了就放弃）。

讲义用“循环占了多少时间”做了个思想实验（原文数字被 PDF 渲染损坏，**推断**为 2^{40} 条指令）：

```
2 GHz、16 核、每核每拍 1 条指令、程序有 2^40 条指令（无循环，全是直线代码）
时间 = 2^40 / (16 × 2×10^9) ≈ 34 秒 ≈ 32 秒
```

结论：**哪怕全部并行，直线代码也慢；性能来自循环里的“每次迭代都省一点”**。所以编译器把大量精力放在循环上。

9.2 编译器会做的：内联与 LICM

9.2.1 内联

项	内容
收益	消掉调用开销、暴露跨函数优化（常量传播、CSE）
代价	代码膨胀 → i-cache 压力
编译器的判断依据	**启发式**：函数大小、调用次数、是否递归
你如何干预	`__attribute__((always_inline))`、`__attribute__((no_inline))`、LTO
跨编译单元	默认不能内联；需 `-flto`（+ 可选 `-fuse-ld=gold`）

讲义原话：

> QUESTION: How does the compiler know whether or not inlining a function will hurt performance?
> ANSWER: **It doesn't know.** It makes a best guess based on heuristics.

9.2.2 LICM（循环不变量外提）

n-body 仿真的例子：

```
for (int i = 0; i < n; i++) {
  for (int j = 0; j < n; j++) {
    double dx = pos[j].x - pos[i].x;   // 对 j 循环不变量?
    ...
  }
}
```

编译器的分析：`pos[i]` 的地址计算只依赖外层 `i`，可以提到外层循环；`dx` 不依赖内层 `j`——**但前提是它能证明内层循环不会改写 `pos[i]`**。

这就引出了下一节。

9.3 编译器做不到的：别名分析

9.3.1 别名分析是不可判定的

讲义直说：

> In general, alias analysis is **undecidable** [HU79, R94].

所以编译器只能用各种保守近似：类型信息（`!tbaa`）、`restrict`、`const`、`__attribute__((malloc))`等。

9.3.2 案例 1：向量化被“可能重叠”挡住

```
void test(uint8_t *a, uint8_t *b) {
  for (uint64_t i = 0; i < SIZE; i++)
    a[i] += b[i];
}
```

clang 生成**两个版本**的循环：

```
if (b + SIZE <= a || a + SIZE <= b)   // 两个数组不重叠
    → 向量化版本
else
    → 标量版本
```

讲义把这段条件写成：

```
((A == A+n) && (B == B+n)) || ...
```

（PDF 里的符号被字体编码破坏，本质是“区间不重叠”的检查。）它给出的语义解释才是重点：

> The condition is **true** if arrays A and B **alias**, meaning they overlap in memory.

9.3.3 加 `restrict` 就有单版本

```
void test(uint8_t * restrict a, uint8_t * restrict b);
```

`restrict` 是**你对编译器的承诺**：“基于这个指针的访问不会与基于其他指针的访问重叠”。承诺错了就是未定义行为（UB），编译器可以生成任何结果。

9.3.4 案例 2：`sqrt()` 挡住了 LICM

```
void normalize(double * restrict out, const double * restrict in, size_t n) {
  for (size_t i = 0; i < n; i++)
    out[i] = in[i] / sqrt(dot(in, in, n));   // 每次迭代都调 sqrt(dot)
}
```

编译器**不敢**把 `sqrt(...)` 提出循环，因为它不知道 `sqrt()` 会不会改写 `in[]` 指向的内存（**函数调用是内存屏障**）。

修法：

```
const double norm = sqrt(dot(in, in, n));    // 手动提出
for (size_t i = 0; i < n; i++) out[i] = in[i] / norm;
```

或者给 `dot` 加 `__attribute__((pure))` / `const`。

9.3.5 案例 3：无符号溢出挡住向量化

```
for (size_t i = 0; i < n; i += 4) {
  out[i] = in[i] + in[i+1] + in[i+2] + in[i+3];
}
```

`size_t` 是**无符号**，C 规定无符号溢出**回绕**。于是编译器必须考虑“若 `n` 接近 `SIZE_MAX`，`i += 4` 会回绕成 0，循环变成无限”——向量化会把“无限循环”变成“有限循环”，语义变了，所以**拒绝向量化**。

修法（讲义原文的建议）：

> Use signed integer types, unless you absolutely need an unsigned type, e.g., for bit-hacking.

因为 C 规定**有符号溢出是未定义行为**，编译器可以假设“永不溢出”，于是向量化合法。

9.4 诊断失败的三件套

```
clang -O3 -Rpass=loop-vectorize          # 成功向量化的循环
clang -O3 -Rpass-missed=loop-vectorize   # 失败的循环
clang -O3 -Rpass-analysis=loop-vectorize # 失败的原因（哪条语句）
```

HW3 的 Makefile 把这三条都打开了，所以 `make VECTORIZE=1` 会直接把报告打到终端：

```
example1.c:12:3: remark: vectorized loop (vectorization width: 16, interleaved count: 2)
```

读法：`width: 16` 表示一次处理 16 个元素（`uint8_t` × 16 = 128 位 = SSE），`interleaved count: 2` 表示展开了 2 份（为了打散依赖）。

9.5 编译器的“不可能三角”

你想要	编译器需要	你该给什么
向量化	证明无别名	`restrict`、`const`、`__builtin_assume_aligned`
LICM	证明调用不改内存	提出循环、`pure` 属性、内联
重排浮点运算	允许不满足结合律	`-ffast-math`（**会改变数值结果**）

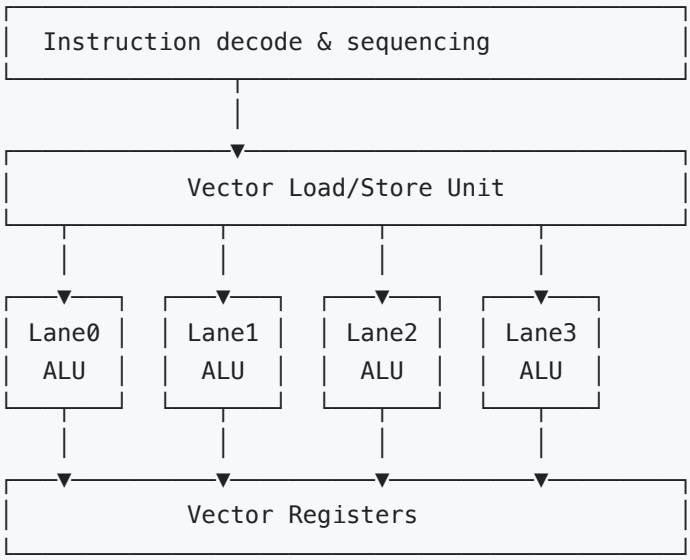
你想要	编译器需要	你该给什么
内联	看到函数体	`-flto`、头文件里定义、`always_inline`

讲义结尾：

> Compiler optimizations can be **fragile**, because analysis can be difficult and the compiler must act conservatively. Take a look at the compiler reports and the LLVM IR to see what the compiler does and **figure out how you can help it out**.

第 10 章 SIMD 与向量化（L4 + HW3）

10.1 向量单元的模型



要点（讲义原文归纳）：

- 所有 lane 锁步（lock-step）执行同一条指令
- 第 i 个元素只能与另一个寄存器的第 i 个元素运算（elementwise）
- 某些架构要求内存操作数对齐（地址必须是向量宽度的倍数）
- 跨 lane 操作（insert/extract/permute/shuffle/gather/scatter）需要额外硬件

10.2 向量化宽度怎么算

数据类型	SSE（128 位）	AVX2（256 位）
`uint8_t` / `char`	16	32
`uint16_t` / `short`	8	16
`uint32_t` / `int` / `float`	4	8
`uint64_t` / `long` / `double`	2	4

这就是 HW3 Write-up 6 的答案框架：`uint32_t` 数组的加速比，若 SSE 是 4×、AVX2 是 8×（理想情况），说明默认向量宽度是 128 位、AVX2 是 256 位。

10.3 对齐：`movdqu` 与 `movdqa`

HW3 用同一段代码的三次编译，把“对齐”讲透了。

10.3.1 无 `restrict`：双版本 + 非对齐指令

```
leaq  65536(%rsi), %rax
cmpq  %rdi, %rax
jbe   .LBB0_2          # 若 b 的末尾 ≤ a 的开头，跳到向量化版本
leaq  65536(%rdi), %rax
cmpq  %rsi, %rax
jbe   .LBB0_2
# 否则走标量版本
```

即使进入向量化分支，用的也是 `movdqu`（unaligned）：

```
movdqu 65536(%rsi,%rax), %xmm0
paddb  %xmm0, %xmm1
```

10.3.2 加 `restrict`：单版本，但仍非对齐

```
.LBB0_1:
    movdqu 65536(%rsi,%rax), %xmm0
    movdqu 65536(%rdi,%rax), %xmm1
    paddb  %xmm0, %xmm1
    ...
```

HW3 的评语：

> The generated code is better, but it is assuming the data are NOT 16 bytes aligned (movdqu is unaligned move). It also means that the loop above can not assume that both arrays are aligned. **If clang were smart, it could test for the cases where the arrays are either both aligned, or both unaligned, and have a fast inner loop. However, it does not do that currently.**

10.3.3 加 `__builtin_assume_aligned`：`movdqa`

```
void test(uint8_t * restrict a, uint8_t * restrict b) {
  a = __builtin_assume_aligned(a, 16);
  b = __builtin_assume_aligned(b, 16);
  for (uint64_t i = 0; i < SIZE; i++) a[i] += b[i];
}
```

```
.LBB0_1:
    movdqa  65536(%rdi,%rax), %xmm0     # 对齐移动
    movdqa  65552(%rdi,%rax), %xmm1
    paddb   65536(%rsi,%rax), %xmm0     # 直接内存操作数，少一次 load
    paddb   65552(%rsi,%rax), %xmm1
    movdqa  %xmm0, 65536(%rdi,%rax)
    movdqa  %xmm1, 65552(%rdi,%rax)
    addq    $64, %rax
    jne     .LBB0_1
```

两处改进：`movdqa`（对齐）+ `paddb` 直接吃内存操作数（省掉一条 `movdqu` 到寄存器的指令）。

10.3.4 三次编译的对比表

版本	别名检查	移动指令	每轮处理	备注
原始	有（双版本）	`movdqu`	64 B	保守
`+restrict`	无（单版本）	`movdqu`	64 B	仍需运行时对齐
`+assume_aligned`	无	`movdqa`	64 B	最快

注意：三次都是 `addq $64`，即每轮处理 64 字节 = 4 个 XMM 寄存器，说明 `interleaved count = 4`。AVX2 版本变成 `addq $128`（4 个 YMM）。

10.4 为什么不是所有循环都能向量化

反例	代码	原因
依赖前一次迭代	`a[i] = b[i+1]`	读的位置在下一轮才写，向量化会读到旧值

反例	代码	原因
归约（浮点）	`y += x[i]`	浮点加法不满足结合律，`-ffast-math` 才允许重排
分支不可消除	`if (y[i] > x[i]) x[i] = y[i];`	可以改写成 `x[i] = max(x[i], y[i])` 才可向量化
变量移位	`C[j] = A[j] << B[j]`	AVX2 之前没有“按变量逐元素移位”的向量指令
跨步访问	`for (j=0;j<N;j+=2) C[j]=A[j]+B[j];`	非单位步长，需要 gather/scatter 或额外打包

10.4.1 `if` → `max`：一次改写换来 `pmaxub`

```
// 不向量化
if (y[i] > x[i]) x[i] = y[i];

// 向量化（clang 生成 pmaxub）
a[i] = (b[i] > a[i]) ? b[i] : a[i];
```

生成的汇编：

```
movdqa  65536(%rsi,%rax), %xmm0
pmaxub  65536(%rdi,%rax), %xmm0
movdqa  %xmm0, 65536(%rdi,%rax)
```

HW3 Write-up 3 的答案：条件表达式是**无副作用的选择**，编译器能把它映射到 `pmaxub`；`if` 语句是**控制流**，编译器要么生成分支，要么生成 select，只有当它能证明“两条路径都无副作用”时才可能转成 select。改写是给编译器“喂”它认识的形式。

10.4.2 浮点归约必须先 `-ffast-math`

```
double test(double * restrict a) {
  double y = 0;
  for (size_t i = 0; i < SIZE; i++) y += x[i];
  return y;
}
```

不加 `-ffast-math`：`addsd` 一条一条加（8 字节一块），因为重排会改变舍入结果。

加 `-ffast-math` 后：

```
addpd  524288(%rdi,%rax,8), %xmm0
addpd  524304(%rdi,%rax,8), %xmm1
...  （两个累加器交替，打散依赖）
addpd  %xmm0, %xmm1
movhlps %xmm0, %xmm0
addpd  %xmm1, %xmm0
```

关键细节：用了**两个累加器**（`%xmm0`、`%xmm1`），因为 `addpd` 延迟是 3 拍——若只用一个累加器，每 3 拍才能加一次，吞吐被延迟卡住。HW3 Write-up 5 让你对比“加/不加 `-ffast-math` 时输出是否相同”，答案通常**不同**（浮点舍入变了）。

10.5 跨步、gather、scatter

操作	含义	x86 支持
单位步长（stride 1）	连续	原生 `movdqa` / `vmovdqu`
小整数步长（stride 2/4）	隔一个取一个	需要 shuffle 或两次 load 拼接
Gather	按索引数组取	AVX2 `vpgatherdd` 等（**很慢**，通常不如标量）
Scatter	按索引数组写	AVX-512 才有；AVX2 无

HW3 Write-up 13/14 的坑：步长循环默认不向量化，需要用 pragma 强制：

```
#pragma clang loop vectorize(enable) vectorize_width(4)
for (j = 0; j < N; j += 2) C[j] = A[j] + B[j];
```

即使强制成功，**加速比往往不如单位步长**，因为需要额外的打包/解包指令。

10.6 打包更小的数据类型（HW3 的核心实验）

同一个 `N`，`__TYPE__` 从 `uint64_t` 换到 `uint8_t`：

类型	每个向量寄存器的元素数（AVX2）	相对加速（趋势）
`uint64_t`	4	最低
`uint32_t`	8	中
`uint16_t`	16	高
`uint8_t`	32	最高

讲义的解释：

> In general, speedup should increase as data type size decreases. This is a fundamental advantage over unvectorized codes where for fixed N, the number of instructions needed to perform elementwise operations over an array of N elements is mostly independent of the data type width.

两个例外（HW3 明确要求你观察）：

1. **`uint64_t` × 乘法**：AWS 机器的向量单元只支持 32×32 位乘法，64 位乘法要被拆成多条 32 位操作，所以 `uint64_t` 的 `*` 反而**没有加速甚至变慢**。
2. **变量移位**：AVX2 才支持逐元素变量移位；只有 SSE 时 `<<` 会被标量化。

10.7 `#pragma clang loop`

pragma	作用
`vectorize(enable)`	强制尝试向量化
`vectorize(disable)`	禁止
`vectorize_width(n)`	指定向量宽度（元素数）
`interleave(n)`	指定展开因子
`unroll(n)`	指定展开次数

讲义提醒：**强制向量化不保证更快**。若循环体里有 gather 或大量 shuffle，强制后可能更慢——所以每次都要实测。

10.8 本章清单：向量化排查顺序

```
1. 加 -Rpass=loop-vectorize，确认"有没有"向量化
2. 没有 → 看 -Rpass-analysis 的原因
     ├─ 别名不确定 → restrict / const
     ├─ 对齐不确定 → __builtin_assume_aligned
     ├─ 浮点重排   → -ffast-math（确认能接受数值变化）
     ├─ 无符号溢出 → 改有符号
     └─ 控制流     → 改写成 select / max / min
3. 有向量化但没加速 → 看汇编
     ├─ 用的是 movdqu 而非 movdqa → 对齐
     ├─ 一条链式依赖 → 多累加器 / interleave
     ├─ 元素太小/太大 → 换数据类型
     └─ 有 gather     → 改数据布局（SoA）
4. 都做了还是不快 → 可能瓶颈根本不在计算，而在内存带宽（第 11 章）
```

第 11 章 缓存与存储层次（L14）

11.1 三种映射方式与地址切分

类型	块的放置	查找代价	地址切分
全相联（fully associative）	任意位置	要搜整个 cache	tag（w） + offset（$\lg B$）
直接映射（direct mapped）	唯一位置	只查一个位置	tag（$w - \lg M$）+ set（$\lg(M/B)$）+ offset（$\lg B$）
k 路组相联（set associative）	组内 k 个位置	查 k 个位置	tag（$w - \lg(M/k)$）+ set（$\lg(M/kB)$）+ offset（$\lg B$）

讲义用一张小例子把三种都画了一遍：$M = 32$ 字节、$B = 4$ 字节、$k = 2$。

地址位数的推导（以 L1 的实测例子为准）：

```
假设：字宽 w = 64 位、cache 大小 M = 32 KB、块大小 B = 64 B、k = 4 路
set 数     = M / (k·B) = 32768 / (4×64) = 128 = 2^7   → set 占 7 位
offset 位数 = lg B = lg 64 = 6
tag 位数    = 64 − 7 − 6 = 51
```

这个推导在 11.4 节的冲突缺失分析里会直接用上。

11.2 缺失的四种类型（讲义定义）

类型	定义	能否避免
Cold miss（冷缺失）	第一次访问该块	不能（除非预取）
Capacity miss（容量缺失）	即使全相联也会被淘汰	只能靠减少工作集/分块
Conflict miss（冲突缺失）	同组块太多；**全相联就不会发生**	靠改变布局/填充/提高相联度
Sharing miss（共享缺失）	另一个处理器独占了该块	见下

Sharing miss 再分两种（这两个词必须分清）：

```
True-sharing miss  : 两个处理器访问同一 cache line 上的「同一份数据」
False-sharing miss : 两个处理器访问同一 cache line 上的「不同数据」
```

伪共享（false sharing）在第 18 章会再出现一次，从分配器角度讨论。

11.3 ideal-cache 模型

L14 引入了一个分析用的抽象机器：

参数	含义
M	cache 大小（字节）
B	cache line 大小（字节）
相联度	**全相联**
替换策略	**最优、全知（omniscient）**
两个度量	work W（普通运行时间）、cache misses Q

为什么要用“最优替换”而不是 LRU：因为最优替换的缺失数是 LRU 的下界，分析起来干净。而下面的引理保证这个抽象不会误导你。

11.3.1 LRU 引理

> **Lemma [ST85].** 若算法在大小为 M 的 ideal cache 上产生 Q 次缺失，则在大小为 $2M$、使用 LRU 的全相联 cache 上，最多产生 $2Q$ 次缺失。

含义：渐近分析里可以把“最优”和“LRU”互换，只差常数 2。这是所有 cache 复杂度分析合法性的基础。

11.3.2 Cache-Miss 引理

> **Lemma.** 若程序读 r 个数据段，第 i 段 s_i 字节，且 $\sum_{i=1}^{r} s_i \le \dots$（容量条件），则所有段都能装进 cache，读它们的缺失数至多 $3N/B$。

证明思路（讲义）：单个段最多 $s_i/B + 2$ 次缺失（首尾可能不对齐），求和后用容量条件放缩。

11.3.3 Tall-cache 假设

$$B^2 < cM \quad (\text{某个足够小的常数 } c \ll 1)$$

为什么需要它：如果 cache“又宽又扁”（B 很大而 M 不大），一个 $n \times n$ 的行主序子矩阵即使 $n^2 < cM$ 也可能装不进去——因为每一行都要占一个独立的 cache line，行数 $\times B$ 才是真实占用。

实测对照：Intel Xeon E5-2666 v3，$B = 64$ B、L1 $= 32$ KB，$B^2 = 4096 \ll 32\text{K}$，满足 tall-cache 假设。

11.3.4 子矩阵引理

> **Lemma.** 若 $n \times n$ 子矩阵 A 读入满足 $B^2 < cM$ 的 tall cache，且 $cM \le n^2 < M/3$，则 A 能装进 cache，读全部元素的缺失数至多 $3n^2/B$。

11.4 矩阵乘法的缺失分析（三种情形）

对最朴素的 `i,j,k` 顺序（行主序），分析矩阵 B 的访问：

情形	条件	缺失数 $Q(n)$	原因
1	$n > cM/B$	$\Theta(n^3)$	B 的每一列都跨 n 个元素，每行都要重新读，**每次访问都缺失**
2	$c'M^{1/2} < n < cM/B$	$\Theta(n^3/B)$	B 能利用空间局部性（同一行的相邻元素在同一块）
3	$n \lesssim cM^{1/2}$	$\Theta(n^2/B)$	三个矩阵全部装得下，每个元素只读一次

换成 `i,k,j` 顺序（第 4 章的版本 4）：$Q(n) = n \cdot \Theta(n^2/B) = \Theta(n^3/B)$——**所有情形都是情形 2**，这就是为什么交换循环能一次拿到 6.5×。

11.4.1 冲突缺失的经典例子（讲义那张图）

设置：4096 列 double（$4096 \times 8 = 2^{15}$ 字节/行）、$M = 32$ KB、$B = 64$ B、$k = 4$ 路。

```
地址切分（w=64）：tag 51 位 | set 7 位 | offset 6 位
取一个 32×32 子矩阵的一列：
  地址 = x, x + 2^15, x + 2·2^15, ..., x + 31·2^15
  由于 2^15 是 set 数（128）的整数倍（2^15 / 128 = 256），
  这 32 个地址的 set 索引完全相同！
```

结果：**32 个元素全部映射到同一个 set**，而该 set 只有 4 路 → 只能装 4 个，其余 28 个每次访问都冲突缺失。

两种修法（讲义给的）：

1. 把子矩阵复制到一个临时的 32×32 矩阵（改变地址分布）；
2. **行填充（padding）**：让每行多几个元素，破坏 2^{15} 的整除关系。

11.5 分块：$\Theta(n^3/(B\sqrt{M}))$

```
for (i1 = 0; i1 < n; i1 += s)
  for (j1 = 0; j1 < n; j1 += s)
    for (k1 = 0; k1 < n; k1 += s)
      for (i = i1; i < i1+s && i < n; i++)
        for (j = j1; j < j1+s && j < n; j++)
          for (k = k1; k < k1+s && k < n; k++)
            C[i*n+j] += A[i*n+k] * B[k*n+j];
```

分析（讲义原文推导）：

- work 不变：$W(n) = \Theta((n/s)^3 \cdot s^3) = \Theta(n^3)$；
- 每次处理三个 $s \times s$ 子矩阵，由子矩阵引理，缺失 $\Theta(s^2/B)$；

- 子矩阵个数 $(n/s)^3$；
- 总缺失：

$$Q(n) = \Theta\left(\left(\frac{n}{s}\right)^3 \cdot \frac{s^2}{B}\right) = \Theta\left(\frac{n^3}{Bs}\right)$$

- 令子矩阵“恰好装进 cache”，即 $s^2 \approx M$，取 $s = \Theta(\sqrt{M})$：

$$\boxed{Q(n) = \Theta\left(\frac{n^3}{B\sqrt{M}}\right)}$$

这是最优的（讲义标注 [HK81]，Hong & Kung 1981）。

工程含义：$Q \propto 1/\sqrt{M}$。所以 L1（32 KB）与 L2（256 KB）的缺失数差 $\sqrt{8} \approx 2.8$ 倍；这也解释了为什么第 4 章里 LLC 缺失能从 8600M 掉到 64M。

11.6 多级分块与“巫毒参数”

层级	参数	循环层数	问题
一级	s	6	一个参数，可扫描
二级	s, t	12	两个参数，**不能二分搜索**（非单峰）
三级	s, t, u	20	三个参数；多道程序环境下有效 cache 大小还在变

讲义的警告：

> Two “voodoo” tuning parameters s and t. **Multidimensional tuning optimization cannot be done with binary search.**

以及三级时的：

> Multiprogrammed environment: Don't know the effective cache size when other jobs are running ⇒ **easy to mistune the parameters!**

11.7 递归分治：同一复杂度，零参数

```
void Rec_Mult(double *C, double *A, double *B, int64_t n, int64_t rowsize) {
  if (n == 1) { C[0] += A[0] * B[0]; return; }   // 实际要粗化
  int64_t d11 = 0;
  int64_t d12 = n/2;
  int64_t d21 = (n/2) * rowsize;
  int64_t d22 = (n/2) * (rowsize+1);
  Rec_Mult(C+d11, A+d11, B+d11, n/2, rowsize);
  Rec_Mult(C+d11, A+d12, B+d21, n/2, rowsize);
  Rec_Mult(C+d12, A+d11, B+d12, n/2, rowsize);
  Rec_Mult(C+d12, A+d12, B+d22, n/2, rowsize);
  Rec_Mult(C+d21, A+d21, B+d11, n/2, rowsize);
  Rec_Mult(C+d21, A+d22, B+d21, n/2, rowsize);
  Rec_Mult(C+d22, A+d21, B+d12, n/2, rowsize);
  Rec_Mult(C+d22, A+d22, B+d22, n/2, rowsize);
}
```

注意 `rowsize` 这个参数：**子矩阵不连续**，必须靠“行跨度”来索引。这是递归版比循环版难写的唯一原因。

11.7.1 递归式与解

$$Q(n) = \begin{cases} \Theta(n^2/B) & n^2 < cM \\ 8Q(n/2) + \Theta(1) & \text{否则} \end{cases}$$

递归树：每层 8 个节点，共 $\lg n - \lg\sqrt{cM}$ 层，叶子数 $8^{\lg n - \lg\sqrt{cM}} = \Theta(n^3/M^{3/2})$，每叶 $\Theta(cM/B)$，乘起来：

$$Q(n) = \Theta\left(\frac{n^3}{BM^{1/2}}\right)$$

与分块完全相同——但不需要知道 M、不需要调 s。

11.7.2 并行版本的缓存行为（重要定理）

> **Theorem.** 设 Q_P 是确定性 Cilk 计算在 P 个处理器（各带私有 cache，大小 M）上的缺失数，S_P 是成功窃取的次数。则在 ideal-cache 模型下：
>
> $$Q_P = Q_1 + O(S_P M/B)$$

证明思路（讲义）：一个 worker 窃取到续延后，它的 cache 在最坏情况下完全冷；但只要再发生 M/B 次冷缺失，它的 cache 就与串行执行时一致了。`cilk_sync` 之后恢复也同样。这两种情况最多出现 $2S_P$ 次。

MORAL（讲义原文）：

> Minimizing cache misses in the **serial elision** essentially minimizes them in parallel executions.

这条极重要：它意味着你**不需要**为并行版本单独设计一套 cache 优化——把串行版本的 cache 行为做好，并行版本自然好（只多 $O(S_P M/B)$）。

11.8 本章自查

问题	答案要点
为什么冲突缺失在全相联 cache 上不存在?	因为没有“同组”概念，任何块都能放任何位置
$s = \sqrt{M}$ 是怎么来的?	让三个 $s \times s$ 子矩阵恰好装进 cache（$3s^2 \le M$，常数吸收）
为什么递归分治不需要 M?	它在**每一级**都分块，总有一级刚好匹配 cache
$Q_P = Q_1 + O(S_P M/B)$ 的实际意义?	串行 cache 优化在并行下基本保留

第 12 章 分块之外：cache-oblivious 算法（L15）

12.1 从 stencil 开始：一个“循环写法必然慢”的例子

一维热传导方程的显式差分：

$$u^{(t+1)}[x] = u^{(t)}[x] + \alpha\left(u^{(t)}[x+1] - 2u^{(t)}[x] + u^{(t)}[x-1]\right)$$

对应代码（讲义用 even-odd 技巧省一半内存）：

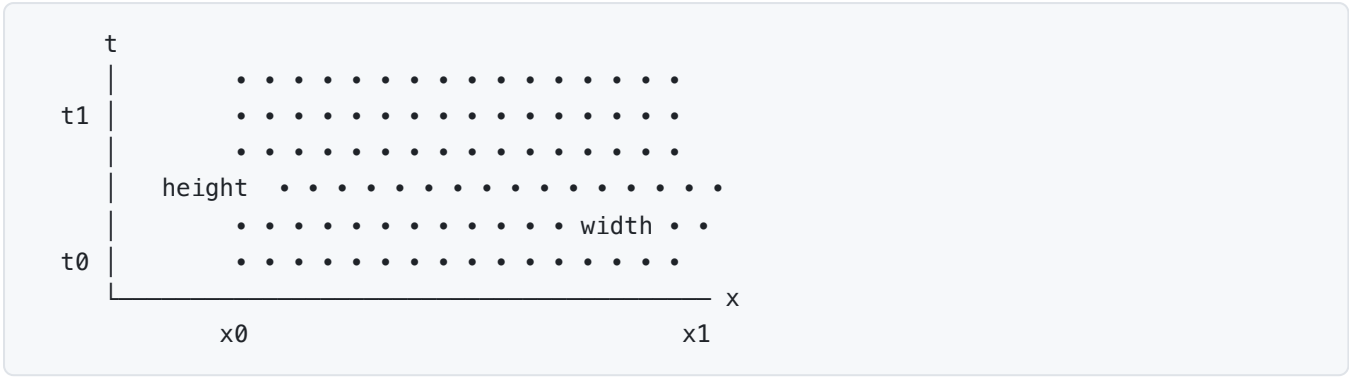
```
double u[2][N];
for (size_t t = 1; t < T-1; ++t)
  for (size_t x = 1; x < N-1; ++x)
    u[(t+1)%2][x] = u[t%2][x] + ALPHA * (u[t%2][x-1] - 2*u[t%2][x] + u[t%2][x+1]);
```

问题：假设 LRU 且 $N > M$，则 $Q = \Theta(NT/B)$——**每一次时间步都要把整个数组从内存读一遍**。

原因：时间维度是**最外层**循环，于是“上一轮算出的值”在下一轮开始时已经被全部逐出。

12.2 时间分块（trapezoidal 递归）

L15 的解法：**在时空平面上递归地划梯形**。



递归规则：把梯形沿**较长的那一维**切成两半；当梯形足够小（能装进 cache）时直接算。

关键：算法**不知道** M 和 B，但递归会在**每一级**产生“刚好装得下”的子问题，所以对**任意** M, B 都接近最优。

12.3 归并排序的 cache 分析

12.3.1 二元归并

merge 子程序的缺失：$\Theta(n/B)$（顺序扫描两个数组）。

$$Q(n) = \begin{cases} \Theta(n/B) & n \le cM \\ 2Q(n/2) + \Theta(n/B) & \text{否则} \end{cases}$$

递归树：每层总代价 n/B，层数 $\lg(n/cM)$，叶子代价 $\Theta(M/B)$：

$$Q(n) = \Theta\left(\frac{n}{B} \lg \frac{n}{M}\right)$$

比值 W/Q：当 $n \gg M$ 时 $W/Q \approx \Theta(B)$；当 $n \approx M$ 时 $W/Q \approx \Theta(B \lg n)$。

12.3.2 多路归并（R 路）

用**锦标赛树（tournament）**合并 R 个子数组：

操作	代价
建树 + 产出第一个元素	$\Theta(R)$
之后每个元素	$\Theta(\lg R)$
合并 n 个元素总计	$\Theta(R + n \lg R) = \Theta(n \lg R)$

work 不变：$W(n) = \Theta(n \lg n)$（与二元归并一样，因为 $\lg R$ 的底数变了但量级同）。

cache 条件：只要 $R < cM/B$，整个锦标赛树 + 每个数组的一个块都能装进 cache，于是

$$Q(n) = \Theta\left(\frac{n}{B} \log_R \frac{n}{M}\right)$$

12.3.3 取最优 R

Q 随 R 增大而减小，最大取 $R = \Theta(M/B)$。代入并用 tall-cache 假设：

$$Q(n) = \Theta\left(\frac{n}{B}\log_{M/B}\frac{n}{M}\right) = \Theta\left(\frac{n\lg n}{B\lg M}\right)$$

与二元归并的对比：

$$Q_{\text{binary}} = \Theta\left(\frac{n\lg n}{B}\right), \qquad Q_{\text{multiway}} = \Theta\left(\frac{n\lg n}{B\lg M}\right)$$

省了 $\Theta(\lg M)$ 倍。讲义给的例子（忽略常数）：

层级	M	节省倍数
L1	2^{15}	15×
L2	2^{18}	18×
L3	2^{23}	23×

但注意：**$R = \Theta(M/B)$ 是显式依赖 M 和 B 的**——所以多路归并是 **cache-aware**，不是 cache-oblivious。

12.4 Funnelsort：真正的 cache-oblivious 排序

[FLPR99] 的方案：

```
1. 把 n 个元素递归地分成 n^(1/3) 组，每组 n^(2/3) 个，各自排序
2. 用一个 n^(1/3)-funnel 把它们合并
```

k-funnel 的保证：合并 k 个有序序列、共 k^3 个元素，缺失数至多

$$O\left(k + \frac{k^3}{B}(1 + \log_M k)\right)$$

（在 tall-cache 假设 $M = \Omega(B^2)$ 下。）

于是 funnelsort 的总缺失为

$$Q(n) = \Theta\left(\frac{n}{B}\log_{M/B}\frac{n}{M}\right)$$

与多路归并同阶，但不依赖 M、B，且这是**渐近最优**的 [AV88]。

k-funnel 的结构（讲义图）：递归的 $\sqrt{k}$ 个子 funnel + 大小为 $k^{3/2}$ 的缓冲区，缓冲区按需 refill，总空间 $O(k^2)$。

12.5 其他 cache-oblivious 结果（讲义表）

问题	缺失复杂度
矩阵转置/加法	$\Theta(1+mn/B)$
Strassen	$\Theta(n+n^2/B+n^{\lg 7}/(BM^{(\lg 7)/2-1}))$
FFT	$\Theta(1+(n/B)(1+\log_M n))$
LUP 分解	$\Theta(1+n^2/B+n^3/(BM^{1/2}))$
有序文件维护	$O(1+\lg^2 n/B)$（摊还）
B 树	INSERT/DELETE $O(1+\log_{B+1} n+\lg^2 n/B)$；SEARCH $O(1+\log_{B+1} n)$
优先队列	$O(1+(1/B)\log_{M/B}(n/B))$

这些结果的意义：它们说明“**不知道 cache 参数也能写出最优的 cache 行为**”不是特例，而是一整套设计方法论。

12.6 cache-oblivious 的五个优点（讲义原文）

- No voodoo tuning parameters
- No explicit knowledge of caches
- Passively autotune
- Handle multilevel caches automatically
- Good in multiprogrammed environments

逐条翻译成工程语言：

优点	工程含义
无巫毒参数	换机器不用重调
不需要知道 cache	代码可移植
被动自动调优	递归深度自动匹配
自动处理多级	一套代码同时优化 L1/L2/L3
多道程序下稳健	别人抢 cache 时不会崩

12.7 讲义对矩阵乘的补充

> The best cache-oblivious codes to date work on **arbitrary rectangular matrices** and perform **binary splitting (instead of 8-way) on the largest of i, j, and k**.

这是从“教科书 8 路分块”到“能用于生产”的关键一步：**沿最长的维度二分**，而不是固定 8 路，这样矩形矩阵也不会退化。

第 13 章 多核编程与缓存一致性（L6）

13.1 为什么需要一致性协议

```
初始：x = 0（内存）
    P0 读到 x=0，缓存了副本
    P1 写 x=1，只写进自己的 cache（写回策略）
    P0 再读 x → 读到 0  ← 错!
```

讲义用一组动画图说明：**写之前必须先让别人的副本失效**（invalidate）。

13.2 MSI 协议

每个 cache line 有一个状态：

状态	含义
M（Modified）	本 cache 修改过；其他 cache 不含此块的 M 或 S 副本
S（Shared）	其他 cache 可能也持有该块的共享副本
I（Invalid）	无效（等同于没有）

核心规则（讲义原文）：

> Before a cache modifies a location, the hardware first invalidates all other copies.

状态迁移（要点版）：

事件	从	到
本地读，未命中，无他人持有	I	S（或 E，若实现 MESI）
本地写，未命中	I	M（先发 invalidate）

事件	从	到
本地写，已在 S	S	M（先发 invalidate，等其他副本确认）
他人写请求命中本地 S	S	I
他人写请求命中本地 M	M	I（先把数据写回/转发）

MESI 比 MSI 多的 E（Exclusive）：只有一个 cache 持有且干净时标 E，此时本地写**不需要**发 invalidate，省一次总线事务。讲义只讲 MSI，MESI 是工程实现（推断补充，不在讲义范围内）。

13.3 四类并发平台（讲义分类）

平台	抽象层次	特点
Pthreads（及 WinAPI threads）	最低	库函数 + 非 C 语义；线程 = 处理器抽象，由 OS 复用
TBB（Threading Building Blocks）	中	C++ 模板库，任务并行，`parallel_for` / `parallel_reduce`
OpenMP	中	编译指示（`#pragma omp`），循环/归约/屏障/原子
Cilk	中高	`cilk_spawn` / `cilk_sync` / `cilk_for`，work-stealing 调度

讲义的定位：

> Programming directly on processor cores is painful and error-prone. A concurrency platform abstracts processor cores, handles synchronization and communication protocols, and performs load balancing.

Cilk 的语义要点（讲义原文，非常重要）：

> Cilk keywords **grant permission** for parallel execution. They **do not command** parallel execution.

也就是说，`cilk_spawn` 是“允许并行”，不是“必须并行”；串行执行时它就是一次普通调用（**serial elision**）。这个性质让 Cilk 程序可以： – 用串行语义推理正确性； – 用 `CILK_NWORKERS=1` 复现串行行为； – 用 Cilkscale 从**一次串行执行**推出 work 与 span。

13.4 Fibonacci：第一个并行程序

```
long fib(int n) {
  if (n < 2) return n;
  long x = cilk_spawn fib(n-1);
  long y = fib(n-2);
  cilk_sync;
  return x + y;
}
```

讲义的免责声明（原文）：

> This recursive program is a poor way to compute the nth Fibonacci number, but it provides for a good didactic example.

为什么适合教学：work 与 span 都能手算（第 14 章），且并行度极高（ϕ^n 级别的 work vs n 级别的 span）。

13.5 本章要点回顾

概念	一句话
一致性（coherence）	保证**单个地址**的读写看起来有序
一致性协议	MSI/MESI，靠 invalidate 实现“写前作废”
伪共享	同一 line 上不同变量被不同核写，触发无谓 invalidate
平台	Pthreads / TBB / OpenMP / Cilk，抽象层次递增
Cilk 语义	关键字是“许可”不是“命令”

第 14 章 多线程算法分析：work、span 与调度（L7 后半 + L8）

14.1 计算 DAG

一个并行程序的执行可以展开成有向无环图 $G = (V, E)$：

概念	定义
strand（顶点）	一段不含 spawn/sync/return 的指令序列
spawn edge	并行分支

概念	定义
call edge	普通函数调用
return edge	返回
continue edge	顺序继续

`cilk_for` 会被编译器改写成递归的分治 + spawn/sync（讲义原文："Loop parallelism is converted to spawns and syncs using recursive divide-and-conquer"）。

14.2 两个度量与两条定律

符号	名称	含义
T_1	work	在 1 个处理器上的执行时间（= DAG 的顶点数，假设每 strand 单位时间）
T_∞	span（critical-path length / computational depth）	沿最长路径的执行时间
T_P	在 P 个处理器上的执行时间	

Work Law：

$$T_P \geq \frac{T_1}{P}$$

Span Law：

$$T_P \geq T_\infty$$

并行度（parallelism）：

$$\bar{P} = \frac{T_1}{T_\infty}$$

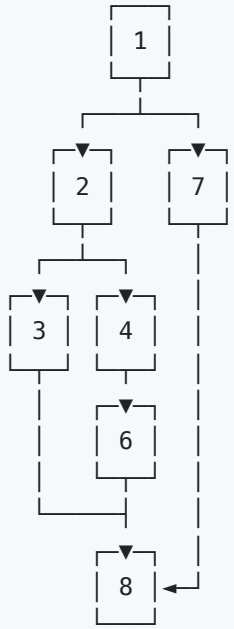
= 沿 span 的平均每步工作量 = 最大可能加速比。

14.3 组合规则

组合	work	span
串行（A 然后 B）	$T_1(A) + T_1(B)$	$T_\infty(A) + T_\infty(B)$
并行（A 与 B）	$T_1(A) + T_1(B)$	$\max\{T_\infty(A), T_\infty(B)\}$

记法：work 永远相加，span 串行相加、并行取 max。

14.4 一个手算例子（讲义原图）



- work $T_1 = 18$（假设每 strand 1 单位）
- span $T_\infty = 9$
- parallelism $= 18/9 = 2$

结论：这台机器上超过 2 个核就基本没用。这就是 span law 的威力。

14.5 Amdahl 定律 = span law 的特例

讲义的 Amdahl 表述：

> If 50% of your application is parallel and 50% is serial, you can't get more than a factor of 2 speedup, no matter how many processors it runs on. In general, if a fraction α of an application must be run serially, the speedup can be at most $1/\alpha$.

与 span law 的关系（推断，但标准）：若串行部分占 α，则 $T_\infty \geq \alpha T_1$，于是 $T_P \geq \alpha T_1$，加速比 $\leq 1/\alpha$。所以 Amdahl 定律不是独立定律，而是 span law 在“串行段”上的推论。

Gustafson 的反驳（推断补充，讲义未展开）：Amdahl 固定问题规模，Gustafson 固定时间、放大问题规模，于是加速比可以随 P 线性增长。两者不矛盾——一个问“同样的问题能多快”，一个问“同样的时间能做多大”。

14.6 各算法的 work/span/parallelism（讲义表，必背）

算法	work	span	parallelism
归并排序	$\Theta(n \lg n)$	$\Theta(\lg^3 n)$	$\Theta(n/\lg^2 n)$
矩阵乘法	$\Theta(n^3)$	$\Theta(\lg n)$	$\Theta(n^3/\lg n)$
Strassen	$\Theta(n^{\lg 7})$	$\Theta(\lg^2 n)$	$\Theta(n^{\lg 7}/\lg^2 n)$
LU 分解	$\Theta(n^3)$	$\Theta(n \lg n)$	$\Theta(n^2/\lg n)$
Tableau 构造	$\Theta(n^2)$	$\Theta(n^{\lg 3})$	$\Theta(n^{2-\lg 3})$
FFT	$\Theta(n \lg n)$	$\Theta(\lg^2 n)$	$\Theta(n/\lg n)$
广度优先搜索	$\Theta(E)$	$\Theta(\lg V)$（用邻接矩阵）	$\Theta(E/\lg V)$

（表中最后一行“BFS”的 span 依赖具体实现；第 22 章的 CSR 版本是 $\Theta(D \lg m)$，D 为直径。）

怎么用这张表：如果你的机器有 64 核，而算法的 parallelism 只有 $\Theta(n/\lg^2 n)$，那么当 n 不够大时（例如 $n = 10^4$ 时 parallelism $\approx 10^4/200 = 50$），**加到 64 核就接近上限了**。

14.7 贪心调度定理

> **Theorem [G68, B75, EZL89].** 任何贪心调度器都能达到
>
> $$T_P \le \frac{T_1}{P} + T_\infty$$

证明（讲义的两句话）：

- **完整步**（complete step，$\ge P$ 个 strand 就绪，跑 P 个）最多 T_1/P 次，因为每步做掉 P 的工作；
- **不完整步**（incomplete step，$< P$ 个就绪，全跑）最多 T_∞ 次，因为每步至少把未执行 DAG 的 span 减少 1。

推论：贪心调度在最优的 2 倍以内。

$$T_P \le \frac{T_1}{P} + T_\infty \le 2 \max\left\{\frac{T_1}{P}, T_\infty\right\} \le 2T_P^*$$

14.8 具体算法的 span 分析（讲义案例）

算法	span 递归式	解
8 路 spawn 矩阵乘	$M_\infty(n) = M_\infty(n/2) + A_\infty(n) + \Theta(1)$	$\Theta(\lg^2 n)$

算法	span 递归式	解
无临时数组的矩阵乘	$M_\infty(n) = 2M_\infty(n/2) + \Theta(1)$	$\Theta(n)$
归并排序（串行 merge）	$T_\infty(n) = T_\infty(n/2) + \Theta(n)$	$\Theta(n)$
并行 merge	$T_\infty(n) = T_\infty(3n/4) + \Theta(\lg n)$	$\Theta(\lg^2 n)$
并行快排（串行 partition）	期望 $T_\infty = \Theta(n)$	并行度 $\Theta(\lg n)$

两个反直觉点：

1. **无临时数组的矩阵乘 span 是 $\Theta(n)$ 而不是 $\Theta(\lg^2 n)$**——因为 8 个递归调用里有 4 个必须等前 4 个写完 `C` 的部分结果（WAR 依赖），于是变成两批，递归式里出现系数 2。
2. **串行 merge 让归并排序的 span 退化到 $\Theta(n)$**——必须写并行 merge 才能拿到 $\Theta(\lg^2 n)$。

14.9 Cilkscale：从一次串行执行推出并行性

工具	开关	输出
Cilksan（竞态检测）	`-fsanitize=cilk`	竞态的文件/行/变量 + 栈回溯
Cilkscale（可扩展性分析）	Tapir/LLVM 提供	work、span、parallelism、加速比上界曲线

讲义的要点：

> Like Cilksan, Cilkscale uses compiler–instrumentation to analyze a **serial execution** of a program.

这意味着：你不需要有 64 核机器就能知道程序在 64 核上能跑多快——只要测出 T_1 和 T_∞，画 $\min(P, T_1/T_\infty)$ 曲线即可。Cilkscale 的输出图上会画出三条线：实测加速比、span law 上界、work law（线性加速）线。

第 15 章 竞态、确定性与锁（L16）

15.1 确定性的定义

> **Definition.** A program is **deterministic** on a given input if every memory location is updated with **the same sequence of values** in every execution.

注意措辞：不要求“两个不同地址的更新顺序相同”，只要求“每个地址看到的更新序列相同”。这比“所有操作全序”弱得多，也正好是能保证可调试性的强度。

两条“规则”（讲义原文，措辞很妙）：

```
Golden Rule:  Never write nondeterministic parallel programs.
Silver Rule:  Never write nondeterministic parallel programs
              – but if you must – always devise a test strategy
              to manage the nondeterminism.
```

Silver Rule 的四种测试策略：

```
• Turn off nondeterminism          (例如 CILK_NWORKERS=1)
• Encapsulate nondeterminism       (把不确定性关进一个小模块)
• Substitute a deterministic alternative  (用确定性算法替换)
• Use analysis tools               (Cilksan / TSAN)
```

15.2 determinacy race 的精确形式

> **Definition.** A determinacy race occurs when two **logically parallel** instructions access the same memory location and **at least one** performs a write.

四种组合（讲义表）：

A	B	是否有竞态	类型
read	read	无	—
read	write	有	read race
write	read	有	read race
write	write	有	write race

“logically parallel”是关键：不是“时间上重叠”，而是“DAG 上不可比”。所以串行执行时也可能存在 determinacy race——只是没暴露。

15.2.1 位域陷阱

```
struct {
  unsigned a : 1;
  unsigned b : 1;
} flags;
```

并行写 `flags.a` 与 `flags.b` **可能产生竞态**，因为两者在同一个机器字里（读–改–写）。讲义的原话：

> Nasty, because it may depend on the compiler optimization level. (Safe on Intel x86-64.)

为什么“在 x86-64 上安全”：因为字节/字的写是原子的（不会读到撕裂的中间值），但**C 标准不保证**，换架构就崩。

15.3 哈希表：一个必然出错的例子

```
// 串行正确
slot = hash(x->key);
x->next = table[slot];
table[slot] = x;

// 两个线程并发执行 → 交错
// T1: slot = hash(x)          T2: slot = hash(y)    (同 slot)
// T1: x->next = table[slot]   T2: y->next = table[slot] (同一个旧头)
// T1: table[slot] = x         T2: table[slot] = y
// 结果：x 丢失
```

修法：每个 slot 一把锁（**锁的粒度 = slot**，不是整表）。

```
lock(&table[slot].L);
x->next = table[slot].head;
table[slot].head = x;
unlock(&table[slot].L);
```

15.4 互斥的实现

15.4.1 自旋锁（讲义汇编）

```
Spin_Mutex:
    cmp     0, mutex          ; 检查是否空闲
    je      Get_Mutex
    pause                     ; x86 提示：让流水线别乱猜
    jmp     Spin_Mutex
Get_Mutex:
    mov     1, %eax
    xchg    mutex, %eax       ; 原子交换
    cmp     0, %eax           ; 检查是否拿到
    jne     Spin_Mutex
Critical_Section:
    <临界区>
    mov     0, mutex          ; 释放
```

为什么要先 `cmp` 再 `xchg`：`xchg` 是原子指令，会锁总线/缓存行；先普通读一次，如果忙就不做 `xchg`，避免无谓的缓存行争抢。

`pause` **的作用**（讲义称“x86 hack to unconfuse pipeline”）：让超线程的另一条流拿到执行资源，并减少自旋带来的功耗与流水线冲刷。

15.4.2 让出锁 vs 自旋锁

类型	阻塞时行为	适用
自旋（spinning）	消耗 CPU 周期空转	临界区极短、核数 ⩽ 线程数
让出（yielding）	`pthread_yield()` 交还 OS	临界区长、可能被换出

15.4.3 竞争锁（competitive mutex）

两个互相竞争的目标：

- 锁一释放就尽快拿到
- 行为要“乖”，少浪费周期

策略：**先自旋，再让出**。自旋多久？**大约一次上下文切换的时间**。这样最坏情况只比最优慢 2 倍：

- 若在自旋期间锁释放 → 最优；
- 若在让出之后才释放 → ⩽ 2× 最优。

讲义的进阶结论：

> A clever randomized algorithm can achieve a competitive ratio of $e/(e-1) \approx 1.58$.

15.5 死锁

15.5.1 三个必要条件（讲义）

```
1. Mutual exclusion   — 每个线程对它持有的资源独占
2. Nonpreemption      — 线程不释放已持有的资源直到用完
3. Circular waiting   — 存在一个线程等待环
```

（经典四条件是加上“请求并保持”；讲义把后两者合并表述。）

15.5.2 锁序定理（防死锁的 silver bullet）

> **Theorem.** 假设可以把所有互斥量线性排序 $L_1 \prec L_2 \prec \cdots \prec L_n$，使得任何线程持有 L_i 再去申请 L_j 时必有 $L_i \prec L_j$。则不会死锁。

证明（讲义）：反设存在等待环，取环中持有“最大”锁 $L_{\max}$ 的线程，它等待下一个线程持有的 L，则必须 $L_{\max} \prec L$，与“最大”矛盾。

15.5.3 哲学家就餐

```
// 会死锁：所有人同时拿起左边的筷子
lock(&chopstick[i].L);
lock(&chopstick[(i+1)%n].L);

// 修法：按全局顺序拿
lock(&chopstick[min(i, (i+1)%n)].L);
lock(&chopstick[max(i, (i+1)%n)].L);
```

15.5.4 Cilk 特有的死锁

```
void main() {
  cilk_spawn foo();
  lock(&L);
  cilk_sync;        // ← 如果 foo() 要拿 L，就死锁
  unlock(&L);
}
```

讲义的规则（三句，必须背）：

```
• Don't hold mutexes across cilk_sync's!
• Hold mutexes only within strands.
• As always, try to avoid nondeterministic programming.
```

为什么：`cilk_sync` 可能让当前 worker 去偷别的活；如果它持着锁去干活，而那个活又要同一把锁，就自己等自己。

15.6 事务内存（TM）

讲义在最后引出 TM 作为“不用锁的原子性”方案：允许执行一个代码块，要么全部生效、要么全部不生效。它列了 TM 的三个实践问题：

```
• Memory management
• Contention
• The ABA problem
```

（ABA 在第 16 章展开。）

第 16 章 无锁同步与内存序（L17）

16.1 顺序一致性（SC）

Lamport 1979 的定义（讲义引用原文）：

> [T]he result of any execution is the same as if the operations of all the processors were executed in some sequential order, and the operations of each individual processor appear in this sequence in the order specified by its program.

三个要点：

1. 所有处理器的指令被交错成一个全局线性序
2. LOAD 读到的是该线性序中「最近一次」STORE 写入的值
3. 硬件可以随便实现，但必须「看起来」满足某个线性序

16.1.1 经典反例（讲义的两处理器例子）

```
初始：x = 0, y = 0
P0:  store x = 1        P1:  store y = 1
     load  y → %eax          load  x → %ebx
```

SC 下不可能出现 `%eax == 0 && %ebx == 0`。但真实机器上可以（因为 store buffer 让 load 越过 store）。讲义列了 6 种交错，说明 SC 排除了那一种。

16.2 为什么现代机器不是 SC



讲义的机制解释：

- store 发得比网络能处理得快 → 需要 store buffer
- load 会阻塞处理器直到满足 → load 优先，绕过 store buffer
- 若 load 的地址与 store buffer 中某项相同，则从 store buffer 取值
- 因此 load 可以绕过「不同地址」的 store

编译器也会重排（讲义明确）：

> No modern-day processor implements sequential consistency. All implement some form of relaxed consistency. Hardware actively reorders instructions. **Compilers may reorder instructions too.**

16.3 x86-64 TSO（Total Store Order）

讲义把 x86-64 的内存序写成一张“house rules”清单（8 条）。由于 PDF 抽取时该页符号被字体编码破坏，这里**只给可确证的核心语义**，逐字措辞请以讲义原页为准：

```
核心：x86-64 的 TSO 只允许一种重排 ——「load 越过它前面的 store」
      （且仅当两者访问不同地址时）。

其余组合都不重排：
  • load  - load    不重排
  • store - store   不重排
  • load  - store   不重排（load 在前时）
  • 带 lock 前缀的指令是全局有序的，且不与任何访存重排
  • 同一地址的 store 之间存在全局全序
  • 内存可见性满足传递性（causality）
```

TSO 比 SC 弱在哪：允许 store 之后的 load 提前完成（因为 store 先进入 store buffer，而 load 可以直接读内存/缓存）。这正是上面那个 `%eax == 0 && %ebx == 0` 的成因。

TSO 比 SC 强在哪：store–store、load–load、load–store 都不重排。所以 x86 是“比较强”的弱内存模型——这也是为什么很多并发 bug 在 x86 上“看起来对”，换到 ARM/POWER 才炸。

16.4 内存栅栏

项	内容
定义	强制栅栏前后的指令满足某个顺序约束的硬件动作
显式指令	x86：`mfence`
隐式	`lock` 前缀的指令、`xchg`、CAS 等都自带 fence 效果
C11 接口	`atomic_thread_fence()`（`<stdatomic.h>`）
代价	**约等于一次 L2 访问**（讲义原文）

“约等于 L2 访问”是个很有用的量级：它意味着 fence 不是免费的，但也不是灾难；一个 fence 大约等于几十条指令。所以“用 fence 保证正确性”通常比“用锁”便宜，但比“什么都不做”贵得多。

16.5 用 C11 恢复正确性

讲义的三个层次：

时代	做法
古代	`volatile` 变量 + 内存栅栏 + 编译器栅栏（`asm volatile("" ::: "memory")`）
现代（C11）	`_Atomic` 变量 + `atomic_load/store/fetch_add/compare_exchange` + `memory_order`

```
#include <stdatomic.h>
atomic_int ready = 0;
int data;

// 生产者
data = 42;
atomic_store_explicit(&ready, 1, memory_order_release);

// 消费者
while (!atomic_load_explicit(&ready, memory_order_acquire)) ;
printf("%d\n", data);   // 保证看到 42
```

release/acquire 的语义（推断补充）：release 保证“之前的写”不会被挪到 store 之后；acquire 保证“之后的读”不会被挪到 load 之前。两者配对形成 happens-before 边。

16.6 CAS 与无锁互斥

16.6.1 CAS 语义（讲义原文）

```
bool CAS(T *addr, T old_val, T new_val) {
  if (*addr == old_val) { *addr = new_val; return true; }
  return false;
}
```

两个关键性质：

```
• Executes atomically
• Implicit fence
```

x86-64 上是 `lock cmpxchg`；C11 里是 `atomic_compare_exchange_weak/strong`。

16.6.2 用 CAS 实现互斥（$\Theta(1)$ 空间）

```
void lock(int *mutex) {
  while (!CAS(mutex, 0, 1)) ;     // 0=空闲，1=占用
}
void unlock(int *mutex) {
  *mutex = 0;
}
```

讲义的定理：

> **Theorem.** 一个 n 线程、无死锁、只用 CAS 的互斥算法，可以用 $\Theta(1)$ 空间实现。

对比另一条（Burns–Lynch）：

> **Theorem [Burns–Lynch].** 任何 n 线程、无死锁、只用 LOAD 和 STORE 的互斥算法，需要 $\Omega(n)$ 空间。

以及：

> **Theorem [Attiya et al.]** 任何 n 线程、无死锁、在现代机器上的互斥算法，**必须**使用 memory fence 或 CAS 这类昂贵操作。

这三条定理合起来说明：硬件提供 CAS 这类原子指令不是“多余功能”，而是**理论上必需的**。

16.7 无锁栈：push/pop 与 ABA

16.7.1 push

```
void push(Node *n) {
  do {
    n->next = head;
  } while (!CAS(&head, n->next, n));
}
```

16.7.2 pop

```
Node *pop(void) {
  do {
    Node *cur = head;
    if (!cur) return NULL;
    Node *next = cur->next;
  } while (!CAS(&head, cur, next));
  return cur;
}
```

16.7.3 为什么“先读再 CAS”更快

讲义原文：

> Compare–and–swap acquires a cache line in **exclusive** mode, invalidating the cache line in other caches. Result: high contention if all processors are doing CAS's to same cache line. **Better way: First read if value at memory location changed before doing CAS, and only do CAS if value didn't change.**

但注意：这个优化**不改变正确性**，只减少失败的 CAS（失败的 CAS 仍然要抢独占，代价高）。所以 pop 的循环里先读 `cur`、比较后再 CAS。

16.7.4 ABA 问题（讲义的分步示例）

```
链表：head → [15] → [94] → [26]

1. 线程 1 开始 pop 15，读到 head = &15、next = &94，然后被换出
2. 线程 2 pop 15          → head = &94
3. 线程 2 pop 94          → head = &26
4. 线程 2 push 一个新节点 7，且复用了 15 那块内存 → head = &7(原15地址)
5. 线程 1 恢复，CAS(head, &15, &94) 成功!
   结果：head = &94，但 94 已经不在链表里 → 链表被破坏
```

根本原因：CAS 只比较**值（地址）**，不比较“这块内存的历史”。

16.7.5 两种解法

解法	做法	问题
版本号（versioning）	把指针和版本号打包进同一个原子字，每次改指针就加一，CAS 同时比较两者	版本号可能要很大（回绕后又 ABA）
回收（reclamation）	有未完成的请求时禁止复用节点（如 hazard pointer、epoch）	实现复杂、有额外开销

16.8 无锁的性质与代价

性质	含义
Lock–free	系统整体总有进展（至少一个线程完成）
Wait–free	每个线程都能在有限步内完成（更强，更难）
Starvation	理论上可能，实践中少见（讲义原话）
实践问题	内存管理、竞争（contention）、ABA

讲义提到事务内存（TM）正在“revolutionizing this area”，但也列了 TM 的同样三个实践问题。

16.9 HW10 的九个问题（锁 vs 无锁 FIFO）

HW10 是纯理论，用两段代码（一个双锁 FIFO、一个无锁 FIFO）问 9–10 个问题。核心考点：

问题类型	你要能答的
为什么用两把锁而不是一把	入队与出队互不阻塞，提高并发度（但要注意 dummy 节点的争用）
每个节点需要几次成功 CAS	无锁入队通常 **1 次**（tail 的 CAS；tail 落后时另一次帮忙）
CAS 失败怎么办	重读、重试；或帮别人完成（helping）
`tail` 能落后多少	任意远；代码必须能“追上”
出队为什么要先读值再改 head	若先改 head 再读值，另一个线程可能已经释放/复用该节点
`count` 怎么解决 ABA	把 count 与指针打包，使“同指针不同代”被 CAS 区分
锁 vs 无锁谁更快	低竞争时无锁快；高竞争时锁（尤其排队锁）可能更好，因为无锁会活锁式重试

第 17 章 串行内存分配器（L11）

17.1 问题定义

C 的 `malloc` / `free` 要满足的语义（Project 3 的接口是它的子集）：

函数	语义
`my_init()`	初始化，成功返回 0，失败 −1
`my_malloc(size)`	返回至少 `size` 字节、**8 字节对齐**、不与任何已分配块重叠的连续内存；`size == 0` 或失败返回 `NULL`
`my_free(ptr)`	释放；`ptr == NULL` 时无效果
`my_realloc(ptr, size)`	`ptr == NULL` 等价于 malloc；`size == 0` 等价于 free；否则保留旧数据（截断或扩展）

17.2 四种基本策略

策略	分配/释放复杂度	空间开销	适用
栈式（stack discipline）	$O(1)$	无	生命周期严格 LIFO

策略	分配/释放复杂度	空间开销	适用
定长 free list	$O(1)$	无（除指针）	所有块同样大小
位图（bitmap）	$O(1)$（需位技巧找空位）	1 位/块	定长，且希望省掉指针
变长 binned free list	$O(1)$（均摊）	内部碎片 ⩽ 2×	通用

17.2.1 定长 free list 的细节（讲义的分步图）

```
初始：free_list → [A] → [B] → [C] → NULL
分配：x = free_list; free_list = free_list->next; return x;
释放：p->next = free_list; free_list = p;
```

讲义的几个坑：

- 释放时若忘记写 `next`，会留下「垃圾指针」
- 分配时若忘更新 `free_list`，会重复分配同一块
- 需要检查 `free_list` 是否为空

17.2.2 位图 + 位技巧

用一位表示一块是否空闲，找空闲块就是找“第一个 0 位”——可以用第 6 章的 `x & -x` 或 `__builtin_ctzll`。

优点：省掉每个块的 `next` 指针（8 字节）；**缺点**：找空位要扫字。

17.3 外部碎片与页级管理

free list 的**时间**很好（$O(1)$），但**空间**有外部碎片：

> Poor spatial locality due to external fragmentation — blocks distributed across virtual memory — which can increase the size of the page table and cause disk thrashing.

讲义的修法（一套组合拳）：

- 每个磁盘页维护一个 `free list`（或位图）
- 从「最满的那一页」的 `free list` 分配
- 释放时归还到该块所在页的 `free list`
- 若某页变空（只剩 `free list` 项），虚拟内存系统可以把它换出而不影响性能

为什么“最满的页”：讲义给了个漂亮的计算——90–10 优于 50–50：

$$\text{两次随机访问命中同一页的概率} = 0.9^2 + 0.1^2 = 0.82 \quad \text{vs} \quad 0.5^2 + 0.5^2 = 0.5$$

17.4 Binned free lists（BFL）

按大小分箱：bin k 存放大小为 2^k 的块。

17.4.1 分配算法

```
allocate x:
  k = ceil(lg x)
  if bin k 非空: 返回其中一块
  else:
    找下一个非空 bin k' > k
    把它的一块分裂成 2^(k'-1), 2^(k'-2), ..., 2^k, 2^k
    把碎片放进对应 bin，返回一块
  若没有更大的块: 向 OS 要更多内存
```

17.4.2 BFL 定理

> **Theorem.** 设程序在任意时刻的最大堆用量为 M。若堆由 BFL 管理，则堆占用的虚拟内存为 $O(M \lg M)$。

证明（讲义）：

- 一个大小为 x 的请求消耗 $2^{\lceil \lg x \rceil} \le 2x$ 的空间；
- 所以大小为 2^k 的块总共占用至多 $2M$；
- 最多有 $\lg M$ 个 free list；
- 相乘得 $O(M \lg M)$。■

讲义的推论：

> In fact, BFL is $\Theta(1)$–**competitive** with the optimal allocator (assuming no coalescing).

“competitive” 的含义：与“最优离线分配器”相比，BFL 的峰值空间只差常数倍。

17.4.3 合并（coalescing）

- 相邻空闲块拼成大块
- 聪明方案存在（如 buddy system），但开销仍高于纯 BFL
- 没有好的理论界能证明合并的有效性
- 实践中合并确实减少碎片，因为堆通常按栈（LIFO）或批方式释放

“没有理论界”这句很重要：它说明合并是**经验性优化**，不是有保证的改进。这也是为什么 Project 3 的评分既要吞吐也要空间利用率——你必须自己权衡。

17.5 为什么“用虚拟内存分配、永不释放”不可行

讲义的问答：

> **Q.** 64 位地址空间要写一个多世纪才能写完（按 40 亿字节/秒），实际上永远不会耗尽虚拟内存。为什么不直接从虚拟内存分配、永不释放？ **A.** 外部碎片会极其可怕。页表的性能会急剧下降，导致磁盘抖动（disk thrashing），因为所有非零内存都必须按页备份到磁盘。

分配器的目标（讲义原文）：

> Use **as little virtual memory as possible**, and try to keep the used portions relatively compact.

17.6 垃圾回收三种方案

方案	机制	优点	缺点
引用计数	每对象计数，归零即回收	增量、无长停顿	**环形结构永远不回收**；计数更新有开销
标记–清除	BFS 标记可达，再扫一遍释放未标记	能回收环	**不处理碎片**；标记阶段要暂停
停止–复制（stop-and-copy）	从 FROM 空间把活对象复制到 TO 空间	复制后**紧凑无碎片**；分配极快（指针 bump）	需要 2× 空间；要更新指针（forwarding pointer）

17.6.1 复制式 GC 的三个细节

1. 用 BFS，TO 空间本身就是 FIFO 队列
2. 对象复制到 TO 时，在 FROM 原位置留 forwarding pointer（隐含“已搬走”标记）
3. 从队列取出对象时，更新它指向的所有指针

“什么时候 FROM 满”：讲义给的策略是“申请等于已用大小的新堆空间，当这块空间被分配完时认为 FROM 满”。于是 GC 代价正比于新堆大小 → **摊还 $O(1)$ 开销**（假设用户程序会触碰所有分配的内存）。

17.6.2 GC 术语表（讲义）

术语	含义
Roots	程序可直接访问的对象（全局、栈等）
Live objects	从 roots 可达的对象
Dead objects	不可达，可回收

术语	含义
Stop-the-world	GC 时暂停整个程序
Incremental	GC 增量执行，停顿有界
Parallel	多个 GC 线程同时工作
Concurrent	至少一个程序线程与一个 GC 线程同时运行

17.6.3 增量复制的两个经典算法

算法	机制	代价
Baker	读屏障：程序访问不在 TO 的对象时，先复制过去再访问	每次读都要检查（贵）
Nettles-O'Toole	写屏障：程序只在 FROM 工作，修改记日志，GC 把日志应用到 TO	写比读少，所以更实用

17.7 本章与作业/项目的对应

内容	对应的动手任务
定长 free list	HW6 Checkoff 1（`simple_allocator.c`，目标利用率 >99%）
缓存对齐分配	HW6 的 `fixed_aligned_allocator.c`
packed / smart allocator	HW6 的 `packed_allocator.c`、`smart_allocator.c`
完整 malloc（含 realloc）	Project 3
GC	只有讲义，无编程作业

第 18 章 并行存储分配与伪共享（L12）

18.1 三个策略与它们的致命缺陷

策略	机制	优点	致命缺陷
全局堆（global heap）	所有线程共享一个堆，用锁或 lock-free 保护	blowup = 1	**慢**（“acquiring a lock is like an L2-cache access”）；竞争抑制扩展性
本地堆（local heaps）	每线程一个堆，无锁	快	**内存漂移**（memory drift）：A 分配的块被 B 释放 → 无界 blowup

策略	机制	优点	致命缺陷
本地所有权（local ownership）	每对象标记 owner，释放时还给 owner 的堆	本地分配/释放快；对伪共享有韧性	释放远程对象需要同步；blowup ≤ P

18.1.1 “为什么小块的竞争更严重”

讲义的问答很精彩：

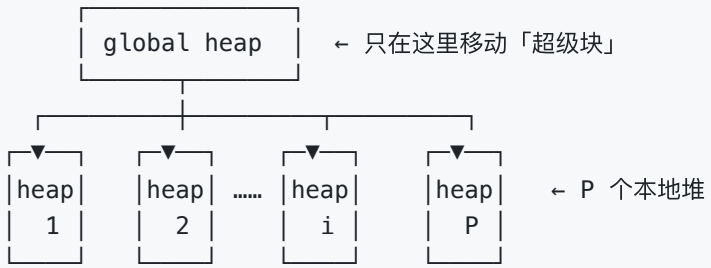
> Q. 锁竞争对“大块”更严重还是“小块”更严重？ A. **小块**。因为用户程序通常会写满它分配的整块内存；大块写起来太慢，线程没法以高频率发起分配请求。相反，大量小块的并行分配会让竞争变得显著。

同一个逻辑也适用于“分配器速度”：

> Q. 最大化大块还是小块的分配速度更重要？ A. **小块。**

18.2 Hoard：全局 + 本地 + 超级块

18.2.1 结构



- 内存按**超级块**（superblock，大小 S）组织；
- **只有超级块**在本地堆与全局堆之间移动；
- 定长分配（讲义假设 w.l.o.g. 所有块同大小）。

18.2.2 分配伪代码（讲义原文）

```
x = malloc() on thread i
if (there exists a free object in heap i) {
  x = an object from the fullest nonfull superblock in i's heap;
} else {
  if (the global heap is empty) B = a new superblock from the OS;
  else                          B = a superblock in the global heap;
  set the owner of B to i;
  x = a free object in B;
}
return x;
```

注意“**fullest nonfull superblock**”（最满的非满超级块）——这和 17.3 节“从最满的页分配”是同一个思想：**尽量把块挤在一起，减少页/超级块的占用数**。

18.2.3 释放与不变量

设 u_i = 堆 i 中在用存储，a_i = 堆 i 拥有的存储。Hoard 维持：

$$u_i \geq \min(a_i - 2S,\ a_i/2)$$

释放流程：

```
free(x) 且 x 属于线程 i:
  把 x 放回 heap i
  if (u_i < min(a_i - 2S, a_i/2))
      把一个「至少半空」的超级块从 heap i 移到 global heap
```

18.2.4 Blowup 定理

> **Lemma.** 全局堆中分配的最大存储不超过本地堆中分配的最大存储。 **Theorem.** 设 U 为用户足迹（footprint），A 为 Hoard 的分配器足迹，则
>
> $$A \leq O(U + SP), \qquad \frac{A}{U} = O\left(1 + \frac{SP}{U}\right)$$

证明（讲义）：不变量 $u_i \geq \min(a_i - 2S, a_i/2)$ 拆成两项看——第一项给出每堆最多 $2S$ 的闲置（共 $O(SP)$），第二项给出“分配量 ⩽ 2× 使用量”（共 $O(U)$）。

含义：Hoard 的 blowup 随 P 线性增长但有界，且当 $U \gg SP$ 时 blowup → 1。

18.2.5 碎片术语表（讲义）

术语	含义
Space overhead	分配器自己的簿记开销
Internal fragmentation	分配的块比请求的大

术语	含义
External fragmentation	无法使用不连续的存储
Blowup	并行分配器比串行分配器多用的空间

18.2.6 分配器速度实测（讲义表，32 线程）

分配器	源代码行数（SLOC）	吞吐（百万次/秒）
Default（glibc）	6,281	**0.97**
Hoard	16,948	**17.1**
jemalloc	22,230	**38.2**
SuperMalloc	3,571	**131.7**

这张表最反直觉的一列是 SLOC：SuperMalloc 用最少的代码（3,571 行，约为 jemalloc 的 1/6）拿到了最高的吞吐（3.4× 于 jemalloc）。讲义的评语是“an up-and-coming contender”。

18.2.7 jemalloc 的三个差异（讲义）

- 每个分配大小有独立的全局锁（而不是一把大锁）
- 总是分配「地址最小」的那个对象（改善局部性）
- 用 `madvise(p, MADV_DONTNEED, ...)` 释放空页（清零但保留虚拟地址有效）

18.3 伪共享（false sharing）

18.3.1 机制图

```
时刻 1：x 和 y 被编译器放进同一个 cache line

        ┌─────────┬─────────┐
        │    x    │    y    │   ← 同一个 64B line
        └─────────┴─────────┘
           P0        P1

时刻 2：P0 写 x  → 该 line 在 P1 处被 invalidate
时刻 3：P1 写 y  → 该 line 在 P0 处被 invalidate
时刻 4：P0 又写 x → 再次 invalidate P1 …
```

结果：两个线程写**不同的变量**，却在缓存一致性协议层面反复争抢同一条 line——称为“伪”共享，因为逻辑上没有共享。

18.3.2 三种成因（讲义）

成因	说明
程序自身	不同线程处理相邻对象
分配器**主动**引发	分配器把不同线程的请求放在同一条 cache line 上
分配器**被动**引发	程序把同一 line 上的对象给不同线程，分配器在对象释放后复用其存储来满足这些线程的新请求

18.3.3 修法

```
// 方案 1：对齐 + 填充到 cache line 大小
struct aligned_padded {
  alignas(64) int counter;
  char pad[64 - sizeof(int)];
};

// 方案 2：每线程一个 padded 副本（常用于计数器）
struct padded_counter {
  alignas(64) _Atomic long v;
  char pad[64 - sizeof(long)];
} counters[MAX_THREADS];

// 方案 3：局部累加，最后归约（最好的方案）
long local = 0;
for (...) local += ...;          // 无共享
atomic_fetch_add(&global, local);
```

讲义的评语：

> The programmer can mitigate this problem by aligning the object on a cache-line boundary and padding out the object to the size of a cache line, but this solution can be **wasteful of space**.

18.4 本章要点

要点	记住什么
三策略	全局堆（慢但有界）、本地堆（快但漂移）、本地所有权（折中）
Hoard	超级块 + 不变量 $u_i \geq \min(a_i - 2S, a_i/2)$ + blowup $O(1 + SP/U)$
速度	SuperMalloc 131.7 M/s » jemalloc 38.2 » Hoard 17.1 » glibc 0.97
伪共享	同 line 不同变量被不同核写；用 padding 或局部累加

第 19 章 Cilk 运行时与工作窃取（L13）

19.1 worker deque

每个 worker 维护一个**双端队列**（deque），里面放“帧”（frame，即待执行的续延）：



角色	操作端	原因
worker（所有者）	bottom（LIFO）	刚 spawn 的子任务最可能还在 cache 里，先做它局部性好
thief（窃取者）	top（FIFO 最老）	偷“最老”的帧 → 它对应最大的子问题 → 窃取开销摊薄

19.2 三条关键性质

1. **thief 必须能“跳进”函数中间**。讲义原文：

   > A thief must be able to jump into the middle of an executing function to steal a continuation.
   > 这就是为什么 Cilk 需要 **cactus stack**（仙人掌栈）：并行分支的栈要能分叉再合并。

2. **work–first 原则**：

   > Optimize for the ordinary serial execution, at the expense of some additional computation in steals. 即：**串行快路径要做到几乎零开销**，把复杂性都推到“发生窃取”的慢路径。

3. **界限**：

$$T_P \le \frac{T_1}{P} + O(T_\infty)$$

（期望意义下；相比贪心调度的 $T_1/P + T_\infty$，多了一个常数因子，但这是**分布式**调度器，不需要全局信息。）

19.3 编译器与运行时的分工（讲义表）

部分	用什么数据结构	负责什么
编译器	少量小结构（worker 记录、栈帧）	无窃取时的快路径
运行时库	较大的结构（deque、cactus stack）	发生窃取时的慢路径

这条分工是“零开销抽象”的典范：不 spawn 的代码与 C 代码等价；spawn 的代价被压缩成“往 deque 里 push 一个帧”。

19.4 工作窃取为什么“缓存友好”（与 11.7.2 呼应）

回忆第 11 章的定理 $Q_P = Q_1 + O(S_P M/B)$：每次成功窃取最多带来 M/B 次冷缺失。由于 $S_P = O(PT_\infty)$（期望），所以

$$Q_P = Q_1 + O\left(\frac{PT_\infty M}{B}\right)$$

实践含义：只要串行版的 cache 行为好（Q_1 小）、且 T_∞ 不太大，并行版的 cache 行为就不会崩。

第 20 章 并行排序与图算法（L15 排序 + L22 图）

20.1 并行排序的三条路线

路线	work	span	备注
并行归并排序（串行 merge）	$\Theta(n \lg n)$	$\Theta(n)$	span 被 merge 卡死，并行度只有 $\Theta(\lg n)$
并行归并排序（并行 merge）	$\Theta(n \lg n)$	$\Theta(\lg^2 n)$	并行度 $\Theta(n/\lg^2 n)$
并行快排（串行 partition）	期望 $\Theta(n \lg n)$	期望 $\Theta(n)$	并行度 $\Theta(\lg n)$，但常数极小、实践很快
并行快排（并行 partition）	期望 $\Theta(n \lg n)$	$\Theta(\lg^2 n)$	需要 prefix sum

并行 merge 的递归式：$T_\infty(n) = T_\infty(3n/4) + \Theta(\lg n)$，解为 $\Theta(\lg^2 n)$。**为什么是 $3n/4$**：二分查找分割点时，最坏情况会把问题切成 $3:1$。

20.2 图的表示：CSR（第 5 章的回归）

```
顶点 ID:   0    1    2    3    4
Offsets:   0    2    5    5    6    7
Edges:     1 3  2 3 4  2 2  2 4
```

操作	邻接矩阵	邻接表（指针）	CSR	边表
判断边 (u,v)	$O(1)$	$O(\deg u)$	$O(\deg u)$	$O(m)$
遍历 u 的邻居	$O(n)$	$O(\deg u)$	$O(\deg u)$	$O(m)$
加边	$O(1)$	$O(1)$	$O(1)$/改造	$O(m+n)$
删边	$O(1)$	$O(m)$	$O(\deg u)$	$O(m+n)$
空间	$O(n^2)$	$O(n+m)$	$O(n+m)$	$O(m)$

讲义的结论：**静态图用 CSR**，因为遍历邻居是图算法的核心操作。

20.3 并行 BFS（自顶向下）

```
BFS(Offsets, Edges, source) {
  parent, frontier, frontierNext, degrees;
  cilk_for (i = 0; i < n; i++) parent[i] = -1;
  frontier[0] = source; frontierSize = 1; parent[source] = source;
  while (frontierSize > 0) {
    cilk_for (i = 0; i < frontierSize; i++)
      degrees[i] = Offsets[frontier[i]+1] - Offsets[frontier[i]];
    prefix_sum(degrees, frontierSize);          // 计算每个顶点的写偏移
    cilk_for (i = 0; i < frontierSize; i++) {
      v = frontier[i]; index = degrees[i]; d = Offsets[v+1] - Offsets[v];
      for (j = 0; j < d; j++) {
        ngh = Edges[Offsets[v] + j];
        if (parent[ngh] == -1 && CAS(&parent[ngh], -1, v)) {
          frontierNext[index + j] = ngh;
        } else {
          frontierNext[index + j] = -1;
        }
      }
    }
    filter_out_minus_one(frontierNext, &frontier, &frontierSize);
  }
}
```

20.3.1 work/span 分析（讲义）

量	值	理由
迭代次数	$\le D$（直径）	每轮推进一层

量	值	理由
每轮 span	$\Theta(\lg m)$	`cilk_for` + prefix sum + filter
总 span	$\Theta(D \lg m)$	
总 work	$\Theta(n+m)$	前缀和 $\Theta(n)$、filter $\Theta(m)$

20.3.2 实测（讲义）

随机图 $n = 10^7$、$m = 10^8$（每顶点 10 条边），40 核 2 路超线程：

指标	值
相对 1 线程的加速比	**31.8×**
相对串行 BFS 的 1 线程性能	并行版**慢 54%**

最后一行非常重要：并行 BFS 的单线程性能比精心优化的串行 BFS 差一半（因为多了 prefix sum、filter、CAS 开销）。但 40 核时仍然是它快——这就是“work 效率 vs 并行度”的取舍。

20.3.3 确定性版本（讲义的两阶段法）

上面代码里的 `CAS` 导致**同一个 `parent[ngh]` 可能被多个顶点竞争写入**，最终写的是谁取决于调度 → **非确定性**。

确定性版本用两阶段：

```
阶段 1：对每个邻居调用 writeMin(&parent[ngh], v)
        （原子地写入「较小」的值）
阶段 2：检查 parent[ngh] == v，若是则把 ngh 放进 frontierNext
        （只有「赢了」的那个顶点才负责推进）
```

```
void writeMin(int *addr, int newval) {
  int oldval = *addr;
  while (newval < oldval) {
    if (CAS(addr, oldval, newval)) return;
    oldval = *addr;
  }
}
```

代价：讲义实测，在 32 核上确定性版本比非确定性版本**慢 5–20%**。

20.4 方向优化 BFS（direction-optimizing）

20.4.1 观察：frontier 的形状

讲义画了两张图（随机图 vs 幂律图）：**frontier 先指数增长，然后迅速收缩**。也就是说，大部分工作发生在“frontier 很大、且大部分顶点已被访问”的阶段。

20.4.2 两种 BFS

方向	做法	何时更优
Top-down（自顶向下）	从 frontier 出发，遍历每个顶点的**出边**	frontier 小
Bottom-up（自底向上）	遍历**所有未访问顶点**，检查它是否有邻居在 frontier 里	frontier 大、且多数顶点已访问

Bottom-up 的收益：它把“遍历边”换成“遍历未访问顶点 × 检查邻居”，当 frontier 很大时能**减少遍历的边数**（因为一旦发现一个在 frontier 的邻居就可以停）。

20.4.3 切换策略

讲义的规则（Beamer, Asanovic, Patterson, SC12）：

```
if (frontier 的规模 / 顶点数 超过阈值)  用 bottom-up
else                                    用 top-down
```

具体阈值是调参的（又是一个巫毒参数）。讲义还提到表示方式也要跟着换：**top-down 用稀疏表示（frontier 数组），bottom-up 用稠密表示（位图/布尔数组）**。

第 21 章 DSL 与自动调优（L18）

21.1 三个 DSL 案例

DSL	领域	关键思想
GraphIt	图算法	算法与调度**分离**：用户写 `updateEdge` / `updateVertex`，调度用 `schedule:` 块描述
Halide	图像处理	算法（what）与调度（how：tile/vectorize/parallel）分离；60 行实现 3×3 模糊
OpenTuner	通用	用集成（ensemble）搜索技术自动找参数

21.1.1 GraphIt 的例子（讲义原文）

```
func updateEdge(src: Vertex, dst: Vertex)
  new_rank[dst] += old_rank[src] / out_degree[src]
end
func updateVertex(v: Vertex)
  new_rank[v] = beta_score + 0.85*new_rank[v];
  old_rank[v] = new_rank[v];
  new_rank[v] = 0;
end
func main()
  for i in 1:11
    #s1# edges.apply(updateEdge);
    vertices.apply(updateVertex);
  end
end
```

调度（同一算法的不同性能）：

```
schedule:
  program->configApplyDirection("s1", "DensePull");
  program->configApplyParallelization("s1", "dynamic-vertex-parallel");
  program->configApplyNumSSG("s1", "fixed-vertex-count", 10);
```

这就是 PageRank。方向（push/pull）、并行化方式、分块方式都被显式暴露成“调度旋钮”，于是可以**自动搜索**。

21.2 自动调优为什么必要

讲义给的对比（GraphIt 的调度搜索）：

方法	时间
穷举搜索	**几周**
OpenTuner	**< 2 小时**

而且：

> Finds a few schedules that **outperform hand-tuned schedules**.

21.2.1 集成搜索（ensemble）

```
Nelder-Mead Simplex（爬山）   ┐
Differential Evolution        │
Particle Swarm Optimization   ├─► AUC Bandit 元技术
Model Driven Optimization     ┘        │
                                       ├─ 把测试预算分配给各子技术
                                       └─ 各技术之间共享结果
```

关键机制：元技术（meta-technique）动态决定“下一个预算给哪个搜索器”，并根据历史收益（AUC）做多臂老虎机决策。

讲义的坦承：

> Hard to write a single autotuner that performs well in different domains.

21.3 自动调优的风险（Project 3 的明文警告）

Project 3 的 Autotuning 一节写得很直接：

> **Be careful**: Autotuners are designed to find the most optimal configuration for whatever test cases you run them on, sometimes in ways that a human coder would never consider. If your tests are not general enough, you can **overfit** your allocator to the autotuning inputs. This may cause your code to run slower on the somewhat-different test suite that will actually be used for grading.

翻译：自动调优会把“测试集”当成“真实分布”。测试集不代表性 → 过拟合 → 评分时翻车。

第 22 章 案例研究：TSP 与 Leiserchess（L21 / L19 / L20）

22.1 Bentley 的 TSP：六个版本的演化

问题：n 个城市，枚举所有 $n!$ 个排列找最短回路。测试机：Intel Core i7-6500U @ 2.50 GHz。

22.1.1 算法 1：朴素枚举

```
排列数 n!，每个排列算 n 次距离 → n × n! 次距离计算
N=8: 0.05 s   N=9: 0.34 s   N=10: 3.97 s   N=11: 45.47 s
N=12: 9 分钟  N=13: 2 小时
```

22.1.2 常数因子：编译器

N	无优化	`-O3`
10	3.97 s	0.12 s
11	45.47 s	1.62 s
12	9 min	19.81 s
13	2 hours	4.3 min

约 25 倍（在这台机器上；在 Raspberry Pi 3 上约 6 倍）。

22.1.3 常数因子：硬件

N	1997 Pentium Pro @200MHz	2016 i7-6500U @2.5GHz
10	20.9 s	0.12 s
11	4 min	1.62 s
12	48 min	19.81 s
13	10 hrs	4.3 min

约 150 倍，其中讲义拆成“×12 来自主频、×12 来自更宽的数据通路与更深的流水线”。

22.1.4 常数因子：内部改动

改动	效果
`double` → `float` 或定点 `int`；用最快的整型宽度	明显
预计算 n^2 个距离存表（`distarr[i][j]`）	2.5–3×

22.1.5 算法 3/4：减少排列数与增量求和

版本	优化	距离计算次数
Alg 1	朴素	$n \times n!$
Alg 2	预计算距离表	$n \times n!$（但每次更便宜）
Alg 3	固定最后一个城市	$n \times (n-1)! = n!$
Alg 4	携带部分和	$\approx (1+\epsilon)(n-1)!$

实测（AMD E-450 @1.65GHz，秒）：

n	Alg 1	Alg 2	Alg 3	Alg 4
10	0.83	0.25	0.02	—
11	10.28	3.22	0.28	0.25
12	120	33.50	3.47	2.59
13	1560	430	34.84	21.02

Alg 1 → Alg 4 在 n=13 上是 74 倍，靠的是“少算”而不是“算得快”。

22.1.6 算法 5：剪枝（本课程最重要的一课）

```
if (sum > minsum) return;       // 部分和已经超过当前最优，直接放弃
```

n	Alg 4	Alg 5
12	0.59	**0.01**
13	5.14	**0.12**
14	54.44	**0.41**
15	—	0.99
16	—	3.92
17	—	39.00

（Intel Core i7–3630QM @2.40GHz，秒。）

n=14 时 133 倍，而且 Alg 5 能解到 n=17 而 Alg 4 到 14 就崩了。

讲义的结论：

> The key to speed is **pruning the search**.

22.1.7 算法 6：更强的下界（剩余点的 MST）

```
if (sum + mstdist(mask | bit[p[m]]) > minsum) return;
```

用 **Prim–Dijkstra 求剩余点的最小生成树**作为下界。MST 是 TSP 回路的下界（回路去掉一条边就是生成树），所以这个剪枝是**安全**的（不会剪掉最优解）。

22.1.8 阶乘有多大（讲义的震撼教育）

$$52! \approx 8.0658 \times 10^{67} \approx 2^{225}$$

讲义的比喻：设一个 $52!$ 纳秒的倒计时，站在赤道上每百万年走一步；绕地球一圈后从太平洋取一滴水；太平洋空了铺一张纸，再装满；纸堆到月球时——倒计时才刚要走完。

> Fact: the universe is about $26! = 4.03 \times 10^{26}$ nanoseconds old. Moral: we're going to have to **ignore** some of the possible tours.

22.2 Leiserchess：从“能跑”到“能赢”

22.2.1 游戏规则要点（L19）

项	内容
双方	Tangerine（先手）与 Lavender
棋子	每方 7 个 Pawn + 1 个 King；Pawn 有 4 个朝向
回合	每回合两部分：**移动** + **开激光**
移动	basic（旋转 90/180/270 或走到相邻 8 格之一，不能又转又走）或 swap（与相邻敌子交换，然后必须再走一步 basic）
激光	从己方 King 射出，经 Pawn 长边反射；击中对方 King 即胜；击中 Pawn 则将其移除
Ko 规则	禁止“撤销对手上一步”式的重复

22.2.2 引擎结构（L19）

组件	要点
棋盘表示	8×8，带 **sentinels**（哨兵边界）避免走法生成时越界检查
走法表示	压进 **28 位**的 `int32_t`
走法生成	生成所有合法走法
perft	统计给定深度的叶子节点数，**走法生成器的黄金测试**
静态评估	从棋盘特征（`LCOVERAGE` 等）打分
搜索	**PVS**（Principal Variation Search），alpha–beta 的改进
置换表	缓存已搜索的位置
killer moves	加速剪枝的启发式

22.2.3 PVS 与剪枝

alpha–beta 的核心：维护窗口 $[\alpha, \beta]$，一旦 $\alpha \geq \beta$ 就剪枝。

```
α = 当前 maximizer 已知的最好下界
β = 当前 minimizer 已知的最好上界
α ≥ β → 这个分支不可能影响结果 → 剪掉
```

PVS 的改进：假设“第一个走法就是最好的”，后续走法先用**零宽窗口**（null window）快速验证；只有当它确实更好时才用完整窗口重搜。

22.2.4 投机并行（L20）

并行化 alpha–beta 的根本困难：**剪枝依赖于搜索顺序**。若并行搜索多个子树，某个子树可能“本可以被剪掉”，于是：

风险	后果
多搜了本可剪掉的子树	浪费工作（性能下降，但**结果仍正确**）
α/β 的并发读写	**竞态**，可能得到错误结果
置换表的并发访问	需要同步或每线程一份
killer moves 的并发更新	同上

讲义把它归为“**投机并行**”（speculative parallelism）：你可能做了无用功，但若猜对了就赚到。L20 用“阈值求和”的例子讲了这个模式：

```
// 串行：超过阈值就提前返回
for (i = 0; i < n; i++) { sum += a[i]; if (sum > threshold) return; }

// 并行：分治，但每个子问题都可能“提前放弃”
if (sum_so_far > threshold) return;      // 用共享的“已超阈值”标志
```

22.2.5 P4 的评分逻辑

Project 4 的最终衡量不是“每秒多少节点”，而是**比赛胜率**：

> The final measure of performance will be your program's ability to beat a selection of game–playing programs in round–robin tournaments using **Fischer Time Control**.

而且讲义明确警告：

> You might be tempted to evaluate your performance by running the UCI command `go depth` and looking at the **Nodes Per Second (NPS)** metric. This is generally a **bad idea**, as NPS may vary at different points in the game depending on the complexity of the search tree traversal. It's better when possible to run many games with the autotester and see whether there's a significant difference in **ELO rating**.

这是一个重要的“指标选择”教训：代理指标（NPS）与真实目标（ELO）不一致时，优化代理指标可能损害真实目标。

第 23 章 十份作业精讲（HW1–HW10）

23.1 作业总览

HW	标题（OCW 原文）	类型	主要工具	与项目的衔接
1	Basic Tools, C Primer	动手	`clang`、`gdb`、`valgrind`、`gcov`、`clint.py`	P1 的前置
2	Profiling	动手	`perf`、`cachegrind`	P1/P4 的定位手段
3	Vectorization	动手	`-Rpass`、AVX2、`restrict`	P1 的位并行、P4 的评估函数
4	Cilk, Reducer Hyperobjects	动手	Cilk、Cilksan、Cilkscale	P2 的并行化
5	Theory of Performance Engineering	理论	CLRS 第 27 章	Quiz 1
6	Simple Storage Allocation	动手	`mdriver`、OpenTuner	**P3 的缩小版**
7	Dynamic Analysis Tools	动手	CSI 框架、LTO	P4 的性能建模
8	Caching and Cachegrind	理论+分析	cachegrind	P4 的 cache 优化
9	Deterministic Execution	动手（环境）	ASLR、`setarch`、Makefile	P4 的可复现调试
10	Data Synchronization	理论	锁 vs 无锁 FIFO	P4 的并行正确性

23.2 HW1：工具链与 C 基本功

23.2.1 覆盖的知识点

小节	内容	关键陷阱
预处理	`#define`、`#ifndef NDEBUG`、`clang -E`	宏参数必须加括号
类型	`short/int/long/long long`、`uint64_t`、`__int128`	精度由标准弱定义，跨平台不同
指针	`int*`、`char**`、`const char*` vs `char* const`	复合类型**从右往左读**
传参	按值传递	`swap(int,int)` 不会交换
构建	`make`、`make DEBUG=1`、`make ASAN=1`	Debug 版才有断言

小节	内容	关键陷阱
调试	`gdb --args`、`bt`、`frame`、`p`	要有 `-g` 才能看行号
内存检查	AddressSanitizer、Valgrind	Valgrind 需要 `-p` 才会输出结果矩阵
覆盖率	`-fprofile-arcs -ftest-coverage`、`llvm-cov gcov`	`#####` 表示从未执行
性能	`awsrun`、改变循环顺序、`-O0` vs `-O3`	个人 VM 上测不准

23.2.2 几个必会的 gdb 操作

```
gdb --args ./matrix_multiply
(gdb) run                    # 或 r
(gdb) backtrace              # 或 bt，看栈
(gdb) frame 3                # 或 f 3，切到第 3 帧
(gdb) print A->cols          # 或 p A->cols
(gdb) print B->values[k]     # 看指针
(gdb) quit
```

讲义的关键教学点：段错误只告诉你“哪一行崩了”，但 `print` 出来的 `B->values[4] == 0x0` 才告诉你“B 没有第 5 行”——**从崩溃点回溯到根因**才是调试。

23.2.3 覆盖率的一个反直觉用途

讲义让你用 `llvm-cov gcov matrix_multiply.c` 看每行执行次数：

```
  10:    93: for (i = 0; i < A->rows; i++) {
  40:    94:   for (j = 0; j < B->cols; j++) {
 160:    95:     for (k = 0; k < A->cols; k++) {
  64:    96:       C->values[i][j] += A->values[i][k] * B->values[k][j];
```

执行次数最多的行就是最该优化的行——这是“用覆盖率找热点”的穷人版 profiler。

23.3 HW2：性能分析与归并排序的五轮优化

23.3.1 工具用法

```
# perf
make isort DEBUG=1                  # -g 让 perf 能标注源码
awsrun perf record ./isort 10000 10
aws-perf-report                     # 注意：不要用 awsrun 跑这个

# cachegrind
valgrind --tool=cachegrind --branch-sim=yes ./sort 10000 10
```

读 cachegrind 输出：`D1` = L1 数据 cache，`LLd` = 最后一级数据 cache（通常是 L3）。`lscpu` 可以查本机 cache 参数。

23.3.2 五轮优化的顺序（这就是“优化优先级”的实证）

轮次	文件	优化	你要解释什么
1	`sort_i.c`	`inline` 一个或多个函数（含递归函数）	为什么递归内联有副作用
2	`sort_p.c`	指针代替数组下标	为什么指针可能更快（少一次乘法/加法）
3	`sort_c.c`	粗化递归（base case 用插入排序）	base case 大小怎么定
4	`sort_m.c`	只用一块临时空间（另一半用输入数组）	编译器能否自动做这个优化
5	`sort_f.c`	临时空间只分配一次	用 profiling 数据解释收益

测试框架的约定：每个 `sort_x.c` 里的函数签名必须与 `sort_a` 一致，且在 `main.c` 的 `testFunc` 里取消注释对应行。**不要删** `static`（讲义明文要求）。

23.3.3 递归内联的坑（Write-up 3）

内联递归函数时，编译器只会**展开固定次数**，然后继续递归。副作用：

- 代码膨胀 → i-cache 压力
- 展开次数是编译器启发式决定的，你可能得不到想要的
- 用 `cachegrind` 看 `I1`（指令 cache）缺失是否上升

23.4 HW3：向量化的 15 个 write-up

这一份作业是**向量化的百科全书**，逐题对应一个概念：

Write-up	主题	关键结论
1	为什么用负偏移（`-65536`）而不是 0	让寻址模式的 disp 落在有符号 32 位内，且便于循环末尾判断
2	AVX2 版本仍非对齐	加 `__builtin_assume_aligned(a, 32)`
3	`if` vs `?:` 生成完全不同的汇编	`?:` 可映射到 `pmaxub`
4	`a[i] = b[i+1]` 为什么不向量化	跨迭代依赖
5	`-ffast-math` 前后输出不同	浮点重排改变舍入
6	`uint32_t` 的加速比 → 推断向量宽度	SSE 4×、AVX2 8×
7	哪条指令做向量加法	SSE `paddb`、AVX2 `vpaddb`
8	`*` 与 `<<` 的差异	变量移位需 AVX2

Write-up	主题	关键结论
9	类型宽度对加速比的影响	越窄越快
10	`uint64_t` 乘法的异常	硬件只支持 32×32 乘法
11	perf 报告里时间花在哪	乘法的瓶颈可能在数据搬运
12	运行时循环边界	增加尾部处理代码
13	步长循环不向量化	需要 pragma 强制
14	`vectorize_width(2)` 的效果	宽度不是越大越好
15	归约的向量化	多累加器 + 最后水平相加

23.5 HW4：Cilk、Reducer 与竞态检测

内容	要点
`cilk_spawn` / `cilk_sync`	前者是“许可”，后者是“等所有子任务”
Cilksan	`-fsanitize=cilk`，能报出竞态的文件/行/变量
Cilkscale	从**一次串行执行**算 work/span/parallelism
Reducer	把“对共享变量的归约”变成“每 worker 一份私有副本 + 合并”
`awsrun8`	8 核机器；并行性能数字必须在这里取

Reducer 的动机（讲义）：并行累加 `sum += x` 会产生 write race；加锁太贵；reducer 让每个 worker 累加到自己的副本，`cilk_sync` 时再合并——**无锁、确定性**。

23.6 HW5：性能工程理论

纯理论，指定读 CLRS 第 27 章（多线程算法）。典型题型：

- 给一段 Cilk 代码，画 computation dag
- 算 T1、T∞、parallelism
- 用 Master Method 解递归式
- 判断某个调度是否贪心、给出 T_P 的上界

23.7 HW6：简单存储分配器（P3 的缩小版）

23.7.1 三个任务

任务	文件	目标
定长 free list	`simple_allocator.c`	`trace_c0_v0` 的利用率 > **99%**
参数化 `BLOCK_SIZE`	同上	支持 `trace_c1_v0`（请求 4096 字节）
自动调优	`opentuner_params.py` + Makefile	自动找到 1024 / 4096
缓存对齐	`fixed_aligned_allocator.c`	对象不跨 cache line
packed / smart	`packed_allocator.c`、`smart_allocator.c`	用位运算压缩指针

23.7.2 mdriver 的用法

```
./mdriver -g -v -B -f rec_traces/trace_c0_v0
```

参数	作用
`-f <trace>`	指定单个 trace
`-v`	每个 trace 的性能明细表
`-V`	更详细（读文件时也打印）
`-B`	只测这一个 trace 的分数
`-g`	生成 autograder 用的汇总

23.7.3 OpenTuner 的调用

```
./opentuner_run.py --test-limit=300 --no-dups --display-frequency=20 \
                   --trace-file=<trace-file>
```

`BLOCK_SIZE` 作为 **2 的幂**参数，范围 2^5 到 2^{15}。

23.8 HW7：自己写动态分析工具（CSI 框架）

这是全课程**最“元”**的一份作业：你不使用 profiler，而是**造一个 profiler**。

23.8.1 CSI 的四个概念

概念	含义
Hooks	编译器在 IR 的六类对象处自动插入的回调：load、store、basic block、函数入口、函数出口、call
CSI ID	每个 IR 对象的唯一整数 ID（每类一个扁平 ID 空间，可直接做数组下标）
Forensic tables	编译期生成的静态信息表（源码位置、基本块里的 IR 指令数）
Properties	每个 IR 对象的编译期常量属性（是否 const、是否可能在栈上、对齐）

23.8.2 CSI-cov 的 40 行

```
long *block_executed = NULL;
csi_id_t num_basic_blocks = 0;

void report() {
  csi_id_t executed = 0;
  for (csi_id_t i = 0; i < num_basic_blocks; i++) {
    if (block_executed[i] > 0) executed++;
    const source_loc_t *loc = __csi_get_bb_source_loc(i);
    if (loc) fprintf(stderr, "%s:%d:%d executed %d times\n",
                     loc->filename, loc->line_number, loc->column_number,
                     block_executed[i]);
  }
  fprintf(stderr, "Total: %ld of %ld basic blocks executed\n", executed, num_basic_blocks);
  free(block_executed);
}

void __csi_init() { atexit(report); }

void __csi_unit_init(const char *const name, const instrumentation_counts_t counts) {
  block_executed = realloc(block_executed,
                           (num_basic_blocks + counts.num_bb) * sizeof(long));
  memset(block_executed + num_basic_blocks, 0, counts.num_bb * sizeof(long));
  num_basic_blocks += counts.num_bb;
}

void __csi_bb_entry(const csi_id_t bb_id, const bb_prop_t prop) {
  block_executed[bb_id]++;
}
```

为什么它能“免费”：未实现的 hook 默认是空函数，靠 LTO 在链接时把空调用当死代码删掉。

23.8.3 线性回归任务

用 CSI 工具数“IR 指令数”与“函数调用数”，再拟合

$$T = a \cdot I + b \cdot F$$

估出**一次函数调用的代价 b**。第二个任务把 F 换成“`cilk_spawn` 次数” S，估出**一次 spawn 的代价**。

讲义的工程细节（很实用）：

> **Note**: The prototype CSI implementation currently contains a bug that affects its instrumentation on simple recursive functions, such as `fib`. To get around this bug, declare all counter variables in your tool as `volatile`.

23.9 HW8：cache 复杂度分析

任务	内容
Checkoff 1	分析**矩形**矩阵乘（$m \times n$ 乘 $n \times r$）在四种情形下的缺失复杂度
Tableau	给 $A[i][j] = f(A[i-1][j-1], A[i][j-1], A[i-1][j])$ 写迭代版与分治版，比较 cache 复杂度

Tableau 的巧思：只需要最终值 $A(N-1, N-1)$，所以只需 $2N-1$ 的空间，用

```
#define A(i, j) A[N + (i) - (j) - 1]
```

把二维索引映射到一维对角线数组。

23.10 HW9：确定性执行

23.10.1 三类非确定性来源

来源	例子	消除方法
未定义的值	未初始化的 `int i`	初始化
地址	ASLR 让指针高位随机	`setarch x86_64 -R ./prog`
环境	环境块长度改变栈对齐	固定环境（`env -i`）

23.10.2 哈希表的两个任务

任务	目标	手段
`hashtable1000-good`	让 1000 次运行**全部成功**	消除非确定性（初始化、固定 hash 种子）
`hashtable1-bad`	让 1 次运行**必定失败**	构造触发 bug 的输入/环境

这份作业的教学目的：把“偶发 bug”变成“必现 bug”，才能调试。

23.11 HW10：锁 vs 无锁 FIFO

纯理论，两个实现（双锁 FIFO、无锁 FIFO），10 个问题。核心考点已在 16.9 节列出。

最值得记住的一题（第 6 题）：

> Which implementation do you expect to run faster — the lock-based or the lock-free?

标准答案的框架（不唯一）：**低竞争时无锁快**（无阻塞、无上下文切换）；**高竞争时排队锁可能更快**（无锁会反复 CAS 失败重试，浪费带宽）。所以“无锁一定更快”是错的。

第 24 章 四个项目精讲（P1–P4）

24.1 项目总览与共同规则

项目	主题	语言/平台	Beta 权重	Final 权重	特殊规则
P1	Bit Hacks（位旋转）	C，纯串行	20%	30%	**禁用 intrinsic 与内联汇编**
P2	Collision Detection	C + Open Cilk	36%	39%	必须报出与参考实现**相同**的碰撞数
P3	Serial malloc	C，单核	27%	40%	全局/静态标量 ⩽ **512 字节**；禁用 `malloc` 等
P4	Leiserchess	C/C++，UCI	—	—	Beta I **禁用并行**

三个所有项目共享的机制：

1. Beta → 代码评审（MITPOSSE）→ Final 三段式
2. Beta 提交匿名公开，可以互相学习（但不能抄代码）
3. 每周四提交个人进度报告（含 `loc_summary` 的行数统计）

24.2 Project 1：Bit Hacks

24.2.1 任务

在 `everybit/bitarray.c` 里重写 `bitarray_rotate()`。位数组按 **8 位/字节**打包，通过 `bitarray_get()` / `bitarray_set()` 访问。

24.2.2 三种算法（讲义原文都给了）

算法	做法	辅助空间	缺点
1. 辅助数组	把 `a` 拷出来，`b` 前移，`a` 放到后面	$O(n)$	大数组时 cache 差
2. 循环置换	每个 bit 前进 k 位（模 n），走完所有环	$O(1)$	**访存分散**，cache 极差
3. 三次反转	$(a^R b^R)^R = ba$	$O(1)$	每个 bit 动两次

第 3 种的正确性：设原串为 ab，目标是 ba。反转整个串得 $b^R a^R$，再分别反转…… 实际上标准做法是：

```
reverse(a); reverse(b); reverse(ab)
```

等价地：$(a^R b^R)^R = ba$。

24.2.3 命令行与测试

```
./everybit -s          # 0.01 秒量级的基准
./everybit -m          # 0.1 秒
./everybit -l          # 1 秒（评分用这个量级）
./everybit -t tests/default       # 跑全部测试
./everybit -n 1 -t tests/default  # 只跑第 1 个
make test              # 跑 tests/ 下所有文件
make testquiet         # 只输出失败的
```

24.2.4 评分表（讲义原文）

项	Beta	Final
Performance	20%	30%
Test coverage	12%	—
Correctness	7%	10%
Addressing MITPOSSE comments	—	10%
Write-up	3%	5%
Team contract	3%	—
合计	**45%**	**55%**

测试覆盖的玩法：你要提交 `tests/mytests` 里的 **20 个最好的测试**，课程会用它们去跑**其他所有组**的代码——你抓到的 bug 越多，分越高。

24.2.5 实现顺序建议（推断，但符合 handout 逻辑）

1. 先让 bitarray_rotate 正确（可以用最笨的逐位循环）
2. 写测试：n=0/1、rotate=0、rotate=n、rotate>n、跨字节边界、非 8 倍数长度
3. 换算法 1（辅助数组）→ 测性能
4. 换算法 3（三次反转）→ 测性能
5. 优化反转函数本身（按字节反转 + 位反转表）
6. 用 perf 看热点：是 bitarray_get/set 的开销还是循环控制

24.3 Project 2：Collision Detection

24.3.1 任务分解

阶段 A（串行）：用四叉树把 $O(N^2)$ 的两两检测降到接近 $O(N \log N)$
阶段 B（并行）：用 Cilk 并行化四叉树遍历 + 用 reducer 保护碰撞列表

24.3.2 四叉树的关键设计问题

问题	讲义给的提示
节点容量 r 取多少	可调参数，要实验
线段的“扫掠区域”	应存**平行四边形**（线段在一个时间步内扫过的区域），不是线段本身
线段跨多个分区怎么办	提示：**不是所有线段都必须存在叶节点**（可存在内部节点）
每步重建还是增量更新	先重建（简单），再考虑增量
最大深度	加一个深度上限，避免退化

24.3.3 正确性要求

- 碰撞数必须与参考实现完全相同
- 碰撞处理顺序由 line ID 排序决定（否则离散时间下结果不同）
- 推荐“引用测试”：assert(intersect_orig(...) == intersect_new(...))

24.3.4 性能要求与命令

```
./screensaver 4000                  # 串行基准（默认 input/mit.in）
./screensaver -g 4000               # 带图形（不要用于测性能）
awsrun  ./screensaver 1000          # 单核（注意：不是 8 核！）
awsrun8 ./screensaver 4000          # 8 核
awsrun8 CILK_NWORKERS=8 ./screensaver 4000
```

讲义的提醒：

> The awsrun nodes, however, are configured to time out after **40 seconds**.

所以先用 1000 帧测，快了再上 4000 帧。

24.3.5 评分表

项	Beta	Final
Performance	36%	39%
Team contract	3%	—
Addressing MITPOSSE comments	—	10%
Write-up	5%	5%
Screensaver aesthetic test inputs	1%	1%
合计	**45%**	**55%**

“输入美学”占 1%：你要设计既“有挑战性”又“好看”的输入文件。

24.4 Project 3：Serial Dynamic Memory Allocation

24.4.1 接口与文件

文件	你能否修改	作用
`allocator.c`	✅	你要实现的 `my_init/my_malloc/my_free/my_realloc/check`
`allocator_interface.h`	❌	接口声明
`memlib.c`	❌	模拟内存系统（`mem_sbrk`、`mem_heap_lo/hi`、`mem_heapsize`、`mem_pagesize`）
`mdriver.c`	❌	trace 驱动
`validator.h`	✅	你要写的**外部验证器**（跨组互测）
`opentuner_params.py`	✅	自动调优参数
`traces/`	❌	trace 文件

24.4.2 四条硬规则

1. 不得调用任何内存管理库函数/系统调用（malloc/calloc/free/realloc/sbrk/brk 及其变体）
2. 不得使用并行化（这是单核项目）
3. 全局与静态标量变量 + 复合数据结构的总大小 ≤ 512 字节
4. 所有堆内存必须通过 memlib 接口申请

第 3 条是最狠的：512 字节意味着你**不能**用全局大数组做 free list 头表——必须把元数据放进堆里。

24.4.3 评分公式（讲义原文）

空间利用率：

$$U = \frac{\max(M, 40\text{KB})}{\max(H, 40\text{KB})}$$

其中 M = 当前已分配字节数，H = 你的堆大小。

性能指数：

$$P = \exp\Big(w \log U + (1-w) \log\big(\min(1, T/T_{\text{libc}})\big)\Big), \qquad w = 0.5$$

怎么读这个公式：

- U 越接近 1 越好（无碎片）；
- $T/T_{\text{libc}} \geq 1$ 时第二项饱和为 0（**超过 libc 不再加分**）；
- $w = 0.5$ 表示“空间与吞吐同等重要”；
- 理想 $P = 1$（100%）。

工程含义：**盲目追求吞吐会毁掉 U**。例如“每个请求都分配一个 4KB 页”能拿到极高吞吐，但 U 会趋近 0，P 直接崩。

24.4.4 mdriver 参数

参数	作用
`-t <dir>`	trace 目录
`-f <file>`	单个 trace
`-v`	每个 trace 的性能明细
`-V`	更详细（读文件时也打印）
`-c`	每次操作后调用你的 `check()`
`-h`	帮助

24.4.5 评分表

项	Beta	Final
Performance（正确代码的 P）	27%	40%
Team contract	3%	—
Addressing MITPOSSE comments	—	10%
Test coverage（你的 validator 抓别人的 bug）	12%	—
Write-up	3%	5%
合计	**45%**	**55%**

24.4.6 实现路线（推断，按 handout 的 hints 排序）

1. 先写 `check()`（堆一致性检查器）——讲义说这一步能省下后面大量时间
2. 实现最简单的隐式空闲链表 + 边界标记（boundary tag）+ 立即合并
3. 用 `mdriver -f` 跑最小的 trace，确认正确
4. 加分离空闲链表（segregated free lists，按大小分箱）
5. 优化 `realloc`（能原地扩就原地扩，否则 `malloc+memcpy+free`）
6. 调整箱数与分箱边界 → 用 OpenTuner 搜
7. 用 `perf` 看热点（通常在 `find_fit` 和 `coalesce`）

24.5 Project 4：Leiserchess

24.5.1 三个阶段

阶段	允许并行	你要做什么
Beta I	❌	纯串行优化：perft 验证正确性、profiling、改数据结构
Beta II	✅	并行搜索（注意 α/β 竞态、置换表同步）
Final	✅	综合 + 应对更大机器

24.5.2 优化清单（讲义明确提到“low-hanging fruit”）

方向	具体做法
棋盘表示	改成 bitboard；**尽早改**（会影响全局）
走法生成	预计算、查表、减少分支
评估函数	简化特征、查表、向量化
置换表	提高命中率、改进替换策略、减少锁

方向	具体做法
搜索	更好的走法排序（killer moves、history heuristic）
内存	减少 malloc、用对象池
并行	根节点分裂（root splitting）、Young Brothers Wait

讲义的核心提醒：

> We recommend that you make any changes to the **board representation** as early as possible. These changes may affect the behavior of other parts of the code base.

以及：

> **Never lose sight of the big picture.** Once you find one hotspot, it's easy to get sucked into optimizing it to death.

24.5.3 测试工具

工具	用途
`perft <depth>`	统计叶子数，验证走法生成
autotester	跑大量对局，算 ELO
参考 bot 对拍	同深度下应走同一步
FEN 串	快速复位到出 bug 的局面
scrimmage server	看对局，发现明显 bug

24.5.4 一个反直觉的权衡

> A program using a **less-accurate evaluator** can be stronger than a program with a more accurate one if it is **faster and can search deeper** in the tree.

含义：评估函数的“精度”与“速度”要一起优化；多搜一层往往比评估更准更值钱。

第 25 章 考试结构与应试（Quizzes）

25.1 形式（四份 Practice Quiz 首页共同写明）

项	内容
时长	**80 分钟**
满分	**80 分**（约 1 分/分钟）
形式	闭卷（closed book）
允许携带	**一张手写双面 crib sheet**（约 $8.5'' \times 11''$）+ 课上发的 **Master Method 卡片**
题型	True/False（**答错扣分**）、多选、填空、代码补全、复杂度分析
页数	13–17 页

True/False 的规则要特别注意：

> Some of the questions are true/false, and some are multiple choice. You need not explain these answers unless you wish to receive partial credit if your answer is wrong. For these kinds of questions, **incorrect answers will be penalized**, so do not guess unless you are reasonably sure.

所以策略是：**确定的才答，不确定的留空**（留空通常不扣分，猜测期望值为负）。

25.2 四份练习卷的题目分布（可当复习清单）

Quiz 1（14 页，80 分）

#	题目	分值	对应讲次
0	每页写名字	2	—
1	True or False（8 小题）	16	全局
2	Bit Tricks for the Queens Problem（5 空）	13	L3
3	Parallel PageRank（3 小题）	9	L7/L22
4	Bitonic Sort（6 小题）	18	L7/L8
5	Heap Allocation（3 小题）	13	L11
6	Compilers & Assembly（2 小题）	9	L4/L9

Quiz 2（17 页，80 分）

#	题目	分值	对应讲次
0	每页写名字	1	—
1	True or False（9 小题）	18	全局
2	Heap space for Cilk programs（3 小题）	7	L12
3	Concurrent free lists（2 小题）	8	L11/L12
4	Parallel Strassen（5 小题）	15	L8
5	Space usage for Strassen（6 小题）	18	L12
6	Cache-oblivious Strassen（3 小题）	13	L15

Quiz 3（14 页，80 分）

#	题目	分值	对应讲次
0	每页写名字	1	—
1	True or False（10 小题）	20	全局
2	Caching in Matrix Multiplication（3 小题）	12	L14
3	Dining Philosophers（4 小题）	16	L16
4	Caching for Karatsuba（4 小题）	17	L14/L8
5	Lock-free FIFO Queue（1 小题）	14	L17

Quiz 4（13 页，80 分）

#	题目	分值	对应讲次
0	每页写名字	2	—
1	True or False（8 小题）	16	全局
2	Back Of The Envelope（3 小题）	10	L1/L10
3	Branch Prediction（3 小题）	9	L4
4	Variable Byte Compression（2 小题）	11	L3
5	LLVM（6 小题）	12	L5/L9
6	Algorithm Analysis（4 小题）	20	L8

25.3 从题目分布推出的复习优先级

主题	出现次数	优先级
复杂度/cache 分析（含 Strassen、Karatsuba、矩阵乘）	4 份都出现	★★★
True/False 全局判断	4 份共 35 小题	★★★
位技巧	2 份	★★
并发/锁/死锁	3 份	★★★
架构/汇编/分支预测	2 份	★★
分配器	2 份	★★

每份卷子的第 1 题都是 True/False，占 16–20 分——这是性价比最高的复习对象，因为它覆盖全课程的概念辨析。

25.4 crib sheet 建议（推断，但符合“一张双面”的约束）

```
正面：
  ├─ Master Method 三种 CASE
  ├─ work/span 定律 + 贪心界 + parallelism 表
  ├─ cache 公式：Q = Θ(n³/(B√M))、归并 Θ((n/B)lg(n/M))、多路 Θ(n lg n/(B lg M))
  ├─ 地址切分：tag | set | offset 的位数公式
  └─ 常见延迟表（L1 2ns、DRAM 50ns、mispredict 15–20 拍、fence ≈ L2）
背面：
  ├─ Bentley 21 条规则（只需关键词）
  ├─ 位技巧：x&-x、popcount SWAR、next_pow2、de Bruijn
  ├─ 分配器：BFL O(M lg M)、Hoard u_i ≥ min(a_i−2S, a_i/2)、blowup O(1+SP/U)
  ├─ 无锁：CAS、ABA、TSO 8 条
  └─ 向量：SSE 128/AVX2 256、每类型元素数
```

第 26 章 与已有笔记的衔接

26.1 与本目录其他笔记的对应关系

本课程主题	对应笔记	衔接点
L4 流水线/冒险/分支预测	`CSAPP_深入理解计算机系统_学习笔记.md` 第 4 章	CSAPP 讲 Y86-64 的 5 级流水线与冒险；6.172 讲**真实** Haswell 的乱序/重命名/预测

本课程主题	对应笔记	衔接点
L14 缓存与存储层次	CSAPP 第 6 章	CSAPP 讲 AMAT、组相联、写策略；6.172 讲**ideal-cache 模型**与**缺失复杂度的闭式解**
L10 测量	CSAPP 第 5 章（性能优化）	CSAPP 讲“消除循环低效、提高 ILP、用分块”；6.172 讲**怎么证明你变快了**
L6–L8、L16–L17 并行	`CMU_15418_并行计算机体系结构与编程_学习笔记.md`	15-418 更偏 GPU/异构与并行模式；6.172 更偏**work/span 理论与 Cilk 运行时**
L14/L15 cache	`UCB_CS61C_计算机体系结构_学习笔记.md`	CS61C 讲 cache 的组织与 AMAT；6.172 讲**算法层面的 cache 复杂度**
L4/L10 向量化	`UIUC_ECE408_应用并行编程_学习笔记.md`	ECE408 讲 CUDA 的 SIMT；6.172 讲 **CPU 的 SIMD**——两者是“同一思想的两种硬件实现”

26.2 三组“同一概念、不同抽象层”的对照

26.2.1 分块（blocking / tiling）

课程	说法	粒度
CS61C	cache 的组相联、替换策略	硬件
CSAPP	“分块提高时间局部性”	代码
6.172	$Q(n)=\Theta(n^3/(B\sqrt{M}))$，$s=\Theta(\sqrt{M})$	**渐近分析**
ECE408	shared memory tiling	GPU block

26.2.2 并行

课程	抽象	度量
CSAPP	线程、信号量、竞争	正确性
15-418	并行模式（map/reduce/scan）、GPU	吞吐/带宽
6.172	work/span DAG、贪心调度	$T_P \le T_1/P + T_\infty$
CS61C	流水线/多核/OpenMP 简介	结构性

26.2.3 性能度量

课程	指标
CS61C	CPU time = IC × CPI × 周期

课程	指标
CSAPP	CPE（每元素周期数）
6.172	$T = aI + bC$ 的**回归模型**、roofline、%peak

26.3 建议的交叉阅读顺序

```
若你已读完 CSAPP 第 4–6 章：
    6.172 第 3 章（测量）→ 第 4 章（矩阵乘案例）→ 第 11–12 章（cache）
    → 第 7 章（机器模型，回头补 CSAPP 第 4 章的空白）

若你已读完 15-418/CS149：
    6.172 第 14 章（work/span）→ 第 13 章（Cilk）→ 第 15–16 章（竞态/无锁）
    → 第 18 章（并行分配器）

若你已读完 CS61C：
    6.172 第 11 章（ideal cache）→ 第 12 章（cache-oblivious）
    → 第 17 章（分配器）→ 第 23 章（HW6/P3）
```

第 27 章 复习清单与自测题

27.1 复习清单（按“能不能手算”分级）

A 级：必须能手算/手写

#	内容	检验方式
A1	峰值 FLOPS 公式	给主频/核数/FMA 宽度，算峰值
A2	地址切分（tag/set/offset）	给 M, B, k, w，算三者位数
A3	矩阵乘三种情形的 $Q(n)$	给 n, M, B，判断落在哪种情形
A4	分块最优块长	由 $3s^2 \le M$ 推 s
A5	递归分治的 $Q(n)$	画递归树，数叶子
A6	归并排序 $Q(n)$	同上
A7	work/span/parallelism	给 DAG 或代码，算 T_1, T_∞
A8	Master Method 三种 CASE	给递归式，判 CASE 并求解
A9	贪心调度上界	用 $T_1/P + T_\infty$ 估 T_P

#	内容	检验方式
A10	BFL 的空间界	复述 $O(M \lg M)$ 的证明
A11	Hoard 的 blowup	复述 $O(1 + SP/U)$ 的证明
A12	几何平均 vs 算术平均	给一组比值，算两种平均并说明哪个对
A13	位技巧	手写 `x&-x`、popcount、next_pow2、queens

B 级：必须能解释为什么

#	内容
B1	为什么 `i,k,j` 比 `j,k,i` 快 17 倍
B2	为什么并行外层循环比内层好
B3	为什么 base case 要粗化
B4	为什么 `-O3` 有时慢于 `-O2`
B5	为什么 `restrict` 能让循环从双版本变单版本
B6	为什么无符号溢出阻止向量化
B7	为什么 `sqrt()` 阻止 LICM
B8	为什么归约要先 `-ffast-math`
B9	为什么 fence 的代价“约等于一次 L2 访问”
B10	为什么无锁栈需要版本号
B11	为什么 Hoard 要“最满的非满超级块”
B12	为什么 cache-oblivious 不需要知道 M
B13	为什么 bottom-up BFS 在 frontier 大时更快
B14	为什么“无锁不一定比锁快”
B15	为什么 P4 不该用 NPS 当指标

C 级：必须能说出名字与结论

#	内容
C1	21 条 Bentley 规则
C2	四类 cache 缺失
C3	MSI 三状态

#	内容
C4	TSO 的 8 条规则（至少记住“store–load 可重排”）
C5	死锁三条件 + 锁序定理
C6	三种 GC 及各自缺陷
C7	三种并行分配策略及各自缺陷
C8	四个项目的主题与关键技术

27.2 自测题（20 题，附答案）

题目

Q1. 一台机器主频 3.0 GHz、2 个芯片、每芯片 8 核、每核每周期可做 2 次 FMA（每次 FMA 算 2 次浮点运算），峰值 GFLOPS 是多少?

Q2. $M = 64$ KB、$B = 64$ B、$k = 8$ 路、地址 48 位。set 索引占几位? tag 占几位?

Q3. 对 $n = 8192$ 的方阵乘法（行主序、朴素 `i,j,k`），$M = 32$ KB、$B = 64$ B。判断缺失复杂度属于哪种情形，并给出 $Q(n)$ 的量级。

Q4. 分块矩阵乘，$M = 256$ KB、$B = 64$ B。最优块长 s 大约多少? 此时 $Q(n)$ 是多少?

Q5. 递归分治矩阵乘的递归式是 $Q(n) = 8Q(n/2) + \Theta(1)$（$n^2 \geq cM$ 时），基线 $Q(\sqrt{cM}) = \Theta(M/B)$。求 $Q(n)$。

Q6. 二元归并排序的 $Q(n) = \Theta((n/B)\lg(n/M))$。若改用 $R = M/B$ 路归并，$Q(n)$ 变成多少? 省了多少倍?

Q7. 一段 Cilk 代码：`cilk_spawn f(); g(); cilk_sync;`，其中 $T_1(f) = 100$、$T_\infty(f) = 10$、$T_1(g) = 50$、$T_\infty(g) = 40$。求 T_1、T_∞、parallelism。

Q8. 上题在 8 核上的 T_P 下界是多少? 贪心调度上界是多少?

Q9. `T(n) = 2T(n/2) + n lg n` 用 Master Method 属于哪个 CASE? 解是什么?

Q10. 为什么 `x & -x` 能取出最低位的 1?

Q11. 写出 64 位 popcount 的 SWAR 四行核心代码，并说明复杂度。

Q12. N 皇后位向量解法中，`(left | place) << 1` 和 `(right | place) >> 1` 分别在做什么?

Q13. 下面这个循环为什么不向量化?

```
for (size_t i = 0; i < n; i++) a[i] = b[i+1];
```

Q14. 下面这个循环为什么不向量化? 怎么改?

```
for (size_t i = 0; i < n; i++) { if (b[i] > a[i]) a[i] = b[i]; }
```

Q15. 浮点归约 `y += x[i]` 在 `-O3` 下不向量化，为什么？加什么开关可以？有什么副作用？

Q16. BFL 分配器的虚拟内存上界为什么是 $O(M \lg M)$？

Q17. Hoard 的不变量 $u_i \geq \min(a_i - 2S, a_i/2)$ 中，两项各自对应什么浪费？

Q18. 无锁栈的 ABA 问题是什么？两种解法各是什么？

Q19. 什么是伪共享？举两种修法。

Q20. 一个程序串行部分占 20%，在 16 核上最大加速比是多少？若用 Gustafson 的观点重新表述，结论会怎样变化？

答案

A1. $3.0 \times 10^9 \times 2 \times 8 \times 4 = 192$ GFLOPS。（每核每周期 2 次 FMA × 每次 2 flops = 4 flops/cycle/core。）

A2. set 数 $= M/(kB) = 65536/(8 \times 64) = 128 = 2^7 \to$ **7 位**；offset $= \lg 64 = 6$ 位；tag $= 48 - 7 - 6 =$ **35 位**。

A3. 判据是 n 与 $M/B = 32768/64 = 512$ 的关系。$n = 8192 > 512$，属于**情形 1**：$Q(n) = \Theta(n^3)$——矩阵 B 的每次访问都缺失。

A4. $s \approx \sqrt{M}$，但要考虑三个子矩阵同时驻留：$3s^2 \leq M \to s \leq \sqrt{M/3} \approx \sqrt{262144/3} \approx 295$。实际取 2 的幂，约 **128–256**。此时

$$Q(n) = \Theta\left(\frac{n^3}{B\sqrt{M}}\right) = \Theta\left(\frac{n^3}{64 \times 512}\right) = \Theta\left(\frac{n^3}{32768}\right).$$

A5. 递归树每层 8 个节点，从 n 到 $\sqrt{cM}$ 共 $\lg(n/\sqrt{cM})$ 层，叶子数

$$8^{\lg(n/\sqrt{cM})} = \left(\frac{n}{\sqrt{cM}}\right)^3 = \frac{n^3}{M^{3/2}},$$

每叶 $\Theta(M/B)$，故

$$Q(n) = \Theta\left(\frac{n^3}{M^{3/2}} \cdot \frac{M}{B}\right) = \Theta\left(\frac{n^3}{B\sqrt{M}}\right).$$

A6. $Q_{\text{multiway}} = \Theta\left(\dfrac{n \lg n}{B \lg M}\right)$；相比二元归并的 $\Theta\left(\dfrac{n \lg n}{B}\right)$，省了 $\Theta(\lg M)$ 倍。

A7. work 相加：$T_1 = 100 + 50 = 150$。span 取 max（`f` 与 `g` 并行）：$T_\infty = \max(10, 40) = 40$。parallelism $= 150/40 = 3.75$。

A8. 下界 $\max(T_1/8, T_\infty) = \max(18.75, 40) = 40$；贪心上界 $T_1/8 + T_\infty = 18.75 + 40 = 58.75$。

A9. $a = 2, b = 2$，$n^{\log_b a} = n^1 = n$。$f(n) = n \lg n = \Theta(n^1 \lg^1 n)$，属于 CASE 2（$k = 1$），故

$$T(n) = \Theta(n^{\log_2 2} \lg^{1+1} n) = \Theta(n \lg^2 n).$$

A10. 设 x 的最低位 1 在第 k 位。则 $\sim x$ 的第 k 位为 0、第 $0..k-1$ 位全为 1；加 1 后低位全部进位、第 k 位变 1、更高位不变（因为第 k 位原本是 0，接收进位变 1）。于是 x 与 $-x$ 只在第 k 位同时为 1，其余位至

少一个为 0。

A11.

```
x =  x - ((x >> 1) & 0x5555555555555555ULL);
x = (x & 0x3333333333333333ULL) + ((x >> 2) & 0x3333333333333333ULL);
x = (x + (x >> 4)) & 0x0F0F0F0F0F0F0F0FULL;
x = (x * 0x0101010101010101ULL) >> 56;
```

复杂度 $\Theta(\lg w) = \Theta(6)$（对 64 位）。

A12. `place` 是这一行放的皇后所在的列（一个 1 位）。`(left | place) << 1`：把“左斜线攻击”整体左移一位，表示下一行的左斜线约束（因为左斜线在下一行会“右移一格”对应到列号 +1）。`(right | place) >> 1` 同理，右斜线在下一行对应列号 −1。

A13. 跨迭代依赖：第 i 次读 `b[i+1]`，而第 $i+1$ 次写 `a[i+1]`；若 `a` 与 `b` 可能重叠，向量化会改变语义。即使不重叠，这个访问模式也是“读比写超前一个”，需要额外的 shuffle。**修法**：加 `restrict` 并确认语义允许（但注意即使不重叠，`b[i+1]` 的读取本身是可向量化的，只是需要编译器证明无别名）。

A14. 因为 `if` 是控制流，编译器要生成分支。**修法**：改写成无副作用的条件表达式

```
a[i] = (b[i] > a[i]) ? b[i] : a[i];
```

这样 clang 能映射到 `pmaxub`（无符号字节）或 `maxpd`（浮点）。

A15. 浮点加法不满足结合律，重排会改变舍入结果，C 标准不允许编译器擅自重排。加 `-ffast-math`（或 `-fassociative-math`）可以。副作用：**数值结果可能不同**（尤其是在有 signed zero、NaN、或大动态范围求和时）。

A16. 一个 x 字节请求消耗 $2^{\lceil \lg x \rceil} \le 2x$ 的空间；所以每个大小类 2^k 的块总共占用至多 $2M$；共有 $\lg M$ 个大小类；相乘得 $O(M \lg M)$。

A17. $a_i - 2S$ 项对应“每个堆最多 $2S$ 的闲置超级块”（**外部碎片/预留**）；$a_i/2$ 项对应“分配量不超过使用量的 2 倍”（**内部碎片**）。

A18. ABA：线程 1 读到 `head = &A`，被换出；线程 2 把 A 弹出、又弹出 B、再 push 一个**复用 A 内存**的新节点；线程 1 恢复后 CAS 成功，但链表已被破坏。解法：**版本号**（指针 + 计数器打包进同一原子字）或**回收**（hazard pointer / epoch，未完成请求期间禁止复用节点）。

A19. 伪共享：两个线程写**同一条 cache line 上的不同变量**，导致该 line 在两者之间反复 invalidate。修法：**对齐 + 填充到 cache line 大小**；或**局部累加后归约**；或调整数据布局让热变量分离。

A20. Amdahl：$1/0.2 = 5$ 倍。Gustafson：若允许问题规模随核数增长，加速比 $S = \alpha + P(1-\alpha)$（α 为串行比例），16 核时 $= 0.2 + 16 \times 0.8 = 13$ 倍——**两者不矛盾**：Amdahl 固定问题规模，Gustafson 固定执行时间。

27.3 三条“如果只记三件事”

1. $T_P \ge \max(T_1/P, T_\infty)$——并行不是加核就行。

2. $Q(n) = \Theta(n^3/(B\sqrt{M}))$——分块/递归分治的理论依据。

3. $T = aI + bC$，且 $b/a \approx 90$——省一次 cache 缺失 ≈ 省 90 条指令。

第 28 章 参考来源

28.1 OCW 页面（2026-11 访问，全部 HTTP 200）

页面	URL
课程主页	https://ocw.mit.edu/courses/6-172-performance-engineering-of-software-systems-fall-2018/
Syllabus	https://ocw.mit.edu/courses/6-172-performance-engineering-of-software-systems-fall-2018/pages/syllabus/
Calendar	https://ocw.mit.edu/courses/6-172-performance-engineering-of-software-systems-fall-2018/pages/calendar/
Lecture Slides	https://ocw.mit.edu/courses/6-172-performance-engineering-of-software-systems-fall-2018/pages/lecture-slides/
Assignments	https://ocw.mit.edu/courses/6-172-performance-engineering-of-software-systems-fall-2018/pages/assignments/
Projects	https://ocw.mit.edu/courses/6-172-performance-engineering-of-software-systems-fall-2018/pages/projects/
Quizzes	https://ocw.mit.edu/courses/6-172-performance-engineering-of-software-systems-fall-2018/pages/quizzes/
Recitation Problems	https://ocw.mit.edu/courses/6-172-performance-engineering-of-software-systems-fall-2018/pages/recitation-problems/
Readings	https://ocw.mit.edu/courses/6-172-performance-engineering-of-software-systems-fall-2018/pages/readings/
Lecture Videos	https://ocw.mit.edu/courses/6-172-performance-engineering-of-software-systems-fall-2018/video_galleries/lecture-videos/

28.2 讲义 PDF（23 份，按讲次）

讲	文件（OCW 资源路径）	署名（PDF 元数据）
1	`.../d0c73dd51c79b95196a2e6faa824e1b4_MIT6_172F18_lec1.pdf`	Charles Leiserson

讲	文件（OCW 资源路径）	署名（PDF 元数据）
2	`.../1a57adbec9520270d4485b42a2e1a316_MIT6_172F18_lec2.pdf`	Julian Shun
3	`.../cc6983c9ebd77c28e8ae85bc0e575360_MIT6_172F18_lec3.pdf`	Julian Shun
4	`.../8955b91dbd47c241fb2904c700fdb697_MIT6_172F18_lec4.pdf`	Charles Leiserson
5	`.../b0d1c7f457920326cef5c958237ea743_MIT6_172F18_lec5.pdf`	Tao B. Schardl
6	`.../cbef71a2b92bc700fbad43e69f951513_MIT6_172F18_lec6.pdf`	Julian Shun
7	`.../543e419bff1ed05b1a649115c24c44b3_MIT6_172F18_lec7.pdf`	Julian Shun
8	`.../5655e50541f85421cc47b7c85d8e5de3_MIT6_172F18_lec8.pdf`	Charles Leiserson
9	`.../df628b3fffb046d18f9442f01a812ccf_MIT6_172F18_lec9.pdf`	Tao B. Schardl
10	`.../4d7f4bb31bf1ed90c669a11867d36d36_MIT6_172F18_lec10.pdf`	Charles Leiserson
11	`.../68624442da6047731a33b3590cefd030_MIT6_172F18_lec11.pdf`	Julian Shun
12	`.../24c247c943024dcda40bcc7caa206581_MIT6_172F18_lec12.pdf`	Julian Shun
13	`.../fe9b746636bea5357ea8269fe6463020_MIT6_172F18_lec13.pdf`	Tao B. Schardl
14	`.../329bfc6e1808c375afa517feb3c4c273_MIT6_172F18_lec14.pdf`	Julian Shun
15	`.../cef17369f91d3140409f2be4ad9246a4_MIT6_172F18_lec15.pdf`	Julian Shun
16	`.../e4fe97d2a250b57bb1e330fba8cf5992_MIT6_172F18_lec16.pdf`	Charles Leiserson
17	`.../fc3b846f6e13e15e4982a92369ca3825_MIT6_172F18_lec17.pdf`	Charles Leiserson
18	`.../6cd2359235c415ad55960be2dfc7e136_MIT6_172F18_lec18.pdf`	Saman Amarasinghe
19	`.../809f3351da7a2e5a6afc7c22e2e68e4d_MIT6_172F18_lec19.pdf`	Helen Xu
20	`.../076be084af69b7d645c30b1f337a3b08_MIT6_172F18_lec20.pdf`	Charles Leiserson
21	`.../dd989b76928a0561851e73da6d028763_MIT6_172F18_lec21.pdf`	Jon Bentley
22	`.../f58df934e205b870e473c2af3fe5e6f7_MIT6_172F18_lec22.pdf`	Julian Shun
23	`.../4dd5830c2bb4e4da2a51c5baae8accb8_MIT6_172F18_lec23.pdf`	Steven Johnson

（`...` 前缀均为 `https://ocw.mit.edu/courses/6-172-performance-engineering-of-software-systems-fall-2018/`。）

28.3 作业与项目 PDF

文档	文件
HW1 Getting Started	`.../2724d8594cb413754669fc4e9c6ce7db_MIT6_172F18hw1.pdf`
HW2 Profiling Serial Merge Sort	`.../796439e646c02f44348d50b1836ff7f9_MIT6_172F18hw2.pdf`
HW3 Vectorization	`.../072651a8229a63376d5720c9a500ae45_MIT6_172F18hw3.pdf`
HW4 Reducer Hyperobjects	`.../b2ff5659ae776c6fa6834967f73a2ab4_MIT6_172F18hw4.pdf`
HW5 Theory of Performance Engineering	`.../cd57dbdecbce3bd4da39d620c1e61d14_MIT6_172F18hw5.pdf`
HW6 Custom Memory Allocators	`.../e8e29570ad0673ec3460af46536a1e04_MIT6_172F18hw6.pdf`
HW7 Dynamic-Analysis Tools	`.../1ca096c3f8a039f3b782479c1deca8a4_MIT6_172F18hw7.pdf`
HW8 Cache-Oblivious Algorithms	`.../5bc7ce0d33656fd3783abcdc3dc4eb92_MIT6_172F18hw8.pdf`
HW9 Deterministic Execution	`.../ddcc1a4046b1893ca38211dc54be44fc_MIT6_172F18hw9.pdf`
HW10 Data Synchronization	`.../64f4db5020ca72ab205e458d40bbdfe3_MIT6_172F18hw10.pdf`
Project 1 Bit Hacks	`.../271aca8dd9fe0ff159ae31396a180f17_MIT6_172F18_project1.pdf`
Project 2 Collision Detection	`.../ac1794b01bec7e14b4fd34d5b1a7d2aa_MIT6_172F18_project2.pdf`
Project 3 Serial Dynamic Memory Allocation	`.../d2fb8ee337632ab3e256fa1704375053_MIT6_172F18_project3.pdf`
Project 4 Leiserchess	`.../b13a935db7ea2a78a16582065bba55be_MIT6_172F18_project4.pdf`
Leiserchess 规则	`.../6f1ff098e89d9c6f1e639de093fd9495_MIT6_172F18_leiserchess.pdf`
Practice Quiz 1 （+Solutions）	`.../6dd24f1d927b16e1e0eec707a44cf6a4_MIT6_172F18_practicequiz1.pdf`
Practice Quiz 2 （+Solutions）	`/resources/practice-quiz-2/`、`/resources/practice-quiz-2-solutions/`
Practice Quiz 3	`/resources/practice-quiz-3/`（无答案）

文档	文件
Practice Quiz 4（+Solutions）	`/resources/practice-quiz-4/`、`/resources/practice-quiz-4-solutions/`

28.4 补充阅读（OCW Readings 页列出的 14 条）

1. Zhang, Yang, et al. “GraphIt: A High-Performance Graph DSL.” *PACMPL* 2 (2018): article 121.
2. Ansel, Kamil, et al. “OpenTuner: An Extensible Framework for Program Autotuning.” *PACT* (2014): 303—316.
3. Leiserson. “The Cilk++ Concurrency Platform.” *J. Supercomputing* 51, no. 3 (2010): 244—257.
4. Leiserson & Mirman. “How to Survive the Multicore Software Revolution.”
5. Frigo, Leiserson, Randall. “The Implementation of the Cilk-5 Multithreaded Language.” *PLDI* (1998).
6. He, Leiserson, Leiserson. “The Cilkview Scalability Analyzer.” *SPAA* (2010): 145—156.
7. Mytkowicz, Diwan, Hauswirth, Sweeney. “Producing Wrong Data Without Doing Anything Obviously Wrong.” *ASPLOS* (2009): 265—276.
8. Berger, McKinley, et al. “Hoard: A Scalable Memory Allocator for Multithreaded Applications.” *ASPLOS* (2000): 117—128.
9. Kuszmaul. “SuperMalloc: A Super Fast Multithreaded Malloc for 64-bit Machines.” *ISMM* (2015): 41—55.
10. Schardl, Moses, Leiserson. “Tapir: Embedding Fork-Join Parallelism into LLVM's IR.” *PPoPP* (2017): 249—265.
11. Frigo, Leiserson, et al. “Cache-Oblivious Algorithms.” *ACM TALG* 8, no. 1 (2012): article 4.
12. Demaine. “Cache-Oblivious Algorithms and Data Structures.” EEF Summer School (2002).
13. Leiserson. “A Simple Deterministic Algorithm for Guaranteeing the Forward Progress of Transactions.” *Information Systems* 57 (2016): 69—74.
14. Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., 第 27 章（多线程算法）。MIT Press, 2009. ISBN 9780262033848.

28.5 两本常被当作伴读的书（非 OCW 指定教材）

书	与本课程的关系
Bryant & O'Hallaron, *Computer Systems: A Programmer's Perspective*（CSAPP）	第 4 章（处理器体系结构）、第 5 章（性能优化）、第 6 章（存储层次）与本课程 L4/L10/L14 高度重叠，但抽象层不同

书	与本课程的关系
Herlihy & Shavit, *The Art of Multiprocessor Programming*	与 L16/L17 对应，讲得更系统（尤其是无锁数据结构与内存模型的理论部分）

28.6 事实核查说明

本文中以下类别的事实**严格来自上述材料**，未做推测：

- 评分比例、项目分值表、作业清单、讲次标题、Quiz 结构与分值分布；
- 运行时间、加速比、GFLOPS、cache 缺失数、分配器吞吐、GFLOPS 峰值公式；
- 定理陈述（LRU 引理、Cache-Miss 引理、子矩阵引理、BFL 定理、Hoard blowup、贪心界、work/span 定律、Burns–Lynch、Attiya）；
- 编译器开关、Makefile 目标、命令行参数、文件名。

以下内容标注为“推断”或“补充”，**不声称来自课程材料**：

- Haswell 的 IPC 估算、L1 延迟换算成拍数；
- MESI 相对 MSI 的 E 状态（讲义只讲 MSI）；
- Gustafson 定律的表述；
- release/acquire 的语义解释；
- 各项目的“实现顺序建议”；
- TSP 的“n=14 时 133 倍”等由表中数字相除得到的派生值。

以下位置的材料存在**已知的抽取困难**，已在正文标注“以讲义为准”：

- L3 的存储层次拍数表（PDF 字体编码导致数字不可靠）；
- L14 的 LLC/L2 延迟数值（讲义表格中该列未完整抽取）；
- L1 关于“cache 引用减少 62% / 缺失减少 68%”的表述与表中 LLC 列（8600→416）口径不一致，正文按原文照抄并标注存疑；
- L9 的“240 instructions”思想实验（推断应为 2^{40}，因为按 2^{40} 计算才得到讲义所说的 32 秒）。

文档结束。配套的实战手册见同目录 `6172_性能工程_项目上手速查.md`。